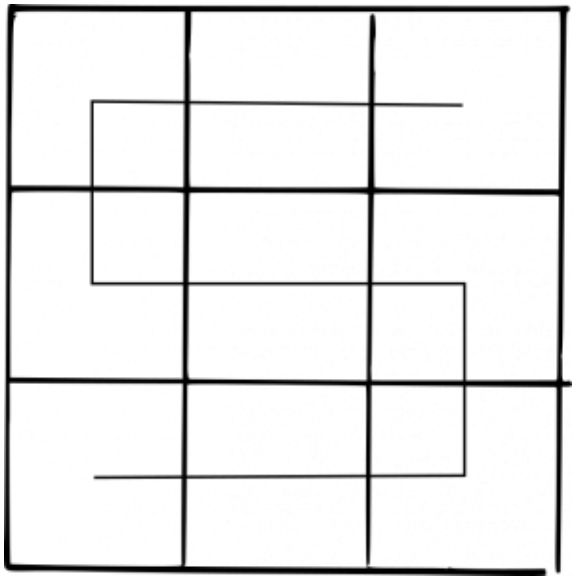


Peano Documentation

Peano Version 4 (Peano 4/P4)

www.peano-framework.org

Dr. rer. nat. habil. Tobias Weinzierl

May 14, 2022

Chapter 1

Preamble

Peano 4 is an open source framework for solvers on dynamically adaptive Cartesian meshes. Its core is built with C++, but many tools around it are written in Python. Peano 4 is based upon the fact that spacetrees, a generalisation of the classical octree concept, yield a cascade of adaptive Cartesian grids. Consequently, any spacetree traversal is equivalent to an element-wise traversal of the hierarchy of the adaptive Cartesian grids. The software Peano 4 realises such a grid traversal and storage algorithm, and it provides hook-in points for applications performing per-element, per-vertex, and so forth operations on the grid. It also provides interfaces for dynamic load balancing, sophisticated geometry representations, and other features. Some properties are enlisted below.

Peano is currently available in its fourth generation. The development of the original set of Peano codes started around 2002. 2005-2009, we merged these codes into one Peano kernel (2nd generation). In 2009, I started a complete reimplementation of the kernel with special emphasis on reusability, application-independent design and the support for rapid prototyping. This third generation of the code ended around 2019 when we released the ExaHyPE code—a hyperbolic equation system solver engine which uses Peano's AMR meshes. The current guidebook discusses Peano 4 , which reuses ideas and lessons learned from Peano 3 as well as lots of code buildling blocks, but can be seen as a major rewrite starting from scratch.

Peano 4 has been the code base for multiple projects over the past years. The most prominent one is likely ExaHyPE. The first generation of ExaHyPE has been a stand-alone projects using Peano as one required library or framework out of many. The second generation, ExaHyPE 2, is merged into Peano 4as spin-off. This document therefore covers ExaHyPE 2, too.

Dependencies and prerequisites

Peano 4's core is plain C++17 code. We however use a whole set of tools around it:

- GNU autotools (automake) to set up the system (required).
- C++17-compatible C++ compiler (required).
- Python3 (optional; not required if you work only with the C++ baseline).
- MPI2 (optional). MPI's multithreaded support is required.
- Intel's Threading Building Blocks (TBB) or OpenMP 5. We also can work with C++ threads, but found their performance suboptimal. TBB is not shipped as part of OneAPI.
- The Visualization Toolkit (VTK) if you want to use the built-in visualisation facilities (optional).

- Doxygen if you want to create HTML pages of PDFs of the in-code documentation.



I test and maintain Peano 4 for Linux only. If you prefer Windows or MacOS, it should work as long as you provide the mandatory tools from above, but I won't be able to help.

Software	Version	Remarks
GNU g++	7.5 or older	Works with Peano 4 core but not with some extensions such as ExaHyPE 2.
GNU g++	9.3.1	Current development version. Works.
Intel icpc	19.1	Works with Peano 4 and all extensions such as ExaHyPE 2. Tests with Intel Fortran have been successful, too.
VTK	7–9	Supported by Peano 4. Default is 8, i.e. if you need other version, you have to inform configure (see Chapter A).
Python	3	Development version. Python 2 is not supported by frontend.

Who should read this document

This LaTeX documentation is written similar to a tutorial in a hands-on style. Therefore, it also contains lots of source code snippets. If you read through a chapter, you should immediately be able to re-program the presented details in your code and use the ideas.

Therefore, this LaTeX documentation is written for people that have a decent programming background as well as scientific computing knowledge. It is a LaTeX documentation that shall help readers to translate their scientific vision into a working code quickly.

How the text is organised

If you feel something is urgently missing and deserves a chapter or things remain unclear, please write me an email and I'll see whether I can provide some additional text or extend the LaTeX documentation.

May 14, 2022
Tobias Weinzierl

Contents

1 Preamble **i**

2 Installation **1**

2.1 Prepare the configure environment 1

2.2 Configure 2

2.3 Build 3

2.4 Installation test 4

2.5 Python configuration 4

2.5.1 Python paths 4

2.5.2 Python packages 5

2.6 Fortran 5

2.7 Peano 4 components and build variants 5

2.8 Documentation 6

2.9 Compile/link options for applications built with Peano 4 7

3 Peano 4's architecture **9**

I Large-scale Peano 4 applications **11**

4 ExaHyPE 2 **13**

4.1 Historical remarks 14

4.2 Case study: minimalist Finite Volumes for the Euler equations 15

4.3 Case study: A Finite Volumes code coupling two solvers (hyperbolic+elliptic) . . . 18

4.4 Generic ExaHyPE 2 features 23

4.4.1 IO, logging and initialisation 23

4.4.2 Running on a parallel computer 25

4.4.3 Adding adaptive mesh refinement (AMR) 27

4.4.4 GPU support 29

4.5 Finite Volume extensions 31

4.5.1 Boundary conditions . . . 31
4.5.2 Interpolation and restriction schemes . . . 33
4.6 Modelling the fluxes and Riemann solvers with SymPy . . . 35
4.7 Adding tracers . . . 36
4.7.1 Dump tracer data into CSV database . . . 37
4.7.2 Write your own particle seed . . . 37
4.7.3 Encourage particles to spread out evenly . . . 38
4.7.4 Changing the grid-to-particle interpolation and/or time stepping . . . 38

5 SWIFT.pidt **41**
5.1 Set up the simulator . . . 41
5.2 Introduce a new particle . . . 42
5.3 IO control . . . 42
5.4 High-performance computing . . . 42

II Examples: Using the high-level Python API **45**

6 Introduction **47**
6.0.1 Parallelisation and GPU support . . . 47

7 A block-structured AMR code for Finite Volumes **49**
7.1 Setting up the data structures . . . 49
7.2 Administration of the data structures . . . 50
7.3 A manually implemented FV scheme . . . 51
7.4 Plot results . . . 53

8 Peano 4's particle support **55**
8.1 Introduce a new particle . . . 55
8.2 Insert particles into domain . . . 56
8.3 Dual trees . . . 57
8.4 Move particles . . . 58
8.5 Dual tree traversal and particle-particle interaction . . . 59
8.6 Further action sets . . . 60

9 A geometric-algebraic multigrid solver **61**
9.1 A matrix-free Jacobi smoother with rediscretisation . . . 61
9.2 A parallel version of the code . . . 61
9.3 A parallel AMR Jacobi solver . . . 62

III Low level (C/C++) programming **63**

10 Guiding principles, high-level concepts and design decisions **67**

10.1 Generic structure of a Peano 4 application . . . 67

10.2 MPI and shared memory programming . . . 68

10.2.1 Realising a domain decomposition . . . 69

10.2.2 Global communication . . . 70

11 Logging, tracing and assertions **71**

11.1 Logging, tracing and assertion architecture . . . 71

11.2 Using the logging and tracing interface . . . 71

11.3 Logging devices (formats) . . . 72

11.3.1 CommandLineLogger . . . 72

11.3.2 ChromeTraceFileLogger . . . 73

11.3.3 NVTX logger (NVIDIA SDK) . . . 73

11.3.4 ITAC logger . . . 73

11.4 Log filter . . . 74

11.5 Assertions . . . 75

12 Unit tests **77**

12.1 Test architecture . . . 77

12.2 Running the tests . . . 78

12.3 Test routines . . . 78

13 Parallel programming **81**

13.1 Shared memory . . . 81

13.1.1 Protecting shared memory access . . . 81

13.2 Distributed memory . . . 81

13.2.1 Synchronisation . . . 82

13.2.2 Exchange global data between ranks (reduction) . . . 82

14 Tools **83**

14.1 Loop macros . . . 83

14.1.1 d-dimensional loops . . . 83

14.1.2 d-dimensional exclusive loops . . . 83

IV Extending the Python API **85**

15 Introducing a new pre-defined action set **87**

15.1 Prepare classes . . . 87
15.2 Implement new action set . . . 87
15.3 Use your new action set . . . 88
15.4 Conventions . . . 88

V Design decisions 89

16 Domain decomposition and data flow 91
16.1 Multigrid topology . . . 91
16.2 Data flow on stationary domains . . . 92
16.2.1 Horizontal data flow . . . 92
16.2.2 Vertical data flow . . . 93
16.2.3 Synchronous vertical data flow . . . 93
16.3 Splits . . . 93
16.3.1 Temporal ordering of tree traversals . . . 93
16.3.2 User data . . . 94
16.3.3 Data ownership . . . 94
16.4 Joins . . . 94
16.4.1 Temporal ordering . . . 95
16.4.2 Data ownership . . . 95
16.4.3 Grid data streaming . . . 96
16.4.4 User data streaming . . . 96

VI Standard benchmarks 97

17 Benchmark: Task priorities 99
17.1 Preparation . . . 99
17.2 Set up the experiment . . . 99
17.3 Run the code . . . 101
17.4 Visualise the outcome . . . 101
17.4.1 What we should see . . . 102
17.4.2 Data flow spec . . . 102
17.4.3 Code regions causing the behaviour . . . 103
17.5 Flaws identified . . . 103
17.5.1 OpenMP: task group . . . 103
17.5.2 OpenMP: task group . . . 103

VII Appendix **105**

A Postprocessing: The visualisation toolkit, IO and Peano 4 scripts **107**

A.1 Displaying Peano 4 files directly within Paraview 107

A.2 Offline/command line conversion with Python/Paraview 109

A.3 Offline/command-line conversion (executable) 110

A.4 Writing Legacy VTK files 111

A.5 Probing data 112

A.6 Tweaking your pictures 112

A.7 Useful scripts 113

B Peano's block-structured output format **115**

B.1 A minimal ASCII example file 115

B.2 More sophiciated dumps 116

C Selected HPC platforms and toolchains **117**

C.1 Clang toolchain 117

C.2 Intel OneAPI toolchain 117

C.3 NVIDIA NVC++ toolchain 119

C.4 Some particular supercomputer testbeds 119

C.4.1 Hamilton 8 119

C.4.2 DINE (Durham Intelligent NIC Environment) 120

C.4.3 Cosma 120

C.4.4 Bede 121

D Configuring Peano 4 in Eclipse **123**

D.1 Clone directly from git 123

D.2 Integrating the C++ core as project 123

VIII Quick help **125**

E FAQs (programmers' point of view) **127**

E.1 C++ core 127

E.2 Parallelisation 127

F Troubleshooting **129**

F.1 Python API 129

F.2 Compilation 129

F.3 Linking 130

F.4 Debugging 131
F.5 MPI troubleshooting 132
F.6 OpenMP troubleshooting 132
F.7 External tools 133
F.8 PDT (legacy) 133

Chapter 2

Installation

There are two ways to obtain Peano 4: You can either download one of the archives we provide on the webpage, or you can work directly against a repository clone (recommended). If you work with the archive, type in a

```
tar -xzvf myarchive.tar.gz
```

in the directory where you've stored your downloaded file.

If you work with the git archive, you have to clone this archive first. I grant access to Peano 4 free of charge and the repository is public. However, I ask users to sign up for the software if they indend to push modifications to the code (which I very much appreciate[1]). This way I can report to funding agencies how frequent the software is used, and I also have at least some ideas which application areas benefit from the software and where it is actively used and developed. If you do not indend to modify the Peano 4 core code base, you can just clone the code anomymously.

```
git clone https://gitlab.lrz.de/hpcsoftware/Peano.git
cd Peano
```



I still maintain the "old" Peano in the repository (version 3), and most users consider this to be the standard Peano generation. For the present document, it is thus important that you manually switch to the branch `p4`.

2.1 Prepare the configure environment

This step should only required if you work directly against the git repository. If you prefer to download a snapshot of Peano 4, then you can skip this section and continue with 2.2.

- Ensure you have the autotool packages installed on your system. They typically are shipped in the packages `autoconf`, `automake` and `libtool`.
- Set up the configure environment:

```
libtoolize; aclocal; autoconf; autoheader;
cp src/config.h.in .;
automake --add-missing
```

[1] Peano 4's guidebook (the file you currently read) is hosted within Peano 4's git repository, too. I'm always happy if people add content to this documentation, too.

These steps should only be required once, unless you push major revisions to the development branch.

2.2 Configure

Peano 4 relies on the autotools to set up its build environment. Change into the project's directory and type in

```
./configure --help
```

The `--help` option should give you some information about the available variants of Peano 4. A sole `./configure` call gives you the minimalist version of Peano 4—might be appropriate for a first test.



I recommend that you start a first test without any additional flavours of Peano 4, i.e. to work with a plain `./configure` call. Once the tests pass, I recommend that you add further options (IO) and/or parallelisation. Please note that the baseline config is not sufficient for most applications as it is merely a stripped down version of the Peano 4 core. ExaHyPE 2 for example requires you to add `--enable-exahype` and `--enable-loadbalancing-toolbox`. But the bare/minimal version is good for first tests.

The output of `./configure --help` should be reasonable verbose, but I summarise some of the most imporant options below:

`--with-multithreading`	Switch on multithreading. By default, we build without multithreading, but a `--with-multithreading=omp`, e.g., makes Peano 4 use the OpenMP threading model. Please consult `--help` for details. Note: To use OpenMP multithreading, you will need a compiler which supports OpenMP 5.0 (e.g - GCC 9.x)[2].
`--with-mpi`	Enable the MPI version of Peano 4. You have to tell the build environment however which compile command to use. Please note that we need a C++ MPI wrapper. So `--with-mpi=mpcxx` is a typical call. Currently, you also need `--with-multithreading` if you want to use MPI.
`--with-toolchain`	Build against a particular vendor toolchain. Currently, I have dedicated support for the NVIDIA NVTX SDK (`--with-toolchain=nvidia`) providing NVIDIA-specific logging/tracing as well as memory allocation calls—the latter is of use if you use GPUs—and the Intel toolchain (`--with-toolchain=intel`). The latter in particular links against the ITAC tracing API. I also have support for Google's Chrome browser (`--with-toolchain=chrome`) which means that traces are written in a way that you can display it in Chrome. Ensure that your CXXFLAGS set the right include path[3]. This is optional.

[2]Peano 4 realises its GPGPU offloading via OpenMP, too. Some compilers struggle to translate these GPGPU instructions. In this case, please configure with the CXXFLAG `-DnoGPUOffloading`.

[3]It is `CXXFLAGS+=-I/opt/nvidia/nsight-systems/2020.3.1/target-linux-x64/nvtx/include/nvtx3/` on my system, e.g.

`--with-gpu`	Add GPU support. The code base currently is prepared to support GPUs via OpenMP, SYCL, C++ tasking and HIP. Not all versions are equally mature. In principle, you can combine different GPU programming approaches with different multithreading techniques, but our recommendation is that you use C++ threading with C++ GPU offloading, OpenMP threading with OpenMP offloading, and OneTBB threading with SYCL.
`--enable-XXX`	A lot of frameworks or engines built on top of Peano 4 have to be enabled explicitly. ExaHyPE 2 is probably the most popular engine shipped with the code, but there is also a particle/SPH module or some linear algebra stuff. Run with `--help` for a list of toolboxes. Some toolboxes have further internal and external dependencies, and the translation of the toolboxes can be time consuming. By default, Peano 4 thus disables all of its toolboxes.

Compilers, linkers and both compiler and linker flags can be changed by resetting the corresponding environment variables (such as `CXX` or `LDFLAGS` or `CCFLAGS`) *prior* to the configure call. Alternatively, you can pass the variables to be used to `configure` through arguments (`./configure CXX=icpc ...`, e.g.). Please consult the `--help` output for details. For some supercomputers that we use frequently, there are recommendations which setting to use (Chapter C).

R Most Linux tools prefer you to specify a dedicated installation directory via `--prefix`. For Peano 4, things are slightly different: Peano 4 is a mere engine, i.e. not "useful" per se. It is applications built on top of Peano 4 that become tools that you might later want to install/deploy on your system. Therefore, I ship Peano 4 with static libraries only, and most users work with the code in the download/installation directory until they have constructed the application with the help of Peano 4. So no need to specify a proper installation directory via `--prefix`.

2.3 Build

Once the configuration has been successful, a simple

```
make
```

should build the Peano 4 core. It also builds some examples and all the extensions and toolboxes you have manually enabled. All of these go into static libraries. The code is not small and you thus might prefer

```
make -jx
```

with x being the number of parallel builds to use.

R Following up on the remark above,

```
make install
```

would deploy the Peano 4 files in the directory specified via the `prefix` before. Most Unix tools want you to install them. For Peano 4, things might be slightly different as it is an engine used to construct tools in the first place.

2.4 Installation test

Once you have compiled Peano 4 I recommend that you run all tests using

```
make check
```

The call launches all of Peano 4's unit test, i.e. all the unit tests for the core plus the extensions that you have enabled.

R I have written the test target such that it works out-of-the-box for non-MPI calls. If you have configured with `--with-mpi`, then the unit tests will fail. But this does not mean that the unit tests are broken. It is simply due to the fact that the makefile's test target does not know that you have to use the `mpirun` prefix. To make a long story short: If you use MPI and you want to run the unit tests, you have to do so manually.

To run 2D and 3D tests individually, use

```
src/unittests/UnitTests2d
src/unittests/UnitTests3d
```

You can run these builds with different core counts and also MPI support if you have compiled with MPI. The executables contain both node correctness tests and MPI ping-pong tests, i.e. you can both assess a valid build plus a working MPI environment. If successful, you should always get a message alike

```
Peano 4 (C) www.peano-framework.org
build: 2d, with mpi (1 ranks), omp (4 threads), debug level=4
it is recommended that you run the unit tests with more than one MPI rank, too (file:main.cpp,line:82)
usage: ././UnitTests2d core-count (use default core count=1 now) (file:main.cpp,line:88)
running test case collection "tarch.la.tests" ..... ok
running test case collection "tarch.la" ..... ok
running test case collection "tarch.mpi.tests" ..... ok
running test case collection "tarch.mpi" ... ok
running test case collection "tarch" ... ok
running test case collection ... ok
running test case collection "peano4.grid.tests" ..... ok
running test case collection "peano4.grid" ... ok
running test case collection "peano4.parallel.tests" ..... ok
running test case collection "peano4.parallel" ... ok
running test case collection "peano4" ... ok
running test case collection ... ok
```

Watch out for the last line that tells you whether the overall unit tests passed through.

2.5 Python configuration

Peano 4 is developed with Python 3. Python 2 is not supported.

2.5.1 Python paths

To make any Python example pass, you have to set the Python paths. There are multiple ways to do so.

The standard way is to set the `PYTHONPATH`:

```
export PYTHONPATH=myPeanoDirectory/python
```

This can even be done in the `.bashrc`.

Alternatively, you can augment your Python scripts later on with

```
import sys
PEANO4_PYTHON_DIR="myPeanoDirectory/python"
sys.path.append(PEANO4_PYTHON_DIR)
```

Please note that most Python scripts require you to set the `LD_LIBRARY_PATH`. If you install Peano 4, then this path should be automatically right. If you work with a local Peano 4 installation, you can either tell Peano 4 through its API where to search for the library (all routines that add a library accept additional search paths), or you can again use the `sys.path.append` command to extend `LD_LIBRARY_PATH`.

R I personally prefer the variant to go through the environment variable. Peano 4's Python API yield plain old makefiles building executables that work completely without any Python. This way, we ensure that we work smoothly on supercomputers, too, where Python sometimes is not available in the latest version or on the compute nodes. As long as you run through environment variables or the Peano 4 routines to specify paths, your makefiles also will continue to work that way.

2.5.2 Python packages

Peano 4 currently relies on the following Python packages/projects:

Package	Description	Ubuntu	OpenSUSE
jinja2	For all the templated glue code.	`python3-jinja2`	
sympy	For the symbolic modelling of partial differential equations (predominantly in ExaHyPE 2).	`python3-sympy`	`python3-sympy`

The table above specifies the corresponding packages in our two main distributions for which we test Peano 4. They can be installed via `zypper install` (SUSE) or `apt-get install` (Ubuntu), respectively. Alternatively, you might want to use Python's `pip`.

2.6 Fortran

R If you don't use Fortran in your own code, ignore this section.

Peano 4's core does not use any Fortran at all. However, many users use Fortran for their domain-specific programming. If you want to have a seemless integration of your particular Fortran choice through Peano 4's Python API, invoke `./configure` ensuring that the Fortran variables—in particular the environment variable `FC` identifying the compiler—are properly set.

For many codes and the GNU Fortran compiler, you need the flag `-fdefault-real-8`. You can export `FCFLAGS` and make it contain this argument before you invoke `configure`. As the Peano 4's core does not use Fortran—it is only applications built on top of Peano 4—you can redefine the flags later on (see 2.9).

2.7 Peano 4 components and build variants

Peano 4 currently is delivered as a set of archives, i.e. static libraries:

- There is a technical architecture (`Tarch`) and the actual `Peano4Core`.
- Each archive variant is available as a release version, as a debug version, as a version with tracing and assertions and as a tracing-only version.
- Each archive variant is available as 2d build and as 3d build. If you need higher dimensions, you have to build the required libraries manually.

The version terminology is as follows:

- **debug** The debug versions of the archives all have the postfix `_debug`. If you link against these versions, the full set of assertions, of all tracing and all debug messages is available; though you can always filter on the application-level which information you want to see (cmp. Chapter 11).
- **asserts** Thsee versions of the archives all have the postfix `_asserts`. If you link against these versions, all assertions are on. The code also contains tracing macros (see below).
- **tracing** The release versions of the archives all have the postfix `_trace`. If you link against these versions, all assertions and debug messages are removed, but some tracing is active. You can switch on/off the tracing per class (cmp. Chapter 11), and different tracing backends allow you to connect to different (profiling) tools.
- **release** The release versions of the archives have no particular postfix. They disable tracing, any debugging and all assertions. These archives should be used by production runs.



Peano's release components still write info messages (they suppress debug, tracing and logging, but not essential information). If you want to filter out these messages, too, you have to apply a log filter (Chapter 11.4).

Besides these archives, the Peano 4 installation also comes along with a set of example applications. They are found in the directory `src/examples`. Most examples create by default two variants of the example: a debug version and one with only tracing enabled. Several examples furthermore come along as 2d and 3d build.

2.8 Documentation

There are three major types/resources of documentation of the software:

1. This guidebook/LaTeX documentation that describes how to use the code base from a high abstraction level and with anecdotal examples.
2. The documentation of the C++ code. Here, I follow an "everything is in the code" philosophy.
3. The documentation of the Python code. Here, I follow an "everything is in the code" philosophy.

For the code documentation, "everything is in the code" means that all documentation is comments within the Python script or C++ header files, respectively. You can create a webpage from this distributed information through the tool `doxygen`[4].

There are two Doxygen configuration files in the repository, i.e. I keep the Python and the C++ documentation output separate. To create the documentation, switch into directory `src` or `python`, respectively. In both directories, the Doxygen config file is called `peano.doxygen-configuration`, i.e. calling

[4] `http://www.doxygen.nl`.

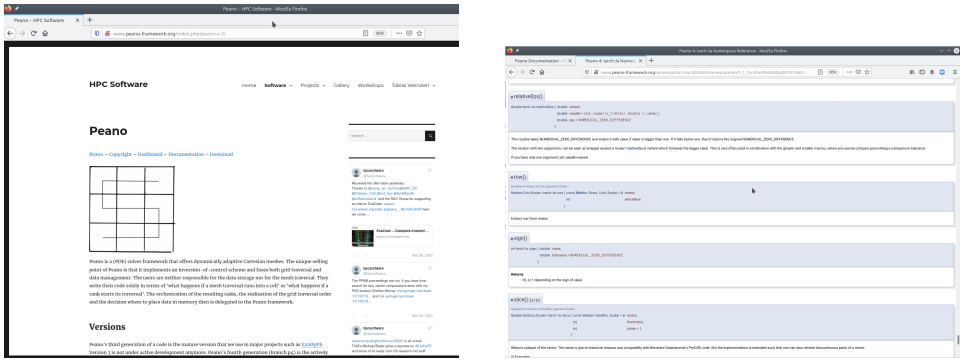


Figure 2.1: Peano 4's webpage (left) and a screenshot from the auto-generated source code docu which can be reached through the webpage if you don't want to generate the pages yourself. This documentation also provides indices and a search function as well as all documentation formulae typeset with LaTeX.

```
doxygen peano.doxygen-configuration
```

gives you the output. If you prefer not to generate and maintain the documentation yourself, the Peano 4 webpage hosts the autogenerated documentation, too. It is updated roughly once a week.

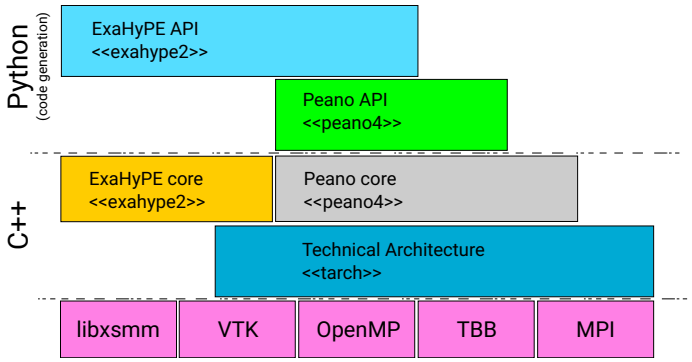
2.9 Compile/link options for applications built with Peano 4

Peano 4 is a framework, i.e. ships very few executables (mainly only for test purposes and for postprocessing). Its core delivery is a set of libraries. Application codes then build on top of these libraries. While it is up to you to decide which build system you use in your application, Peano 4 natively favours makefiles.

If you use Peano 4's Python API, it will create dummy makefiles. Higher-level Peano 4 applicaitons such as ExaHyPE 2 build upon the Python API and thus work with makefiles, too. When they create these makefiles, they parse your local Peano 4 installation's makefiles and extract the information from there. This is, when you've selected a particular compiler or compiler option when you install Peano 4, this choice will automatically propagate through to your domain-specific codes. There's however the option to overwrite the chosen settings—either by defining appropriate environment variables before you invoke `make` or by resetting variables manually within your Python scripts that configure the environment.

Chapter 3

Peano 4's architecture



Peano 4 is organised into layers, and only the bottom two layers are found in all Peano 4 applications. I discuss the layers bottom-up:

tarch The technical architecture is a set of loosly coupled building blocks that hide away technical details. There's for example a namespace to handle multicore parallelism, a logging component, or a collection of primitive linear algebra routines for small problems. It is written solely in C++ and its purpose is to hide away third-party dependencies such as OpenMP or MPI. I originally designed it as a stand-alone thing which is not tied to Peano though I'm not aware of any other project that currently uses it. The layer is pronounced "T Arch" and is encapsulated in a namespace `tarch`, as I didn't want to write the full name "technical architecture" every time.

peano4 The namespace `peano4` hosts the Peano 4 core. It is entirely written in C++ (using the tarch) and holds the heart and brain of the code, i.e. all the data management, the tree traversals and the domain decomposition.

Native Peano 4 applications use solely the core and the tarch. I write such codes from time to time myself, but it is a tedious task as all the user data management and management of "which routine is used when" has to be programmed manually.

Other projects can obviously supplement the core and tarch with further routines using either of the two components. The most prominent project extending the core is maybe ExaHyPE 2. The illustration here is not 100% correct as noone forces developers to write these extensions in C++. In practice however, most of them are (with small Fortran imports, e.g.).

Native projects can use the core extensions using C++. Most projects (like ExaHyPE) however do all offer a more abstract interface (see below).

Python API Peano 4 offers a Python API which is basically a code generator yielding lots of C++ snippets that plug into the core and use the tarch. Some of my examples use this Python API only.

API extensions are also offered by other projects such as ExaHyPE or ExaClaw. They often supplement the core Python API with further classes.

Organisation of guidebook

The guidebook is split into parts anticipating the layered architecture of the code.

One part discusses the C++ features offered both by the tarch and Peano 4's core. This part is very useful for anybody who writes C++ code within Peano 4, and it also gives information how Peano 4 works under the hood. Some developers might be able to work completely without the details described in this part. However, as soon as you have some linear algebra operations for example, you might want to be familiar with the core features.

One part gives some examples how to work with the Python API. Most people who use this API won't be able to stick to Python only, but: The API is powerful mechanism generating C++ glue code and C++ stubs. The latter still have to be befilled with actual computations by the user. So the API takes away the responsibility to assemble the application, but it does not completely hide away C++.

One part discusses how to use big projects built on top of Peano 4 that have a very high abstraction level. ExaHyPE 2/ExaClaw are the two biggest fishes at the moment. If you work on this level, you can likely completely ignore the other Python details and a lot of the C++ low level details for quite a while.

Another part finally discusses how to extend the Python API. This is relevant for all the developers that do not only want to create a code with Peano 4 but want to make their stuff easily available to others. After that, I spend a few pages to summarise design decisions (rationale) underrlying the code. This might help to understand why certain things work the way they do.

An appendix gives further information on particular machines, file formats, and so forth. It hosts an FAQ and tips how to combine/use Peano 4 with other tools.

Part I

Large-scale Peano 4 applications

Chapter 4

ExaHyPE 2







The original ExaHyPE project delivering the first generation of the ExaHyPE software has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 671698 (ExaHyPE). ExaHyPE 2 has been made possible by EPSRC under the grant EP/V00154X/1 (ExaClaw).

ExaHyPE 2 is both a set of C/C++ and Fortran core routines complementing the Peano 4 core and a high-level augmentation of Peano 4's Python API. ExaHyPE 2's Python API can be read as builder mechanism: You configure your particular ExaHyPE 2 application, i.e. you tell the ExaHyPE 2 engine which application you want to run (which PDE terms, which solver variants, which hardware properties). The Python API then yields both a Peano 4 Python project which you can run plus a set of template classes which you implement with your particular application.

The present document discusses how to create an ExaHyPE 2 application from scratch. Most application experts might not want to do this and instead use a read-to-use ExaHyPE 2 application. The most prominent (and maybe powerful) code using the ExaHyPE engine is maybe ExaSeis; used to simulate seismic waves. The details of these domain-specific codes are not covered by the present document.

A first ExaHyPE 2 run

- Clone the repository (cmp. page 1). Skip this step if you already have a Peano 4 checkout, i.e. if you have completed this step once.
- Create the build system (cmp. page 2). Skip this step if you already have a working configure, i.e. if you have completed this step once.

```
libtoolize; aclocal; autoconf; autoheader; cp src/config.h.in .; automake --add-missing
```

- Configure Peano 4 with the flags `--enable-exahype`, `--enable-loadbalancing`, `--enable-blockstructured`:

```
./configure --enable-exahype --enable-loadbalancing --enable-blockstructured
```

These are the mandatory extensions. You might want to add further options. You will have to add additional flags on particular system. In particular the flags `CXX`, CXXFLAGS and `LDFLAGS` to pick your compiler, you compiler settings and your linker parameters. Chapter C discusses some popular choices. Skip this step if you have already completed it once.

- Recompile your Peano 4 code by typing in

```
make
```

This should give you a couple of `ExaHyPE2Core` libraries in the respective subdirectories. Skip this step if you have already completed it once.

- Ensure that `python` subdirectory is in your Python search path and
- run the Jupyter notebook in `examples/exahype2/euler`:

```
export PYTHONPATH=../../../python/
jupyter-notebook Euler.ipynb
```

or—if you don't like Jupyter notebooks—type in

```
export PYTHONPATH=../../../python/
python3 example-scripts/finitevolumes.py -cs 0.2
./peano4
```

- Finally, jump straight into Chapter A and pick one of the pathways there to convert the output files into vtk files which Paraview or VisIt can display.

4.1 Historical remarks

ExaHyPE 2 is the follow-up development of the ExaHyPE project which has been funded by the EU from 2015–2019. The present version is the second generation of the code (ExaHyPE 2) and has seen substantial rewrites of core routines. However, many paradigms and even code blocks remain the same. Migrating from the original ExaHyPE to ExaHyPE 2 thus should be straightforward.

The original ExaHyPE had been built on top of Peano (third generation) and tried to hide as much of Peano away as possible. In ExaHyPE 2, I go the opposite way: ExaHyPE 2 is a full-blown Peano add-in and I try not to hide anything away. Where we designed our own data management on top of Peano in ExaHyPE, all the data management (as well as parallelisation, e.g.) is native Peano 4.

With the migration from a sole-C++ philosophy to C++ supplemented by a Python API in Peano 4, I also dumped ExaHyPE's former configuration/specification file pradigma. An ExaHyPE 2 application now is championed by a sole Python script. This Python script yields a native Peano 4 application (builder mechanism) which then assembles the application.

4.2 Case study: minimalist Finite Volumes for the Euler equations

R In the `examples/exahype2` directory, there are multiple ExaHyPE 2 experiments. An ExaHyPE 2 project is mainly a Python script supplemented by some C++ user code. We ship of these scripts as Jupyter notebooks. If you prefer to run the Jupyter notebooks without the GUI, use `ipython3`.

A minimalist ExaHyPE 2 solver resembles:

```
project = exahype2.Project( ["examples", "exahype2", "euler"], "finitevolumes", ".", "myexecname" )

project.add_solver( exahype2.solvers.fv.rusanov.GlobalAdaptiveTimeStep(
  "Euler",
  patch_size,
  unknowns, 0,
  min_h, max_h,
  time_step_size,
  flux = exahype2.solvers.fv.PDETerms.User_Defined_Implementation,
  eigenvalues = exahype2.solvers.fv.PDETerms.User_Defined_Implementation,
  time_step_relaxation = 0.01
))

project.set_global_simulation_parameters(
  dimensions, [0.0,0.0,0.0], [1.0,1.0,1.0],
  time_step_size *args.timesteps, # end time
  0.0, 0 # snapshots
)

project.set_Peano4_installation( args.peanodir, build_mode )
peano4_project = project.generate_Peano4_project()
peano4_project.generate()
```

1. We first create a new project. This is an ExaHyPE 2 project which is, for the time being, completely independent of a Peano 4 project.

2. We next create a solver. There's a number of solvers currently available within ExaHyPE 2. Once created, we add it to the ExaHyPE 2 project before

3. we set the global simulation parameters.

4. As Peano 4 has already been installed on the system, the developer did set all pathes, all compilers, all libraries, ... It would be inconvenient to ask a user to provide this information all over again. Therefore, we parse the outcome of `configure` and copy the relevant information over.

5. Finally, we ask the ExaHyPE 2 project to generate a new Peano 4 project.

Once we have a Peano 4 project, we can add further features to this one. Eventually, we make the Peano 4 project generate the actual C++ code.

Created code. The above snippet does set up a complete ExaHyPE 2 simulation. The solver will consist of two classes: `AbstractEuler` and a subclass `Euler`. The `Euler` is an empty stub when you call the script the first time. You will have to befill it with the actual Physics—flux functions for example. See the files in the directory `examples/exahype2/euler` which hosts such example code. If you want to avoid to write C++ code at all, you can also use some symbolic Python interface. An example for this one is provided via the Jupyter notebook.

Changing simulation properties. ExaHyPE 2's Python API is written as end-to-end interface: Whenever you change something, you have to run the whole script or Jupyter notebook all over again. In practice, this might be too much. Often, it is simply sufficient to repeat the following steps:

- Change something within your ExaHyPE 2 project (such as the simulation runtime, or mesh size, or ...).
- Regenerate the resulting Peano 4 project:

```
peano4_project = project.generate_Peano4_project()
```

- Re-add all the stuff you had added to the Peano 4 project before, i.e. additional libraries, additional compile flags, additional Fortran modules, ... Also set all the constants within the Peano 4 project that have to be exported into C++ code.
- Regenerate the C++ code:

```
peano4_project.generate( throw_away_data_after_generation=False )
```

Finite Volume Euler kernel benchmark ExaHyPE 2's implementation of the Euler equations with Finite Volumes is shipped with a benchmark to assess the speed of the compute kernels. The benchmark solely computes the throughput on one core or one GPU and thus provides key information how efficient these core kernels perform on your system. It neglects other effects such as multithreading or MPI data exchange, grid management, time stepping behaviour, halo data exchange, and so forth. To run the benchmark, follow the following steps:

1. Clone the repository (cmp. page 1). Skip this step if you already have a Peano 4 checkout, i.e. if you have completed this step once.
2. Create the build system (cmp. page 2). Skip this step if you already have a working configure, i.e. if you have completed this step once.

   ```
   libtoolize; aclocal; autoconf; autoheader; cp src/config.h.in .; automake --add-missing
   ```

3. Configure Peano 4 with the flags `--enable-exahype`, `--enable-loadbalancing`, `--enable-blockstructured`:

   ```
   ./configure --enable-exahype --enable-loadbalancing --enable-blockstructured
   ```

   These are the extensions. If you want to benchmark the kernels on your GPU, too, you have to add add `--enable-gpu=xxx`. The `xxx` clarifies which GPU backend to use. Type in `--help` for additional information. You will furthermore have to add additional flags on particular system. In particular the flags `CXX`, CXXFLAGS and `LDFLAGS` to pick your compiler, you compiler settings and your linker parameters. Chapter C discusses some popular choices. Skip this step if you have already completed it once.
4. Recompile your Peano 4 code by typing in

   ```
   make
   ```

5. Set the `PYTHONPATH` and run the benchmark script within the Euler example directory:

```
cd exahype2/examples/euler
export PYTHONPATH=../../../python
python3 example-scripts/kernel-benchmarks.py
```

This default builds the benchmarks for 2d with 4×4 patches. You can type in `--help` for additional information.

6. Run the actual benchmark

```
./kernel-benchmark
```

The benchmark gives you the cost per Finite Volume update (throughput) for various patch sizes, for patches which exploit stateless operations, i.e. Finite Volume updates which alter only the Finite Volume solution using PDE terms and global, constant values, and it also computes the throughput on GPUs including the offloading time. Here is what you can expect:

```
=========================================
  Number of patches fused: 1
=========================================
completed baseline measurements: 0.000270283 / 5.50138e-08
completed optimisation measurements no side effects, patch-wise, heap, AoS: 0.000179448 / 3.65251e-08
completed optimisation measurements no side effects, batched, heap, AoS: 0.000119682 / 2.43603e-08
completed optimisation measurements no side effects, patch-wise, heap, AoSoA: 0.000119399 / 2.43027e-08
completed optimisation measurements no side effects, batched, heap, AoSoA: 0.000117449 / 2.39058e-08
completed optimisation measurements no side effects, patch-wise, heap, SoA: 0.000119457 / 2.43145e-08
completed optimisation measurements no side effects, batched, heap, SoA: 0.000119485 / 2.43202e-08
no device found
```

1. Baseline is the throughput for a normal Finite Volume patch. This number will go down for more patches (fused) due to ammortisation effects, but it will not change dramatically. This is indeed the baseline throughput that you can expect for ExaHyPE 2.
2. No side effects studies variants of compute kernels which exploit state-free PDE operators (see literature). Here, the compiler can aggressively inline and vectorise. Depending on your architecture, you should see an increase of the throughput by a factor of X where X is your vectorisation width. This effect becomes visible only for a certain patch count.
3. For too high patch counts, the performance will deteriorate due to cache effects.
4. We found in-situ to give the best performance on CPUs for such a bandwidth-bound problem. in-situ recomputes some terms redundantly but in return operates without any additional temporary variables.
5. There are kernel variants which store data on the heap and some which store data on the call stack. The latter are typically faster yet are not supported by all compilers.
6. The GPU data comprises all data transfer. Offloading to GPUs thus pays only off for certain problem sizes aka patch counts.

4.3 Case study: A Finite Volumes code coupling two solvers (hyperbolic+elliptic)

If you want to solve systems of PDEs where some PDEs advance on different time scales, you have two options on the table in ExaHyPE 2: You can model the two solvers as one big system or you can logically code the two solvers as two totally separate things and couple them. The first approach is simple: If the two solvers feature N_1 and N_2 unknowns, you simply create one new solver with $N_1 + N_2$ unknowns and you provide flux functionsm, ncps, ... for all of the components. The big disadvantage of this approach is that all solvers have to advance in time at the same speed, that you might replicate code if you have already two existing solvers, and that you also might run a lot of unneeded calculations if only some PDE equations evolve actually in time.

In this case study, we go down the second route and show how to realise a PDE consisting of two systems of PDEs via two separate ExaHyPE 2 solvers. While this approach allows us to have a clear separation-of-solvers, the coupling of the two systems is not that straightforward. There's an additional gimnick/opportunity arising from this separated approach: We will eventually have one solver which advances in time with extremely small time steps every time the other solver does a (large) time step. This approach will allow us to realise a hyperbolic+elliptic system along the lines of Schlottke-Lakemper et al.

Problem formulation We solve the problem

$$\partial_t Q + \nabla \cdot F(Q) = \begin{pmatrix} 0 \\ \rho \nabla \phi \\ -(v, \nabla \phi)\rho \end{pmatrix} \tag{4.1}$$

which gives the Euler equation with $Q = (\rho, v, E) \in \mathbb{R}^5$ subject to a modified source term (right-hand side). While $F(Q)$ stems from the Euler equations in the previous example, a second PDE

$$-\Delta \phi = -\pi G / \rho$$

adds self-gravitation to this setup. We rephrase the latter elliptic equation into a hyperbolic variant with $\phi(\tau)$ and three helper functions $(q_1, q_2, q_3)(\tau)$ where

$$\partial_\tau \begin{pmatrix} \phi \\ q_1 \\ q_2 \\ q_3 \end{pmatrix} + \nabla \cdot \begin{pmatrix} -q_1 & -q_2 & -q_3 \\ -\phi/T_\tau & 0 & 0 \\ 0 & -\phi/T_\tau & 0 \\ 0 & 0 & -\phi/T_\tau \end{pmatrix} = \begin{pmatrix} -\pi G/\rho \\ -q_1/T_\tau \\ -q_2/T_\tau \\ -q_3/T_\tau \end{pmatrix}. \tag{4.2}$$

We will model (4.1) and (4.2) as completely separate PDEs. Equation (4.2) advances on tiny time scales which we indicate by switching from $Q(t)$ to a $\phi(\tau)$, i.e. two time arguments t and τ. Our strategy is to make (4.1) do a large time step, then we advance (4.2) in time with tiny steps until it has reached a stationary solutions. The current ρ from (4.1) feeds into these tiny time steps as fixed (time-independent) parameter. Once we have reached a stationary state, we take the arising ϕ and make it feed into the next time step of (4.1). Equation (4.2) will finally be restarted at the new time stamp reached by the Euler system.

For the Euler equations, all ingredients are well-known from the introductory example. The Poisson-ish operator is degenerated and yields (for the flux in x-direction)

$$F_x = \begin{pmatrix} -q_1 \\ -\phi/T_\tau \\ 0 \\ 0 \end{pmatrix} \quad \text{i.e.} \quad \nabla F_x = \begin{pmatrix} 0 & -1 & 0 & 0 \\ -1/T_\tau & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

with a trivial maximum eigenvalue/singular value $\sqrt{1/T_\tau}$. We note that this value does not depend on ϕ, i.e. is solution-independent.

Realisation with two individual solvers As we go down the route with separate solvers, we simply create two solvers over the corresponding number of unknowns, i.e. 5 for Euler and 4 for the Poisson operator (which we have transformed into a stationary first-order hyperbolic system). In the simplest case, this looks similar to

```
euler_solver = exahype2.solvers.fv.rusanov.GlobalFixedTimeStep(
    "Euler",
    args.patch_size,
    5, #unknowns,
    3, #auxiliary_variables,
    min_h, max_h,
    time_step_size,
    flux = exahype2.solvers.fv.PDETerms.User_Defined_Implementation,
    eigenvalues = exahype2.solvers.fv.PDETerms.User_Defined_Implementation,
    source_term = exahype2.solvers.fv.PDETerms.User_Defined_Implementation
)
poisson_solver = exahype2.solvers.fv.rusanov.GlobalFixedTimeStep(
    "Poisson",
    args.patch_size,
    4, #unknowns,
    1, #auxiliary_variables,
    min_h, max_h,
    time_step_size / 4 / 3.1415**2,
    flux = exahype2.solvers.fv.PDETerms.User_Defined_Implementation,
    eigenvalues = exahype2.solvers.fv.PDETerms.User_Defined_Implementation,
    source_term = exahype2.solvers.fv.PDETerms.User_Defined_Implementation
)
project.add_solver( euler_solver )
project.add_solver( poisson_solver )
```

The interesting thing here is the auxiliary variables: Our Poisson-type equations (4.2) require the solution's density ρ of the Euler equation (4.2) as input. As we want to develop the two solvers completely separate, we model introduce a material parameter for (4.2) which "mirrors" the density of the other equation. For the Poisson implementations, it seems as if the density indeed were some fixed value prescribed by the code. The other way round, our modified Euler terms in (4.1) require the gradient from (4.2). There are different ways to realise the coupling in this direction. In the present example, we go down the simplest route and explicitly store the gradient entries as material parameters.

The implementation of the three PDE terms (flux, eigenvalue and source term) is straightforward for both PDEs. Each routine accepts a vector Q which holds both the solution, i.e. five or four components respectively, and then appended to these the one or three "material parameters". For the flux and the eigenvalue, we ignore these material parameters, i.e. the code looks exactly as the plain Euler equations for the first part. The source term however now takes the material parameter as input. Here's the kernel for the Poisson-type equation:

```
void ::examples::exahype2::selfgravitation::Poisson::sourceTerm(
  const double *__restrict__ Q, // Q[4+1]
  const tarch::la::Vector<Dimensions,double>& volumeX,
  const tarch::la::Vector<Dimensions,double>& volumeH,
  double t,
  double dt,
  double *__restrict__ S // S[4]
) {
  logTraceInWith4Arguments( "sourceTerm(...)", volumeX, volumeH, t, dt );

  const double T_tau = 1.0 / 4.0 / tarch::la::PI/tarch::la::PI;
  const double G = 6.67e-11;
  const double rho = Q[4]; // This is the input term from the other solver

  S[0] = -tarch::la::PI *G / rho;
  S[1] = -Q[1]/T_tau;
  S[1] = -Q[2]/T_tau;
  S[1] = -Q[3]/T_tau;

  logTraceOut( "sourceTerm(...)" );
```

```
}
```

Coupling the equations The actual coupling in this example is done by mapping some unknowns from one equation onto material parameters in the other equations. As we deal with a quasi-stationary second PDE, this coupling is trivial in the sense that we do not have to interpolate in time.

There are two places where we can add this coupling: It can either be a preprocessing step of the time step or a postprocessing step. While, in principle, both approaches do work here, there's a delicate difference (see documentation): In the pre-processing step, we have access to the patch data plus the halo. In the post-processing step, we have only access to the patch content. In our example, these things don't make a big difference, so we run for a coupling after each time step.

For the density, which feeds from the (4.1) into the material parameters of the Poisson-ish equation, we need a plain copy from

$$Q[0]|_{\text{Euler}} \mapsto Q[4]|_{\text{Poisson}}$$

for which Peano 4's toolbox provides a generic implementation:

```
euler_solver.add_user_action_set_includes( """
#include "toolbox/blockstructured/Copy.h"
""")
euler_solver.postprocess_updated_patch = """
  ::toolbox::blockstructured::copyUnknown(
    {},      // no_of_unknowns_per_dimension
    fineGridCellEulerQ.value, // source
    0,       // the_first_index (0) is_rho
    5+3,     // unknowns_in_source (incl_material_parameters)
    0,       // no_overlap/halo_here
    fineGridCellPoissonQ.value, // dest
    4,       // index_four_in_destination, i.e. fifth_entry
    4+1,     // four_unknowns_in_source (incl_material_parameters)
    0        // no_overlap/halo_here
  );
""".format(args.patch_size)
```

In this special case, we use a plain copy. If the two different solvers have different patch sizes, i.e. unknowns per dimension, we have to interpolate accordingly. The invocation is kind of self-explaining: We include a helper function from the toolbox and call the `copyUnknown`. The "nasty" part is the `fineGridCellPoissonQ` part, for which we have to study the naming conventions within ExaHyPE 2—a detail usually hidden from the user: The unknowns per cell (so the plain double array) are called `fineGridCell` followed by the name of the solver `Euler` and then a `Q`. The data is actually a struct for our plain double fields which hosts a pointer `value`.

R If you work in the preprocessing step, i.e. inject the coupling prior to the time step, you work with a temporary array `reconstructedPatch` which hosts the patch data plus the information from the halos. That is, you work with some `fineGridCell...Q` value as pre-image, but you write into `reconstructedPatch` which now already has a halo. You can also hook into this step to "save" data from `reconstructedPatch` before it is handed over to the time stepping.

For the coupling the other way round, we follow exactly the same principle. There is room for optimisation here, as we would only have to couple after the Poisson equation has run into a stationary solution, but I omit such optimisations here. I simple invoke the mapping

$$Q[0]_{\text{Poisson}} = \phi \mapsto \begin{pmatrix} \partial_x \phi \\ \partial_y \phi \\ \partial_z \phi \end{pmatrix} = \begin{pmatrix} Q[5] \\ Q[6] \\ Q[7] \end{pmatrix}_{\text{Euler}}$$

after each and every solver update step:

```
poisson_solver.add_user_action_set_includes( """
#include "toolbox/blockstructured/Derivative.h"
""")
poisson_solver.postprocess_updated_patch = """
  ::toolbox::blockstructured::computeGradient(
    {},      // no_of_unknowns_per_dimension
    fineGridCellPoissonQ.value, // source
    0,       // take_first_entry_of_solution_vector, ignore_helper_entries
    4+1,     // unknowns_in_source_(incl_material_parameters)
    0,       // no_overlap/halo_here
    fineGridCellEulerQ.value, // source
    #if_Dimensions==2
    {{5,6}},   // indices_to_which_we_write_the_gradient. Depends_on_dimension
    #else
    {{5,6,7}}, // and_uses_two_brackets_as_escape_symbol
    #endif
    5+3,     // four_unknowns_in_source_(incl_material_parameters)
    0,       // no_overlap/halo_here
    marker.h()
  );
""".format(args.patch_size)
```

Time step synchronisation So far, we couple the two solvers in each and every time step. This makes no sense, if the two solvers don't run in-syn, i.e. with exactly the time step size. For the coupling in time, we exploit some simple properties of ExaHyPE 2:

1. Once we switch to `GlobalAdaptiveTimeStep` or literally any solver that does not use fixed time step sizes, we can tweak the time step sizes to meet our needs.
2. Each ExaHyPE 2 cell has a time stamp. Even if you employ some global time stepping scheme, every single cell still holds its local time stamp (though they are, obviously, then all the same).
3. Every time step update is protected by a guard. This is one large sequence of boolean expressions connected via ands. Through the guards, ExaHyPE 2's local time stepping ensures, for example, that cells do not run ahead too much.

Therefore, we first switch to an adaptive time stepping scheme. The Poisson solver usually employs a smaller relaxation of the time step size. Our strategy for a coupled solver is to add one more boolean evaluation to the time step guard of these adaptive solvers. As we have two solvers per cell (Poisson and Euler), the cell hosts two time stamps and the guard can compare these two time stamps. In our example, we know that the Poisson solver always is the slow solver. In a first step, we thus have to throttle down the Euler solver. It has to wait for its cousin solver:

```
euler_solver._action_set_update_cell.guard += " and tarch::la::greaterEquals(fineGridCell" + \
  poisson_solver.name + "CellLabel.getTimeStamp(), fineGridCell" + euler_solver.name + \
  "CellLabel.getTimeStamp() )"
```



Throughout the coupling example, I take two existing ExaHyPE 2 solvers and modify/augment their attributes. In a larger code, it might be reasonable to create two completely new Python classes which inherit from the respective solvers, i.e. to model them as real new solvers. A lot of the string construction above violates object-oriented information hiding paradigms, so it might be better to accommodate it within classes of their own.

Next, we have to make a decision how to handle the paper's original idea that the Poisson solver should run into a stationary solution before we actually progress in time. In a world with a fixed global time step, this can be achieved by making the time step of the Poisson solver extremely

small. We then get hundreds and thousands of Poisson time steps per Euler step and "just" have to ensure that these Poisson steps are sufficient to obtain an almost stationary solution just before Euler hops in time once again. This approach quickly becomes computationally infeasible. We need an approach that switches gears once we are stationary.

To determine if the solver is (close to) stationary, we evaluate the difference between the gradient of ϕ after each Poisson step. For this, we tweak the coupling term:

```
poisson_solver.postprocess_updated_patch = """
  double delta =
  ::toolbox::blockstructured::computeGradientAndReturnMaxDifference(
    {},      // no_of_unknowns_per_dimension
    fineGridCellPoissonQ.value, // source
    0,       // take_first_entry_of_solution_vector, ignore_helper_entries
    4+1,     // unknowns_in_source (incl_material_parameters)
    0,       // no_overlap/halo_here
    fineGridCellEulerQ.value, // source
    #if Dimensions==2
    {{5,6}}, // indices_to_which_we_write_the_gradient. Depends_on_dimension
    #else
    {{5,6,7}}, // and_uses_two_brackets_as_escape_symbol
    #endif
    5+3,     // four_unknowns_in_source (incl_material_parameters)
    0,       // no_overlap/halo_here
    marker.h()
  );
  repositories::InstanceOf{}.reportGradientDelta(delta);
""".format(args.patch_size, poisson_solver.name)
```

The new computation preserves the semantics of the old one plus computes a value `delta` which it then pipes into the Poisson solver object. Here, we once again exploit our knowledge about ExaHyPE 2's internal working and naming conventions. To make the whole thing finally work, we tweak the Poisson solver:

- We let the Poisson solver keep track of the gradient deltas. Actually, we use two deltas. `_currentMaxGradientDelta` is used to analyse the current delta which we change throughout the grid traversal, while `_previousMaxGradientDelta` holds the result from the previous grid traversal.

```
    static tarch::multicore::BooleanSemaphore _maxGradientDeltaSemaphore;

    double _currentMaxGradientDelta;
    double _previousMaxGradientDelta;
```

- We introduce a new function `reportGradientDelta` that accepts the `delta` as we have computed it above.

```
void examples::exahype2::selfgravitation::Poisson::reportGradientDelta(double delta) {
  tarch::multicore::Lock lock( _maxGradientDeltaSemaphore );
  _currentMaxGradientDelta = std::max(_currentMaxGradientDelta,delta);
}
```

- We hook into `startTimeStep` to roll the deltas over.

```
void examples::exahype2::selfgravitation::Poisson::startTimeStep(
  double globalMinTimeStamp,
  double globalMaxTimeStamp,
  double globalMinTimeStepSize,
  double globalMaxTimeStepSize
) {
  AbstractPoisson::startTimeStep( globalMinTimeStamp, ... );
  _previousMaxGradientDelta = _currentMaxGradientDelta;
  _currentMaxGradientDelta = 0.0;
}
```

- We overwrite `getAdmissibleTimeStepSize`. If the two delta values do not differ that much anymore, we assume we have converged and we use the standard time step size the solver would use. If the two delta values differ significantly, we assume that we have not yet convered and thus use a tiny time step size.

```
double examples::exahype2::selfgravitation::Poisson::getAdmissibleTimeStepSize() const {
  double originalAdmissibleTimeStepSize = AbstractPoisson::getAdmissibleTimeStepSize();

  if (_previousMaxGradientDelta>1.0e-8) {
    return tarch::la::NUMERICAL_ZERO_DIFFERENCE*2.0; // not converged yet
  }
  else {
    return originalAdmissibleTimeStepSize;
  }
}
```

4.4 Generic ExaHyPE 2 features

The following features and techniques hold for all ExaHyPE 2 applications and are always used in the same way:

4.4.1 IO, logging and initialisation

ExaHyPE 2 codes by default yield snapshots in Peano 4's block format. That is, there's no distinguished ExaHyPE 2 format. Logging/state data is piped to the terminal or a file. It depends whether you have compiled Peano 4 with a special toolchain.

Patch file output The format as well as the postprocessing of Peano 4's patch-file output is discussed in Chapter A. By default, ExaHyPE 2 dumps all data as one large vector `Q` per Finite Volume/integration point, i.e. you get a set of scalars. It also dumps all the resolution levels of the grid simultaneously. Therefore, one of the first postprocessing steps usually is to remove all data from the output data set besides the fine grid level.



In principle, Peano 4 follows the philosophy that postprocessing should be done by postprocessing tools. It therefore just dumps the data (though efficiently in a bespoke format). For many applications, each dump of the whole simulation data is already too heavy. Therefore, you can add filters.

Filters can be added to your setup via

```
probe_point = [0,0.693,0]
project.add_plot_filter( probe_point,[0.0,0.0,0.0],1 )
project.add_plot_filter( offset,size,10 )
```

The example above sets a probe, i.e. every plot step only the patch data around the probe is dumped. Every 10 snapshot steps however, the whole domain is written. I assume that `offset` and `size` are configured accordingly.

The command

```
project.set_output_path( "/scratch/myfolder" )
```

configures Peano 4 with a proper output path. Without this command, all output data is written to the working path.

Logging output ExaHyPE 2 searches for a text file `exahype.log-filter` in the working directory. If no such file is found (or the file is corrupted), then it will use some default filter rules, i.e. dump the information to the terminal that I consider to be most important. If you want to adopt the output, I strongly recommend that you add a log filter configuration file.

Often, users are only interested in a particular phase of the simulation or want to specify different log entries per program phase. The program phases offered by default are

- `create-grid-but-postpone-refinement`
- `create-grid`
- `create-grid-and-converge-load-balancing`
- `init-grid`
- `plot-solution`
- `time-step`

The syntax of log filter files is discussed in Section 11.4.

Solver initialisation If you have a complex solver setup, you might want to run some complex initialisation at startup. However, important parts of the ExaHyPE 2 infrastructure might not be up yet when the solver's constructor is invoked. So it is generally not a good idea to add too much setup routines to the constructor.

Each solver has eight further routines that are of interest to squeeze in any initialisation or global state update:

- startGridConstructionStep
- finishGridConstructionStep
- startGridInitialisationStep
- finishGridInitialisationStep
- startTimeStep
- finishTimeStep
- startPlottingStep
- finishPlottingStep

If you plug into these, please ensure you continue to call the original (superclass) routine still. A typical pattern we've seen is to run some initialisation just prior to the first grid construction step:

```
class MyClass ... {
  void startGridConstructionStep() override;
  ...
};
```

```
void MyClass::startGridConstructionStep() {
  // call superclass routine
  AbstractMyClass::startGridConstructionStep();
  static bool isInitialised = false;
  if (not isInitialised) {
    isInitialised = true;
    // your initialisation
  }
};
```

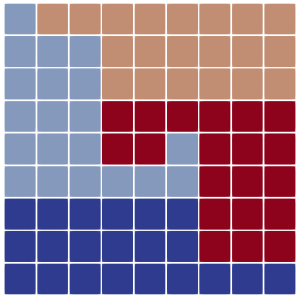


Figure 4.1: Domain decomposition in ExaHyPE 2 for a run with four threads/ranks. Each individual cell hosts a patch.

4.4.2 Running on a parallel computer

To run ExaHyPE 2 in parallel, you have to build Peano 4 with support for multithreading and/or MPI. Once this is done, the engine more or less runs in parallel out of the box.

Domain decomposition The core domain decomposition scheme realised in ExaHyPE 2 is a non-overlapping multiscale domain decomposition based upon space-filling curves (SFC). To use this scheme, you have to specify an appropriate load balancing scheme. Without any load balancing, the code will never split up the domain and thus effectively run serially.

I plan to support multiple different dynamic load balancing schemes over time, but the straightforward decomposition scheme shipped with Peano 4 at the moment is recursive subdivision, a modification of OpenMP's guided scheduling. To use it, you have to call

```
project.set_load_balancing( "toolbox::loadbalancing::RecursiveSubdivision" )
```

on your ExaHyPE 2 project. There are (optional) arguments to this call that configure the load balancing. ExaHyPE 2 has its own default configuration. Via

```
project.set_load_balancing("toolbox::loadbalancing::RecursiveSubdivision",
 "new␣::exahype2::LoadBalancingConfiguration()" )
```

you switch to this default. You can tailor it further. With

```
project.set_load_balancing(
 "toolbox::loadbalancing::RecursiveSubdivision",
 "new␣::exahype2::LoadBalancingConfiguration(0.8,200)" )
```

for example, you instruct the load balancing that you want it to be well-balanced accross ranks up to 80% (the default is 90%) and you also to constrain the tree (subpartition) size by 200 cells. This means that no domain on a rank will be split if this rank's load is ±20% off, and a partition will never split up if it has less than 200 cells.

R There is a always a trade-off between the different settings: If you increase the quality of the domain decomposition, then the time per time step will go down. However, the load balancing will need way longer. If you pick a minimum cell size, then the grid will build up first until it can accommodate the constraint, and then it will decompose. Grid decomposition however is both time consuming and requires (temporarily) quite a lot of memory.

Task parallelism Some ExaHyPE 2 solvers add further task parallelism to this decomposition. In general, the enclave solvers tend to perform better on massively parallel systems.

Load balancing settings



Consult the troubleshooting FAQs in Chapter F.

While ExaHyPE 2 makes it relatively straightforward to parallelise a code, the tuning of a particular experiment is often tricky. As ExaHyPE 2 is an engine supporting multiple application (domains) and as the "perfect" load balancing strategy depends strongly on the application type plus the chosen experiments plus the machine used, it can only provide vague heuristics of settings that work in most of the cases. To get really good performance, you will have to play around with the load balancing settings.

Here are some recommendations if you use the recursive load balancing. The recommendations describe how to tweak the three parameters that the ExaHyPE 2 configuration offers:

- Do not set the load balancing quality too high. I am usually fine if the balancing between the ranks is about 80%–90% accurate.

```
project.set_load_balancing(
 "toolbox::loadbalancing::RecursiveSubdivision",
 "new␣::exahype2::LoadBalancingConfiguration(0.8)" )
```

  First get something up and running and test it. After that, you might want to increase the quality closer to 100%. However, the higher this quality threshold the longer your grid setup and balancing time.

- Do not set a minimum tree size initially. Prescribing a number of cells that a subpartition has to have is a powerful tool to ensure that the overhead does not become excessive. However, a threshold massively slows down the grid construction. So it is better to use this parameter as the last one (see below):

```
project.set_load_balancing(
 "toolbox::loadbalancing::RecursiveSubdivision",
 "new␣::exahype2::LoadBalancingConfiguration(0.8,0)" )
```

- If your code tends to hang in MPI (consult Chapter F please), try to kick the simulation off with only one partition per rank:

```
project.set_load_balancing(
 "toolbox::loadbalancing::RecursiveSubdivision",
 "new␣::exahype2::LoadBalancingConfiguration(0.8,0,1)" )
```

  Once this passes through, increase the last counter to at most the number of cores per rank. As long as you use an enclave-based FV solver, restricting the number of subpartitions per rank does not constrain your shared memory efficiency too much. It simply means that domain decomposition is not used too aggressively to get BSP-type parallelism. Instead, Peano 4 will try to use tasks on a per-patch basis.

- If you work without periodic boundary conditions, the initial load balancing will likely be poor if the mesh is too shallow: The code cannot deploy cells along the periodic boundary to other ranks. These always have to be handled by rank 0. So it makes sense to set the minimal mesh size (second argument) to at least $3^d + 1$. This prolongs the load distribution sufficiently long until enough cells are available. The knock-on problem then is that we have a lot of cells and the load balancing tries to split them up evenly—unaware that all the boundary cells can't be distributed. So you will have to restrict the number of cells per

inter-rank balancing. I recommend to to add 3^d as fourth argument to ExaHyPE 2's load balancing configuration.

- If your code suffers from the fact that the load balancing moves around too large trees between ranks, use the fourth argument to restrict this behaviour.

Writing your own parallel extensions Most users that write extensions to ExaHyPE 2 which need some global data exchange are users which add new fields to their solver. Examples are global statistics that they want to track over the solution or global variables that they control.



Each MPI rank holds one instance of the solver. All threads per MPI rank share the same instance of the solver.

Peano 4 splits up the domain into chunks along the space-filling curve. If you use an enclave solver, it then splits up these chunks further into tasks. That is, operations per cell such as the Riemann solves and the patch postprocessing can run in parallel on each thread.

If you add new attributes to your solver, this attribute exists on each solver on each rank. You therefore have to

- ensure that no two threads access the attribute at the same time;
- ensure that the attributes of the individual solver instances per rank remain consistent.

The first item is relatively simple: You might, for example, decide to protect all accesses to your own variables via a semaphore. These techniques are discussed in Section 13.1.1. There is no specific place to take care of the protection: Just protect the actual data access.

The second item requires slightly more work, as you have to globally reduce the attribute: After each time step, you have to ensure that all ranks exchange their attributes with each other, and you then have to merge these data. I recommend not to use MPI's data exchange routine directly, but to use Peano 4's wrappers around these routines. These wrappers are discussed in Section 13.2.2. The actual data exchange should be done at the end of a each time step. For this, you add

```
void finishTimeStep() override;
```

to your solver. The implementation should first invoke the superclass and then add the additional data exchange.

```
void mynamespace::MyFancySolver::finishTimeStep() {
  MyFancyAbstractSolver::finishTimeStep();
  #ifdef Parallel
  // additional data exchange
  #endif
}
```

ExaHyPE 2 internally realises these two steps for all of its solver attributes such as the maximum mesh size or the admissible time step sizes.

4.4.3 Adding adaptive mesh refinement (AMR)



AMR is application- and numerics-specific. Therefore, the signatures of the routines discussed below might change if you switch from Finite Volumes to ADER-DG, e.g. The discussed principles however always are the same.

By default, ExaHyPE 2 solvers operate on a regular grid. To facilitate an adaptive mesh, you have to ensure that your solver is aware that you operate with different $h_{\max} > h_{\min}$. So you have to alter these two flags of your solver instantiation. As Peano 4 operates with three-partitioning, you have to pick $h_{\min} \leq \frac{1}{3} h_{\max}$ to switch on adaptivity.

Once AMR is, in principle enabled, you still have to supplement a refinement and/or coarsening criterion:

Supplementing the refinement criterion On the Python side, dynamic adaptivity is added via a one-liner:

```
mysolver.set_implementation(
 refinement_criterion=exahype2.solvers.fv.PDETerms.User_Defined_Implementation
)
```

Once you did rerun your Python script, you have to add your solver a new routine:

```
::exahype2::RefinementCommand refinementCriterion(
  const double *__restrict__ Q,
  const tarch::la::Vector<Dimensions,double>& volumeCentre,
  const tarch::la::Vector<Dimensions,double>& volumeH,
  double t
) override;
```

If this is the first time you create your solver, ExaHyPE 2 adds this routine into the solver blueprint. If you add AMR to an existing solver, you have to add the routine manually, as ExaHyPE 2's code generator never overwrites a user solver. A simple default implementation of this routine resembles

```
::exahype2::RefinementCommand MyClass::refinementCriterion(
  const double *__restrict__ Q, // Q[5+0],
  const tarch::la::Vector<Dimensions,double>& volumeCentre,
  const tarch::la::Vector<Dimensions,double>& volumeH,
  double t
) {
  ::exahype2::RefinementCommand result = ::exahype2::RefinementCommand::Keep;
  return result;
}
```

This version does not (yet) any adaptivity. Your solver returns `Keep` all the time, so Peano 4 will create a mesh that just matches $h_{\max}$ as specified via the solver and then keep this match. Once you return `Refine`, it will however refine. If your mesh however becomes finer than your solver's $h_{\min}$, Peano 4 will ignore the refinement request of your ExaHyPE 2 solver.

`Keep` keeps the mesh as it is. That is, if you return `Keep` for an adaptive grid part, this part will not change. You have to invoke `Erase`[1]. Both erase and keep are only recommendations to Peano 4—Peano 4 will try to meet these requests, but will always make the mesh slightly finer or not refine if the minimum mesh size would be violated too harshly. That is, if some parts of the mesh ask for a refinement and direct neighbours want to coarsen, the refinement gets higher priority. Along the same lines, erase instructions will simply be ignored if the resulting mesh would be coarser than $h_{\max}$.



It is important to keep in mind that refinement requests are not immediately realised by Peano 4. Refinements are requested in one time step, and then realised in the subsequent one. Also, Peano 4 will try to accommodate coarsening requests, but if there are any issues (with mesh partitioning or balancing, e.g.), then these considerations willl have higher priority and Peano 4 might decide to continue with a finer mesh than requested.

[1] I chose erase deliberately and not coarsen, as coarsening has different semantics in multiscale/multigrid algoritms where coarsening does not remove mesh entities but "changes the resolution view".

Different solvers come along with different interpolation and restriction seems. The discussion around Finite Volume solvers for example can be found on page 33.

4.4.4 GPU support

D ExaHyPE 2 uses GPUs in a classic *offloading, accelerator style*. This means that it takes particular tasks, ships them to the GPU and then ships them back. This happens under the hood, and neither does data reside on the accelerator permanently nor is the accelerator used all the time.

The whole approach assumes that there are computations (cell/patch updates) which can be done without any effect on global variables. It works if and only if we can ship a task to the GPU, and then get the solution data for this task back and no other global variabled changes. We furthermore assume that the computations that fit onto the GPU have no state. They can use some global, static variables, but they cannot access the solver's state which can change over time. We rely on code parts with *have no side effects and do not depend on the solver state* (minus global variables).

This might not work for all patches: There are always patches which evaluate and analyse some global data, or build up global data structures. In this case, ExaHyPE 2 only offloads the remaining, simple patches to the GPU.

ExaHyPE 2 supports GPUs via task offloading. To use GPUs, you have to follow the following steps:

First, your environment has to be compiled with GPU support. For this, configure with the argument `--with-gpu` and select an appropriate GPU programming model. There is a brief discussion on this in Section 2.2. Rebuild the whole Peano 4 core.

R If you want to profile or optimise your code, you might want to pick a particular vendor toolchain. See configure's help on `--with-toolchain`.

Second, use an ExaHyPE 2 solver which has support for GPUs and instruct it that you plan to have patch updates without access to the solver's internal state. The Finite Volume enclave solvers for example all support GPUs, but the versions without enclave tasking do not support them. The solver documentation should clarify which one to select. For most of these GPU-enabled solvers, it is sufficient to pass an additional flag (`pde_terms_without_state=True` for the FV variants, e.g.) that tells the solver that we have PDE terms which have no side effect which also do not need the solver object.

Third, provide the additional PDE terms without state. To faciliate the offloading, we have to create alternative versions of our PDE term functions that can work independent of the solver object (and in return cannot modify it). Depending on which terms we have, we'll need stateless versions of the flux, the non-conservative product and the source term. We'll always needs the eigenvalue function. Per function, keep the original one and add a second one which is

- static;
- accepts an additional flag of type `Offloadable`. This is the last argument, i.e. a static version of a function has exactly the same arguments as the non-static, default variant but then has one more argument. The last argument is solely there to be able to distinguish the static version from the normal one, as C++ cannot overload w.r.t. static vs. not static. `Offloadable` is automatically defined in the abstract base class which ExaHyPE 2's Python API generates.

- is embedded into

```
#if defined(OpenMPGPUOffloading)
#pragma omp declare target
#endif

my new static function variants

#if defined(OpenMPGPUOffloading)
#pragma omp end declare target
#endif
```

  Though you can leave these annotations away if you use SYCL only.

Very often, the standard flux and eigenvalue routines can invoke the static variants and you can thus eliminate code redundancies. For example, you might have the normal flux function and a static flux function with the additional `Offloadable` parameter, but the normal function just invokes the static cousin.

In many of our codes, we take a function like

```
void examples::exahype2::SSInfall::SSInfall::flux(
  ...
) {
  logTraceInWith4Arguments( "flux(...)", faceCentre, volumeH, t, normal );
  assertion4( Q[0]>0.0, faceCentre, volumeH, t, normal );
  compute something;
  assertion4( F[0]>0.0, faceCentre, volumeH, t, normal );
  logTraceOutWith4Arguments( "flux(...)", faceCentre, volumeH, t, normal );
}
```

and simply split it into two variants:

```
void examples::exahype2::SSInfall::SSInfall::flux(
  ...
) {
  logTraceInWith4Arguments( "flux(...)", faceCentre, volumeH, t, normal );
  assertion4( Q[0]>0.0, faceCentre, volumeH, t, normal );
  flux(Q,faceCentre,volumeH,t,normal,F,Offloadable::Yes);
  assertion4( F[0]>0.0, faceCentre, volumeH, t, normal );
  logTraceOutWith4Arguments( "flux(...)", faceCentre, volumeH, t, normal );
}


void examples::exahype2::SSInfall::SSInfall::flux(
  ...,
  Offloadable
) {
  compute something;
}
```

The static version is used on the GPU. The normal one is used on the CPU yet defers immediately to the static version. Please note that GPUs are quite restrictive w.r.t. terminal outputs and assertions. You will have to remove both from your static routine versions. In the example above, the non-static version wraps the core functionality into assertions. Therefore, we have the assertions on the CPU, but the core part that goes to the GPU is free of assertions. The same holds for the logging.

Finally, pick a threading backend which supports GPU offloading. The default threading model of Peano 4 does not support task fusion or the offloading to GPUs. If you run the code with `--help`, it will report on various threading models. Use one that has `fuse` in its title, and you should finally get GPU offloading.

R If only some patches/cells can be offloaded to the GPU, then you can redefine the routine

```
virtual bool patchCanUseStatelessPDETerms(
  const tarch::la::Vector<Dimensions,double>& patchCentre,
  const tarch::la::Vector<Dimensions,double>& patchH,
  double t,
  double dt
) const;
```

in your solver. By default, this routine returns `true` always. Here's the clue: This is a normal function, i.e. you can use the solver's state and make the result depend on this one.

4.5 Finite Volume extensions

4.5.1 Boundary conditions

Boundary conditions often are tricky to implement and solely up to the user. In ExaHyPE 2, Dirichlet and Neumann boundary conditions on Cartesian meshes are close to trivial. More sophsticated boundary conditions require slightly more work. Here are some remarks how to implement them.

R The boundary condition semantics are solely determined by the user's boundary condition callback. Therefore, it is possible to use different boundary conditions in different domain areas.

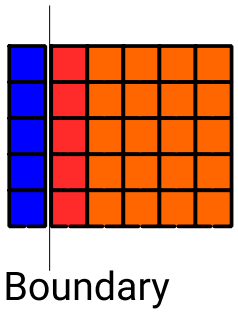


Figure 4.2: ExaHyPE 2's Finite Volume scheme organises its mesh into patches. Here, we use a 5×5 patch (orange/red). Prior to the update of a patch, ExaHyPE 2 calls `boundaryConditions` for each of the volumes adjacent to the boundary (darker red). The routine allows you to alter the value of the halo surrounding the patch (blue). The outer (blue) cells are subsequentially used to determine a flow through the boundary into the domain.

Boundary conditions in general are realised via their own callback called `boundaryConditions`. When you run the Python toolkit, you get an empty implementation of this routine that you can befill. The routine is called per outer volume (Figure 4.2).

Dirichlet boundary conditions . Dirichlet boundary conditions

$$Q(x)|_{\partial\Omega} = f(x,t)$$

are close to trivial to implement. You set outer values due to

```
void ...::MySolver::boundaryConditions(
  ...
```

```
) {
  Qoutside[0] = ...
  Qoutside[1] = ...
  ...
}
```

Obviously, the values on the right-hand side can depend on time and spatial position. We find two different ways to impose Dirichlet conditions:

1. You simple set the outer volumes (blue in Figure 4.2) to $f(x,t)$.
2. You compute the average between the blue value and the interior value as $Q|_{\partial\Omega} = \frac{1}{2}(Q_{\text{outside}} + Q_{\text{inside}}) = f(x,t)$ which yields $Q_{\text{outside}} = 2f(x,t) - Q_{\text{inside}}$.

The latter formula is inspired by Finite Differences and might give a slightly more accurate values. With a sole averaging Riemann solver, it directly sets the flux. If you use something like Rusanov, you might tweak Q_{outside} further such that the eigenvalue term along the boundary face has the correct (imposed) value too.

Neumann boundary conditions (Finite Volumes) As we can read the volume values inside the domain prior to a time step and modify those outside of the domain, it is straightforward to implement homogeneous Neumann conditions. The snippet below implements homogeneous Neumann conditions

$$(\nabla Q(x), n)\,|_{\partial\Omega} = 0$$

```
void ...::MySolver::boundaryConditions(
  ...
) {
  Qoutside[0] = Qinside[0];
  Qoutside[1] = Qinside[1];
  ...
}
```

but obviously you can use a simple Finite Differences scheme to impose non-homogeneous conditions as well.

Inflow/outflow boundary conditions Inflow or outflow boundary conditions are conceptually close to Neumann conditions. Often, it is however reasonable to supplement them with some limiter, i.e. if the flow is close to zero and inwards, a modified outflow condition could manually set it (limit it) to zero.

While this approach does work in most of the cases, it struggles to handle information flow that is not orthogonal to the Cartesian faces. It assumes that all flow is axis-aligned: You can only modify the face-connected neighbour volume in the ghost region outside of the domain. You cannot alter diagonal volumes. Unless you want to use distorted, i.e. boundary-aligned, meshes as discussed below, there's no way around this. Yet, increasing the domain size usually helps in such a case.

Extrapolating boundary data (Finite Volumes) It is more complicated to realise boundary conditions that extrapolate data into the outside. Such boundary conditions read as

$$Q(x)|_{\partial\Omega} = \lim_{x \mapsto \partial\Omega} f(Q(x,t), \nabla f(Q(x,t)) \text{ or } (\nabla Q(x), n)\,|_{\partial\Omega} = \lim_{x \mapsto \partial\Omega} f(Q(x,t), \nabla f(Q(x,t)).$$

It is obviously trivial to extrapolate the solution value within the domain to the boundary volumes (dark red to blue value in Figure 4.2), but once you need the derivatives at the boundaries, you

have to invest more work. After all, the boundary routine only provides access to the boundary data, but not to any derivatives.

Users have successfully implemented non-homogeneous Neumann boundary conditions where they extrapolate the derivatives onto the boundary and use those to inform the actual value. This requires work however:

- Introduce a new material parameter that stores the directional derivative. If you need the non-homogeneous Neumann conditions all around the domain, you will need `Dimensions` material parameter per unknown:

```
mySolver = MySolver( "Myname", my_patch_size, number_of_unknowns,
  auxiliary_variables=3*number_of_unknowns, ... );
```

- Compute the derivatives per volume after each time step in a Finite Differences sense. We can add such a code snippet in Python:

```
my_solver.preprocess_reconstructed_patch_kernel = """
  dfor(cell,patchSize) {
    //tarch::la::Vector<Dimensions,int> currentPosition=marker.getOffset()+(cell+0.5)*volumeH;
    tarch::la::Vector<Dimensions,int> currentCell = cell + tarch::la::Vector<Dimensions,int>(1);

    tarch::la::Vector<Dimensions,int> leftCell  = currentCell;
    tarch::la::Vector<Dimensions,int> rightCell = currentCell;
    leftCell(d)  -= 1;
    rightCell(d) += 1;

    if (cell(0)==0) { // left layer of patch, so use right differences
      const int rightCellSerialised  =
        peano4::utils::dLinearised(rightCell,patchSize + 2*1);
      const int currentCellSerialised =
        peano4::utils::dLinearised(rightCell,patchSize + 2*1);
      for(int i=0;i<unknowns;i++) {
        target[currentCellSerialised*(unknown+auxiliaryVariables)+i] =
          1/h *(
            target[rightCellSerialised*(unknown+auxiliaryVariables)+i] -
            target[centralCellSerialised*(unknown+auxiliaryVariables)+i]
          );
        ...
      }}}"""
```

- With this information, we can finally set the boundary conditions: The routine `boundaryConditions` is given the data at the boundary. Due to our modifications, these data are augmented by additional "material" parameters[2] which store the derivatives from the current time step. We can thus realise boundary conditions which use the (boundary-extrapolated) derivative besides the actual value.

Boundary-aligned geometries ExaHyPE 2's mesh is topologically Cartesian. That does not mean that you have to work in a Cartesian world. You can distort, squeeze, rotate the mesh locally as long as your mesh topology remains adaptive Cartesian. Codes like the ExaSeis simulation package introduce material parameters per degree of freedom to encode mesh distortions. This allows them to represent a boundary exactly, i.e. to adopt the Cartesian mesh to the real domain boundaries. Obviously, the actual boundary condition implementations then have to be adopted accordingly, too.

4.5.2 Interpolation and restriction schemes

If you employ adaptive mesh refinement, you have to project coarser resolutions onto finer resolutions (interpolation), and you have to couple back fine grid solutions to coarser regions of the

[2]The term "material parameter" implies that they are not subject to the flux equations.

mesh (restriction). By default, all ExaHyPE 2 Finite Volume solvers employ a piece-wise constant interpolation and simple averaging as restriction. If you require a better inter-grid transfer scheme or if you want to introduce your own one, you have to do the following steps:

- Make your solver (in Python) use another interpolation scheme. I present a variant here where you want to use linear interpolation:

```
my_solver._action_set_couple_resolution_transitions_and_handle_dynamic_mesh_refinement.
  switch_interpolation_scheme( "linear" )
```

- For a different restriction scheme use the routine `switch_interpolation_scheme()`.
- Call the solver's `create_action_sets()`. If you omit this call, your switch of the scheme is not "committed", i.e. taken into account (though subsequent changes of the solver state might in turn call `create_action_sets()` and thus implicitly make your switch known to the class).
- Depending on your Finite Volume scheme, you might have to change the size of the halo layer. The Rusanov solvers for example all work with a halo layer of 1. They don't need larger overlaps between neighbouring volumes, as the Rusanov Riemann solver only needs the solution left and right of a face. If you want to use a higher-order interpolation however, you need larger overlaps. In this case, you have to extend the halo layer used:

```
my_solver.overlap=2
```

Realisation and available schemes All the interpolation and restriction schemes that we offer can be found in the file `toolbox/blockstructured/Interpolation.h`. The documentation within this file[3] explains how the actual realisations work. ExaHyPE 2's philosophy is that you implement the inter-grid transfer operators within this header, and the Python API then helps you to pick from the routines therein.

At the moment, we have only implemented `averaging` for the restriction. For the interpolation, we offer the following routines:

scheme	description
`piecewise_constant`	Interpolate piece-wise constant (defa
`linear_with_constant_extrapolation`	Interpolate linear (2d) or bi-linear (and use constant extrapolation alon patch boundaries.
`linear_with_linear_extrapolation`	Interpolate linear (2d) or bi-linear (and use constant extrapolation alon patch boundaries.
`linear_with_constant_extrapolation_and_linear_normal_interpolation`	Variation of `constant_with_constar` where we interplate linearly betw (coarsened) voxel data and the adja grid values.
`linear_with_linear_extrapolation_and_linear_normal_interpolation`	Variation of `linear_with_constant_`

Postprocessing Some codes need to postprocess their interpolation data. At the moment, I only support volume-wise postprocessing. To enable it, you pass a function call to the DynamicAMR argument:

[3]We recommend to run `doxygen` on the sources as discussed in Section 2.8 and then to use a webbrowser to study the documentation. The interpolation/restriction details are discussed on a separate page which you find when you click on "Related Pages" within the generated webpage.

```
my_solver._action_set_couple_resolution_transitions_and_handle_dynamic_mesh_refinement.
  switch_point_wise_postprocessing_of_interpolation("""
  targetVolume[0] = 0.1;
  targetVolume[1] = 0.1;
  targetVolume[2] = std::max(targetVolume[2],0.0);
  """ )
```

Alternatively, just invoke a function directly:

```
my_solver._action_set_couple_resolution_transitions_and_handle_dynamic_mesh_refinement.
  switch_point_wise_postprocessing_of_interpolation(
  "examples::exahype2::ccz4::enforceCCZ4constraints(targetVolume)" )
```

With this variant, you trigger the interpolation of your choice, and, after that, each individual volume is subject to a function `enforceCCZ4constraints` which might change the value. As this `enforceCCZ4constraints` in this example is user-defined, ExaHyPE 2's core routines cannot know its definition. We manually have to add an include:

```
my_solver._action_set_couple_resolution_transitions_and_handle_dynamic_mesh_refinement.
 additional_includes += """
  #include "../CCZ4Kernels.h"
 """
```

`targetVolume` is a double pointer to the volume that has been interpolated. There are a few more values available for any function call you wanna make, but the pointer to the quantities usually is the most important one. You can use the attribute `additional_includes` to give the resulting source code access to other routines.

4.6 Modelling the fluxes and Riemann solvers with SymPy

You can model your fluxes, Riemann solvers and so forth with SymPy, i.e. through symbolic formulations—as long as you use a generic numerical Riemann solver such as Rusanov. The ExaHyPE 2 SymPy wrapper is held within the ExaHyPE 2 subpackage `exahype2.sympy`.

I provide a couple of classes for different PDE variants that should be compatible to particular ExaHyPE 2 Finite Volume and ADER-DG solvers. All SymPy classes wrap around SymPy's symbolic expressions, and thus allow users to work with these expressions directly. Once you have finished you modelling, the wrappers offer some code generation which again decorate SymPy's code generation, adds some glue code and can inject the generate into a Peano 4 solver.

The Euler equations modelled via this interface then add the following code snippet to the Python script that models Peano 4:

```
import sympy
import exahype2.solvers.sympy.FirstOrderConservativePDEFormulation



#
# Create a new instance of symbolic ExaHyPE PDE interface.
#
pde = exahype2.solvers.sympy.FirstOrderConservativePDEFormulation(unknowns = 5,dimensions = 3)

#
# Give entries in input vector symbolic names. We first declare some
# helpers/constants. Then we tell the solver how we would like to name the Q
# entries
#
gamma = sympy.symbols( "gamma")
rho = pde.name_Q_entry( 0, "rho" )
j = pde.name_Q_entries( 1, 3, "j" )
```

```
E = pde.name_Q_entry( 4, "E" )

#
# Finally, define the equation system
#
p = (gamma -1 ) *(E-1/2 *exahype2.solvers.sympy.dot(j,j) / rho)

pde.F[0,:] = j
pde.F[1:4,:] = 1/rho *exahype2.solvers.sympy.outer(j,j) + p *sympy.eye(3)
pde.F[4,:] = 1/rho *j *(E+p)

c = sympy.sqrt( gamma *p /rho )
pde.eigenvalue[0] = [ j[0]/rho -c, j[1]/rho -c, j[2]/rho -c ]
pde.eigenvalue[1] = [ j[0]/rho, j[1]/rho, j[2]/rho ]
pde.eigenvalue[2] = [ j[0]/rho, j[1]/rho, j[2]/rho ]
pde.eigenvalue[3] = [ j[0]/rho, j[1]/rho, j[2]/rho ]
pde.eigenvalue[4] = [ j[0]/rho + c, j[1]/rho + c, j[2]/rho + c ]

pde.substitute_expression( gamma, 1.4 )
```

This `pde` object here can create C++ code that we can plug directly into the respective Peano 4 routines:

```
my_solver.set_implementation( flux=pde.implementation_of_flux(),eigenvalues=...
```

With this call, the modelled PDE feeds, via code generation, directly into the ExaHyPE 2 solver. When we remodel the PDE, everything is updated automatically.

If you study the interface of `set_implementation`, you will notice that you can also model boundary or initial conditions this way. In this example, we however only modelled flux and eigenvalues. The remaining callbacks remain plain C++.

For further resources, please visit SymPy's homepage, and maybe you also want to have a look into projects like EinsteinPy or galgebra which provide tools for astrophysics or geometric algebra, respectively. As we rely on SymPy and expose all the SymPy expressions, they are compatible with ExaHyPE 2.

4.7 Adding tracers

To add tracer support for ExaHyPE 2, you first have to reconfigure our run and add the

```
./configure ... --enable-particles
```

Let `thesolver` be your ExaHyPE 2 solver instance in the Python script. Not all solvers currently support tracers (efficiently), but we are continuously improving the tracer support. Adding tracers is fairly simple. Each tracer has to have a name, and you also have to inform the project how many attributes you plan to track per tracer:

```
tracer_particles = project.add_tracer( name="MyTracer",attribute_count=2 )
```

This prepares ExaHyPE 2 to handle your tracer, and it also dumps tracers already with each snapshot, but obviously you haven't inserted any tracers yet. There are different inserters shipped with ExaHyPE 2. Alternatively, you might want to write your own, bespoke particle seed (see below). The snippet below inserts particles into your grid along a Cartesian topology and adds some random noise:

```
project.add_action_set_to_initialisation(
 exahype2.tracer.InsertParticlesAlongCartesianMesh(
  particle_set=tracer_particles,
```

```
  h=args.h/8.0, noise=True
))
```

With this additional seed, you can run your code and visualise the particles. However, these particles do not move at all yet or track the actual data of interest. To facilitate this, you have to map your solution onto the particle first. After that, you can decide to move the particles—though this is optional, i.e. the tracing API also allows you to have stationary trackers (probes).

```
project.add_action_set_to_timestepping(
 exahype2.tracer.FiniteVolumesTracing(tracer_particles,thesolver,[1,2],[0,4],1.1*patch_size
))
```

4.7.1 Dump tracer data into CSV database

The likely most important tool for many developers using only few particles is the fact that particles can dump their solution into a database:

```
project.add_action_set_to_timestepping(
 exahype2.tracer.DumpTrajectoryIntoDatabase(
  tracer_particles,thesolver,particle_spacing/10.0,"TracerDB"
))
```

Dumping tracers into a (CSV) database is attractive, as these files are easier to parse/postprocess than mesh data files. At the same time, you can dump particles more often than you plot data snapshots, and these dumps can even be adaptive, i.e. you dump if and only if a particle actually moves. The `DumpTrajectoryIntoDatabase` documentation provides more detail.

4.7.2 Write your own particle seed

If you want to write your own particle seed, I recommend that you copy over an existing one and adopt it to your need. Switch to `python/exahype2/tracer` and copy the class `InsertParticlesAlongCartesianMesh` over to modify it. The invocation within your Python application script then has to be altered, too:

```
project.add_action_set_to_initialisation( MyFancySeed(. ..) ))
```

The core ingredients in the seed is the code snippet that decides what happens if you touch a vertex for the first time:

```
  _Template_TouchVertexFirstTime = jinja2.Template("""
␣␣if␣(␣not␣marker.isRefined()␣)␣{
␣␣␣␣auto␣newParticles␣=
␣␣␣␣toolbox::particles::createEquallySpacedParticles<globaldata::{{PARTICLE}}>...
␣␣␣␣for␣(auto&␣p:␣newParticles)␣{
␣␣␣␣␣␣p->setNumber(0,␣spacetreeId);
␣␣␣␣␣␣p->setNumber(1,␣particleNumberOnThisTree);
␣␣␣␣␣␣particleNumberOnThisTree++;
␣␣␣␣}
␣␣␣␣fineGridVertex{{PARTICLES_CONTAINER}}.insert(
␣␣␣␣␣␣fineGridVertex{{PARTICLES_CONTAINER}}.end(),␣newParticles.begin(),
␣␣␣␣␣␣newParticles.end()
␣␣␣␣);
␣␣}
""")
```

The routine only triggers some activities for unrefined vertices. If a vertex is not refined further, it invokes the routine `createEquallySpacedParticles` from the particle toolbox which returns a

container over particles, and then takes these particles and inserts them into the domain. While you can alter this mechanism obviously, a modified, simpler seed can look like this:

```
  __Template_TouchVertexFirstTime = jinja2.Template("""
␣␣if␣(
␣␣␣␣not␣marker.isRefined()
␣␣␣␣and
␣␣␣␣marker.x(0)==...␣␣␣␣␣␣//␣check␣where␣the␣centre␣of␣this␣vertex␣is
␣␣␣␣and
␣␣␣␣marker.x(1)==...␣␣␣␣␣␣//␣check␣where␣the␣centre␣of␣this␣vertex␣is
␣␣␣␣and
␣␣␣␣marker.x(1)==...␣␣␣␣␣␣//␣check␣where␣the␣centre␣of␣this␣vertex␣is
␣␣␣␣and
␣␣␣␣marker.h(0)==...␣␣␣␣␣␣//␣check␣how␣big␣the␣area␣around␣this␣vertex␣is
␣␣␣␣and
␣␣␣␣...
␣␣)␣{
␣␣␣␣globaldata::{{PARTICLE}}*␣p␣=␣new␣globaldata::{{PARTICLE}}(␣...␣);
␣␣␣␣p->setNumber(0,␣spacetreeId);
␣␣␣␣p->setNumber(1,␣particleNumberOnThisTree);
␣␣␣␣p->setX(0,␣...␣);␣␣␣␣␣//␣x␣coordinate
␣␣␣␣p->setX(1,␣...␣);␣␣␣␣␣//␣y␣coordinate
␣␣␣␣p->setX(2,␣...␣);␣␣␣␣␣//␣z␣coordinate
␣␣␣␣p->setCutOffRadius(0.0);␣//␣you␣can␣force␣particles␣to␣certain␣resolution
␣␣␣␣␣␣
```

Links and further reading

- The "official" ExaHyPE release paper is

```
@article{Reinarz:2019:ExaHyPE,
  title = "ExaHyPE: An engine for parallel dynamically adaptive simulations of wave problems",
  journal = "Computer Physics Communications",
  pages = "107251",
  year = "2020",
  issn = "0010-4655",
  doi = "https://doi.org/10.1016/j.cpc.2020.107251",
  url = "http://www.sciencedirect.com/science/article/pii/S001046552030076X",
  author = "Anne Reinarz and Dominic E. Charrier and Michael Bader and Luke Bovard and Michael Dumbser
    and Kenneth Duru and Francesco Fambri and Alice-Agnes Gabriel and Jean-Matthieu Gallard and
    Sven Köppel and Lukas Krenz and Leonhard Rannabauer and Luciano Rezzolla and Philipp Samfass and
    Maurizio Tavelli and Tobias Weinzierl",
  keywords = "Hyperbolic, PDE, ADER-DG, Finite volumes, AMR, MPI, TBB, MPI+X",
}
```

  If you use the software, it would be great if you could cite this one.

- Every ExaHyPE 2 build yields a file `Readme...md` which contains a lot of literature that you might want to cite as well.
- The hyperbolic+elliptic coupling as discussed follows

```
@article{SchlottkeLakemper:2021:SelfGravitation,
title = {A purely hyperbolic discontinuous Galerkin approach for self-gravitating gas dynamics},
journal = {Journal of Computational Physics},
volume = {442},
pages = {110467},
year = {2021},
issn = {0021-9991},
doi = {https://doi.org/10.1016/j.jcp.2021.110467}
}
```

Chapter 5

SWIFT.pidt



SWIFT.pidt is a SWIFT (SPH With Inter-dependent Fine-grained Tasking) clone which combines the SWIFT SPH kernels (tasks) with Peano's particle-in-dual-tree concept. The original SWIFT codes is available from `https://swift.dur.ac.uk`. Peano 4's particle handling is a follow-up to the 2015 paper "Two Particle-in-Grid Realisations on Spacetrees" (`https://arxiv.org/abs/1508.02435`). It is discussed in detail in Section 8.

The integration of the SWIFT kernels and Peano 4 has been made possible by the EPSRC ExCALIBUR grants Massively Parallel Particle Hydrodynamics for Engineering and Astrophysics (EP/V001523/1) and PAX-SPH. SWIFT.pidt is realised as Python layer on top of Peano 4 and brings Peano 4's core, its particle toolsbox and SWIFT together.

A first Peano 4 pidt run (native usage of particle interface)

- Configure Peano 4 with the flags `--enable-particles`:

```
libtoolize; aclocal; autoconf; autoheader; cp src/config.h.in .; automake --add-missing
./configure --enable-particles
```

  These are the two mandatory extensions. You might want to add further options.
- Recompile your Peano 4 code. This should give you a couple of `ToolboxParticles` libraries in the `src/toolbox/particles` subdirectory.
- Ensure that `python` subdirectory is in your Python search path and
- Yet to be written

5.1 Set up the simulator

Our interface for SWIFT.pidt is almost complete realised in Python. Only specialised potentials have to be injected via C++.

5.2 Introduce a new particle

The generation of a particle is straightforward. We employ a tool called DaStGen2 which really just is a set of Python classes which represent the attributes of a C++ class. We build up a set of Python objects and SWIFT.pidt eventually dumps them into C++ code from which it is embedded into the actual simulator.

```
import dastgen2

particle = peano4.toolbox.particles.Particle("Particle")

v_attr = peano4.dastgen2.Peano4DoubleArray("v","Dimensions")
mass_attr = dastgen2.attributes.Double("mass")

particle.data.add_attribute(v_attr)
particle.data.add_attribute(mass_attr)


@todo something fancy in here that adds a new species
```

R Though conceptionally a plain C++ pretty printer, the DaStGen2 Python API can create some fancy C++ annotations. You can, for example, specify ranges in which integers live, or you can prescribe the precision you wanna have for floating point values. If you have a specialised LLVM compiler, these annotations will be used to generate aggressively optimised code.

D In SWIFT.pidt's *dual tree* storage scheme, particles are always stored to their nearest neighbour.

5.3 IO control

Peano 4 dumps a lot of information to the terminal by default. Consult Section 11.4 for some remarks on how to filter the output properly.

5.4 High-performance computing

To use Peano 4's high-performance computing features, you have to configure your baseline code properly. That is, you need to invoke `./configure` with an appropriate `--with-multithreading=...` call to enable multithreading, you need to invoke it with `--with-mpi=...` for MPI support, and you need to specify `--enable-gpus=...` if you want to use accelerators. Furthermore, you will have to enable Peano 4's load balancing toolbox (`--enable-loadbalancing`), unless you want to implement load balancing all on your own.

Links and further reading

- The "official" pidt paper is

```
@article{WEINZIERL201642,
title = "Two particle-in-grid realisations on spacetrees",
journal = "Parallel Computing",
volume = "52",
pages = "42 - 64",
year = "2016",
issn = "0167-8191",
doi = "https://doi.org/10.1016/j.parco.2015.12.007",
```

```
url = "http://www.sciencedirect.com/science/article/pii/S0167819115001635",
author = "T. Weinzierl and B. Verleye and P. Henri and D. Roose",
keywords = "Particle-in-cell, Spacetree, Particle sorting, AMR, Lagrangian{Eulerian methods, Communication-avoiding"
}
```

If you use the software, it would be great if you could cite this one.

Part II

Examples: Using the high-level Python API

Chapter 6

Introduction

- What is a action set
- How to set up a project
- ...

6.0.1 Parallelisation and GPU support

Follow the configuration recommendation of Section 5.4 before you continue.

Domain decomposition

All domain decomposition is realised via your main routine, and you don't have to modify anything besides your main loop if you work shared memory only. Once you enable MPI, too, you will have to make some minor modifications. Not sure bout this

Peano 4 allows you to control the domain decomposition via explicit splits (and mergers) of subdomains. However, we rely on the load balancing toolbox here. The toolbox collects some global information and then triggers splits and merges accordingly. You have to perform the following steps:

- Include

```
#include "toolbox/loadbalancing/RecursiveSubdivision.h"
```

- Create an instance of the load balancer on each MPI rank. It is convenient to insert

```
toolbox::loadbalancing::RecursiveSubdivision loadBalancer;
```

  just before the initial `tarch::mpi::Rank::getInstance().isGlobalMaster()` check. The object accepts a lot of configuration parameters which you can look up in the source code documentation.

- You have to inform your load balancer when an individual step has terminated. This tells the balancer to collect some global statistics and (eventually) to trigger some rebalancing.

```
loadBalancer.finishStep();
```

I typically use the load balancer only within the grid construction. Totally dynamic load balancing is something this simple pre-manufactured balancer doesn't properly support anyway.

```
int gridConstructionSteps = 0;
while (peano4::parallel::SpacetreeSet::getInstance().getGridStatistics().getStationarySweeps()<5) {
  examples::particles::observers::CreateGrid createObserver;
  peano4::parallel::SpacetreeSet::getInstance().traverse(createObserver);
  gridConstructionSteps++;
  loadBalancer.finishStep();
}
```

Chapter 7

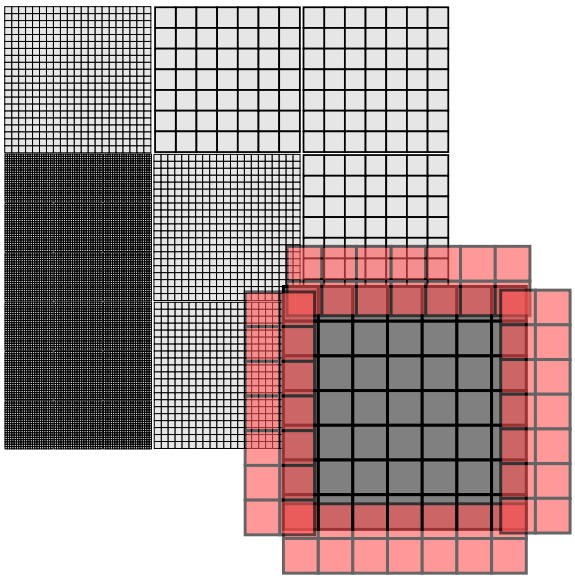
A block-structured AMR code for Finite Volumes

We create a simple Finite Volumes solver here. The examples starts with a simple regular grid, parallelises this grid, and finally adds adaptive mesh refinment (AMR). It employs a block-structured formalism and relies heavily on premanufactured actions and solver ingredients from the ExaHyPE2/ExaClaw project.

R Writing all the Finite Volume (FV) core routines manually is a tedious and time-consuming process despite the fact that I do offer a toolbox for block-structured code. For most applications, I therefore recommend to realise FV codes through ExaHyPE instead.

7.1 Setting up the data structures

We model our grid as an AMR grid consisting of $N \times N$ blocks. We call these blocks *patches*. In the example below, we have chosen $N = 7$. In general, odd choices of N are advantageous once you enable AMR as we will discuss later.



```
project = peano4.Project( ["examples", "finitevolumes"], "." )
patch_size = 7
unknowns = 5
patch = peano4.datamodel.Patch( (patch_size,patch_size,patch_size), unknowns, "Q" )
project.datamodel.add_cell(patch)
```

In the above example, every volume within the $N \times N$ patches holds five unknowns. In general, I prefer the term volume here, as cell again is ambigous given that we construct the Peano 4 host mesh from cells.

With one patch per cell, we could traverse the mesh and do something per cell. This however is of limited value. We have to couple the phenomena what is going on in neighbouring cells. Peano 4 realises a strict element-wise traversal, i.e. there's no way to access the neighbour cell of a cell directly. However, we can hijack the faces.

The idea here is that we embed a $2 \times N$ ($d = 2$) or $2 \times N \times N$ ($d = 3$), respectively, patch into each face. Let this auxiliary patch overlap the adjacent cells. Then, we effectively have a halo of one cell available within each cell: We know the cell data. We also have access to the $2d$ faces where each hosts a degenerated patch. One later of this patch is a copy of our own data, i.e. does not give us additional information. The other layer of the auxiliary patch however holds data from the neighbour. It gives us information from the neighbour patch.

In theory, we could only replicate those quantities that we really need. But it makes our live easier to just hold all five quantities in the auxiliary face data structures, too:

```
patch_overlap = peano4.datamodel.Patch( (2,patch_size,patch_size), unknowns, "Q" )
project.datamodel.add_face(patch_overlap)
```

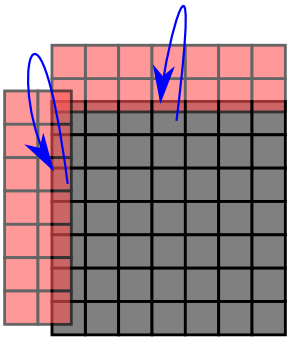
Before we generate the code, we export some of the Python constants that we use into C/C++, so we have it available there, too:

```
project.constants.export( "PatchSize", patch_size )
project.constants.export( "NumberOfUnknownsPerCell", unknowns )
```

7.2 Administration of the data structures

Projecting the patches onto the face data structures and back is a mechnical task. Therefore, Peano 4 offers a toolbox to relieve you from the pain to recode it over and over again. Using this toolbox, you add the projections to your algorithmic steps, and the API then automatically injects these features (aspects) into your code:

```
perform_time_step = peano4.solversteps.Step( "TimeStep" )
perform_time_step.use_cell(patch)
perform_time_step.use_face(patch_overlap)
...
perform_time_step.add_action_set(
  peano4.toolbox.blockstructured.ReconstructPatchAndApplyFunctor(patch,patch_overlap,...)
)
perform_time_step.add_action_set(
  peano4.toolbox.blockstructured.ProjectPatchOntoFaces(patch,patch_overlap)
)
```



The image above illustrates what `ProjectPatchOntoFaces` does: It knows the dimensions of both the patches and the face auxiliary data structures and thus can ensure that the right data is copied from the cell into the 2^d faces when we leave a cell throughout the grid traversal.

The action set `ReconstructPatchAndApplyFunctor` works slighlty different yet can be read, from a patch projection point of view, as transpose of `ProjectPatchOntoFaces`: It creates a auxiliary variable `reconstructedX` with X being the name you gave the Unknowns of the patch. This auxiliary variable has the dimensions $N + 2 \times N + 2 \times N + 2$[1]. It then copies over the patch data into this auxiliary patch and uses the faces to supplement it with halo data around it. So that it, you get the original patch data plus the cells around it in one big patch.

7.3 A manually implemented FV scheme

As the name suggests, the reconstruction step is typically combined with the invocation of some computation on the patch. We give a simple example of a Finite Volume solver for the Euler equations here.

Once a patch is reconstructed, we have a huge double array (AoS) of the reconstructed patch, i.e. the patch plus its surrounding halo, available. It holds a copy of the actual patch data. This patch data is still available as well, so we can use the reconstructed array as a preimage and write any outcome into the patch data structure.

We begin with defining a two simple functions that implement the PDE—we need the flux function and the eigenvalues—and then write a simple 1d Rusanov flux. I use some lambda notation here when I write the Rusanov flux, as I want to make it generic, i.e. it accepts a generic flux and eigenvalue realisation realised somewhere else in my code.



```
// All in besides lambda which is out
void eigenvalues(
  double Q[5],
  const tarch::la::Vector<Dimensions,double>& faceCentre,
  const tarch::la::Vector<Dimensions,double>& volumeH,
  double t,
  int normal,
  double lambda[5]
) {
  ...
}


// All in besides F which is out
void flux(
  double Q[5],
  const tarch::la::Vector<Dimensions,double>& faceCentre,
```

[1] All tools work with overlaps greater than 2. But to keep the text simple, I stick to 2.

```
  const tarch::la::Vector<Dimensions,double>& volumeH,
  double t,
  int normal,
  double F[5]
) {
  ...
};
```

```
auto splitRiemann1d = [&flux, &eigenvalues](
  double QL[5], double QR[5], const tarch::la::Vector<Dimensions,double>& x,
  double dx, double dt, int normal, double F[5]
) -> void {
  assertion(normal>=0);
  assertion(normal<Dimensions);

  double FL[unknowns];
  double FR[unknowns];
  flux(QL,x,dx,t,dt,normal,FL);
  flux(QR,x,dx,t,dt,normal,FR);

  double lambdas[unknowns];
  double lambdaMax = 0.0;

  eigenvalues(QL,x,dx,t,dt,normal,lambdas);
  for (int unknown=0; unknown<unknowns; unknown++) {
    assertion(lambdas[unknown]==lambdas[unknown]);
    lambdaMax = std::max(lambdaMax,lambdas[unknown]);
  }
  eigenvalues(QR,x,dx,t,dt,normal % Dimensions,lambdas);
  for (int unknown=0; unknown<unknowns; unknown++) {
    assertion(lambdas[unknown]==lambdas[unknown]);
    lambdaMax = std::max(lambdaMax,lambdas[unknown]);
  }

  for (int unknown=0; unknown<unknowns; unknown++) {
    F[unknown] = 0.5 *FL[unknown]
                 + 0.5 *FR[unknown]
                 -0.5 *lambdaMax *(QR[unknown] -QL[unknown]); }
};
```



An implementation of these routines can either be done manually (download/copy the code snippets), or you can use ExaHyPE 2. ExaHyPE 2 does provide generic implementations of these guys in its directories.

Let the three functions be one string assigned to a variable `functor` in our Python API. We now have to run over the reconstructed patch with $N+2 \times N+2$ cells and write it back into the actual user data with cardinality $N \times N$.

There are lots of options how to do this. We can use Peano's dimension-generic volumes, or we can write it down for our particular dimension. We can loop over the faces (similar to ClawPack) or loop over the cells and compute the Riemann problems twice per face. Many more options do arise. I don't want to show some codes how to do it here, but I link to the directory `src/exahype2/fv` where I've realised a couple of them.

Whatever implementation variant I use, it is convenient now to rely on `ReconstructPatchAndApplyFunctor`. I create, in Python, a string that calls my Rusanov solver:

```
  myApplyRusanovToPatch( ..., reconstructedPatch, originalPatch );
```

This string can hand over all of the cell attributes that I've introduced to my `myApplyRusanovToPatch` implementation. The important thing is, it also forwards two double pointer (`reconstructedPatch` and `originalPatch`) to my routine. These are the plain user records.

The string holding this routine is passed into the instantiation of the type `ApplyFunctorOnPatch` and thus is injected into the generated code.

7.4 Plot results

For the plotting of patch-based AMR, Peano 4 provides predefined action sets, too. They dump the output into Peano's patch format. Chapter B describes this format. If you build your code with VTK support, Peano 4 offers a conversion script into VTK.

To use the predefined file dump feature, add an instance of `PlotPatchesInPeanoBlockFormat` to the step of your choice. It then looks similar to

```
print_solution = peano4.solversteps.Step( "PlotSolution" )
print_solution.use_cell(patch)
print_solution.use_face(patch_overlap)
print_solution.remove_all_actions()
plotter = peano4.toolbox.blockstructured.PlotPatchesInPeanoBlockFormat("solution",patch,"Q")
print_solution.add_action_set( plotter )
project.solversteps.add_step(print_solution)
```

Chapter 8

Peano 4's particle support

Peano 4's particle handling is a follow-up to the 2015 paper "Two Particle-in-Grid Realisations on Spacetrees" (`https://arxiv.org/abs/1508.02435`).

A first Peano 4 pidt run (native usage of particle interface)

- Configure Peano 4 with the flags `--enable-particles`:

```
libtoolize; aclocal; autoconf; autoheader; cp src/config.h.in .; automake --add-missing
./configure --enable-particles
```

  These are the two mandatory extensions. You might want to add further options.

- Recompile your Peano 4 code. This should give you a couple of `ToolboxParticles` libraries in the `src/toolbox/particles` subdirectory.
- Ensure that `python` subdirectory is in your Python search path and
- run the Jupyter notebook in `examples/particles/sph`:

```
export PYTHONPATH=../../../python/
jupyter-notebook peano-sph.ipynb
```

Peano 4's particle code is lightweight. There are some C++ building blocks, but the majority of its features are available through the Python API as `toolbox.particles`. You will need

```
import peano4.toolbox.particles
```

I assume that you are familiar from hereon with Peano 4's basics, i.e. you know how to create a project,

8.1 Introduce a new particle

With the Python API, the generation of a particle is straightforward:

```
import dastgen2
```

```
particle = peano4.toolbox.particles.Particle("Particle")

v_attr = peano4.dastgen2.Peano4DoubleArray("v","Dimensions")
mass_attr = dastgen2.attributes.Double("mass")

particle.data.add_attribute(v_attr)
particle.data.add_attribute(mass_attr)

particles = peano4.toolbox.particles.ParticleSet(particle)
```

R Though conceptionally a plain C++ pretty printer, the DaStGen2 Python API can create some fancy C++ annotations. You can, for example, specify ranges in which integers live, or you can prescribe the precision you wanna have for floating point values. If you have a specialised LLVM compiler, these annotations will be used to generate aggressively optimised code.

The Peano 4 particle has some additional attributes as they are discussed in Section **??** and thereafter. For the user, it is usually sufficient to recognise that each particle by default has a position `x` and and a cut-off radius `cutOffRadius`. You can query and modify both values through `setX`, `getX`, `getCutOffRadius` and `setCutOffRadius`, respectively.

We next add the particles to the project and we notify the vertices that there are particles which have to be administered. I discuss why and how pidt uses vertices in Section **??** though users might (initially) skip such background info.

```
project.datamodel.add_global_object(particle)
project.datamodel.add_vertex(particles)
```

Once you have added the particle and the particle container `particles` to your Peano 4 datamodel, you have to inform all solver steps that they have to administer the particles.

```
create_grid.use_vertex(particles)
```

Let's assume that our project is spanning the namespace `examples::someparticles`. In the example above where our particle is called `Particle`, we will end up with two classes: `examples::someparticles::globaldata::Particle` is the actual particle class. When you open the generated C++ code, you'll recognise that it is a 1:1 mapping of the Python attributes onto a C++ class with some additional routines used by Peano 4 internally. There is a further class `examples::someparticles::vertexdata::ParticleSet`. As the name says, this is the actual data associated with each vertex. It is a specialisation of `std::vector` over pointers to `examples::someparticles::globaldata::Particle`.

We summarise that each vertex holds a vector to pointers over particles. The actual particles reside on the heap. From hereon, you introduce, as with any Peano 4 project additional data fields, action sets, and so forth, and you write you main to actually control the program flow. We assume from hereon that the reader is familiar with the concept of an program phase (algorithmic step) mapped onto an action set.

8.2 Insert particles into domain

To create a particle, you in principle just allocate it on the heap, and then you add it to its nearest vertex. Furthermore, there are some mandatory attributes that aways have to be set. Therefore, I recommend that you use the toolbox' initialisation routine to set up particles correctly.

In principle, it does not matter where you create the particle. You can do so in `touchCellFirstTime()` or in a cell event. I usually use a cell event and I rely on the helper function `insertParticleIntoCell` to take care of the vertex association:

```
#include "toolbox/particles/ParticleFactory.h"



...::touchCellFirstTime(
  const peano4::datamanagement::CellMarker& marker,
  peano4::datamanagement::VertexEnumerator<...::vertexdata::ParticleSet> fineGridVerticesParticleSet,
  ...
  peano4::datamanagement::VertexEnumerator<...::vertexdata::ParticleSet> coarseGridVerticesParticleSet,
  ...
) {
examples::someparticles::globaldata::Particle* myNewParticle =
  new examples::someparticles::globaldata::Particle();

// assume x and cutOffRadius are set to something meaningful
toolbox::particles::init( *myNewParticle, x, cutOffRadius );

// here's your user code's initialisation
myNewParticle->setXXX(...);

toolbox::particles::insertParticleIntoCell(
  marker,
  myNewParticle,
  fineGridVerticesParticleSet
);
}
```

There is no need to write such initialisation code yourself. Peano 4's particle toolbox provides a couple of pre-defined particle generators that you can adopt to your needs. Probably the most popular particle seeds are

- `InsertParticlesAlongCartesianLayoutIntoUnrefinedCells`
- `InsertRandomParticlesIntoUnrefinedCells`

8.3 Dual trees

D Particles in our *dual tree storage scheme* are stored within the spacetree and associated to vertices:

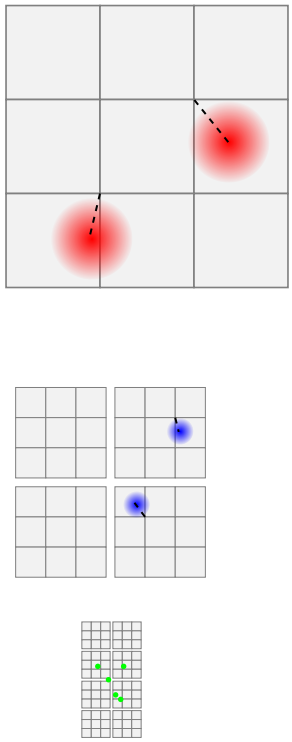
- A particle resides on a mesh level within the tree such that the cut off radius $r \leq h$ if h is the resolution of this level.
- A particle is associated to the vertex of this level that's closest to its centre x.

These two conventions effectively span a dual grid over the spacetree if you think of particles to be stored within cells: Just impose a grid on top of your actual grid which is shifted by half a mesh lenght. Particles are contained within these shifted grid's cells. Alternatively, you can think of the vertices of the tree spanning a Voronoi diagram, and particles are then held by the cells of this Voronoi diagram.

R Our storage is vertex-oriented whereas most particle literature stores particles within cells. If we think of a dual multiscale mesh, i.e. a mesh that is shifed by $h/2$ alon each coordinate axis, then Peano 4's approach also works with cells. Such shifted meshes are often called *dual* meshes and I therefore speak of a *dual tree*.

If you add the action set `peano4.toolbox.particles.PlotParticlesInVTKFormat` to your algorithm step, the resulting file will contain all associativity information. You can plot to which vertex a particle belongs to.

Here's a hand-written cartoon to explain the particle storage:



A tree with three levels. Particles with a large cut-off radius (red) are held on rather coarse levels. Particles with very small cut-off are sorted (dropped) into the fine grid levels. Each particle is always associated to its closest vertex (dotted line). It is the mesh which holds (owns) the vertices. The toolset's predefined action set can automatically maintain/update these associations after each grid sweep. It also moves particles up and down the grid hierarchy if the particles' cut-off radius changes or the grid changes.

8.4 Move particles

To move a particle, all you have to do is to alter its position:

```
myParticle->setX( {17.3, 8.2} );
```

Obviously, any change of a particle position could mean that the particle is no longer associated to its nearest vertex. We have to sort them.

Peano 4 automatically maintains these relations if you add the action set `peano4.toolbox.particles.UpdateParti` to your algorithmic steps. As the convention relies on a smaller-equals sign, adaptive meshes are automatically supported. If particles travel through your domain and the mesh is not very fine, they end up in the tree leaves. If particles have a large cut-off radius, they are stored in rather coarse levels of the tree.

If you change the cut-off radius or the position of a particle, the particle will stay within its current cell/vertex within the tree. In the next grid sweep, it will however we held by the "correct" vertex, i.e. its grid-vertex association will be updated: The action set `UpdateParticleGridAssociation` updates the particle-tree association, i.e. sorts the particles, on-the-fly between two mesh traversals.

While you do not have to worry about how this sorting is realised, it is important to study the action set's documentation (just open the Python code) and read when data is consistent. Here is the general invariant however:

- The very moment you alter a particle's position, you cannot be sure that it is still associated with the nearest vertex. Consequently, neighbourhood searchers (cmp. Section 8.5) might yield invalid results.
- In the grid sweep after the position change, the particle will be associated with the correct vertex (unless you change its position once again obviously).

8.5 Dual tree traversal and particle-particle interaction



In Peano 4's *dual tree* storage scheme, we always hold the complete spacetree with all tree (grid) levels. Particles are always stored on the finest mesh (tree) level where the mesh size is bigger than the particle's cut-off radius. Different particles with different cut-off radii might thus be stored on different tree levels.

Peano 4 always runs through the mesh top-down, i.e. it traverses the spacetree in a depth-first fashion. With the definition above, we can realise particle-particle interactions as follows:

Per cell, the code assembles a `local set`. This is a list of pointers to those particles that are associated to a cell's 2^d vertices. With the local set, you can make particles held on one resolution level interact with each other. You get an cubic area of $2h$ side-lenght per cell: As a particle is always associated to its closest vertex, the local set holds all particles whose cut-off radius overlap with the cell.

Adaptivity and particles with different cut-off are not yet captured through the local set notion. There's thus a second set that I call active set. The active set is the local set plus all local sets that have been active on coarser levels within the tree.



The active set is built up top-down while we run through the tree. So it is a data structure that grows and shrinks while you run through the meshes. The local set `localParticles` is created on-the-fly every time you hit a cell or a vertex. Do not alter the active set: You can modify the values of the particles within an active set, but you should not add or remove entries from it. The local set can be modified.

If you use the `toolbox.particles.ParticleParticleInteraction` action set, you pass it some C source code that works with the particles. This C code has access to three variables: the active set, the local set and the cell marker (of type `peano4::datatraversal::CellMarker`). Let's assume that each particle carries some velocity and that we could cut-off gravity—which we clearly should not.

The straightforward particle-particle interaction then looks conceptionally as follows:

```
// Run over local set
for (auto& p: localParticles) {
  // Pick only those particles that reside within the current cell, otherwise
  // we would update each particle up to 2^d times. I do not exploit the
  // symmetry of forces here
  if ( marker.isContained( p->getX() ) ) {
    for (auto& pp: activeParticles) {
      // No interaction with particle itself
      if (p!=pp) {
        tarch::la::Vector<Dimensions,double> dist = pp->getX() -p->getX();
        [...]
      }
    }
```

```
  }
}
```

Typical Peano 4 codes introduce some variables per particle into which they accumulate data. In `touchVertexFirstTime` we use a vertex for the very first time. Consequently, we also read the particles associated to this vertex for the first time and thus set the accumulation variable to the neutral element 0. Within `touchCellFirstTime`, we now evaluate particle-particle interactions and write data to the accumulated values. In `touchVertexLastTime`, we finally have all data accumulated and hence can update the particle state.



Study the sort algorithms that we use if you update the vertex (cmp. Section **??**). Once you alter a particle position, particles might not be tied to their closest vertex anymore. Neighbourhood searches thus might yield invalid results.

8.6 Further action sets

There are a few further action sets in the toolbox that help you to write your Lagrangian code:

1. The action set `toolbox.particles.PlotParticlesInVTKFormat` provides a basic particle plotter which dumps particles, their cut-off radius plus their particle-vertex associativity. If you have to dump further properties, you have to write your own plotter. The generated plotter however can serve as a blueprint.
2. The action set `toolbox.particles.ParticleTreeAnalysis` provides some fundamental tree analysis such as a multiscale counting how many particles reside within each cell. You'll need some additional data structures (per cell) to use this action set.
3. The action set `toolbox.particles.ParticleAMR` uses the tree analysis to guide dynamic grid adaptivity. You can specify how many particles are held per cell at least, e.g., while the cut-off radius feeds naturally into the AMR.

Links and further reading

- The "official" pidt paper is

```
@article{WEINZIERL201642,
title = "Two particle-in-grid realisations on spacetrees",
journal = "Parallel Computing",
volume = "52",
pages = "42 - 64",
year = "2016",
issn = "0167-8191",
doi = "https://doi.org/10.1016/j.parco.2015.12.007",
url = "http://www.sciencedirect.com/science/article/pii/S0167819115001635",
author = "T. Weinzierl and B. Verleye and P. Henri and D. Roose",
keywords = "Particle-in-cell, Spacetree, Particle sorting, AMR, Lagrangian{Eulerian methods, Communication-avoiding"
}
```

  If you use the software, it would be great if you could cite this one.

Chapter 9

A geometric-algebraic multigrid solver

All the examples from this chapter can be found in Peano's `python/examples` directory.

9.1 A matrix-free Jacobi smoother with rediscretisation



This example corresponds to the `matrix-free.py` script.

9.2 A parallel version of the code



This example corresponds to the `amr-parallel.py` script.

For the time being, we again keep stuff pretty simple. We stick to a regular grid, create this grid serially on one rank or core, and then distribute it. That is, we don't do any load balancing and, as long as the grid remains stationary, we will have a brilliant balancing. For the remainder, the text will use rank/core/partition as synonyms. Peano 4 does not distinguish between them, i.e. it uses domain decomposition to keep both ranks and threads busy.

The first step to integrate multicore parallelisation—I always recommend to start with multithreading first and then to add MPI on top—is to set an appropriate number of threads. Different to OpenMP, Peano requires the user to configure the thread count within the code. This is done via the configuration method of the singleton `tarch::multicore::Core`[1].

To split up a tree, you have to call

```
peano4::parallel::SpacetreeSet::getInstance().split(mytreeNumber,cellsPerCore,destinationRank)
```

on your rank. This operation wants to know which spacetree to split, how many cells to cut from it (we always count fine grid cells), and to which rank to deploy the split tree. The operation establishes a logical tree topology on the spacetrees, as we always start with tree number 0 on rank 0 and then fork and fork and fork. Joins are currently not supported by Peano 4's API: the code automatically fuses trees if they degenerate.

[1] Peano's C++14 multithreading variant at the moment does not allow you to set the thread count, as C++ offers no such feature. I therefore recommend TBB.

After each split, you need at least two iterations to actually realise it, as Peano has to move the data, update all data meta information, and so forth. The iterations (grid sweeps) are no special ones, i.e. you can continue to compute. It is just reasonable to keep in mind that after a split of a rank, this rank will not be able to do any further domain decomposition for a few more sweeps until it has "recovered". Likewisely, new trees will not enter the game immediately, but with a 1–2 iterations delay. If you want to avoid that computations and domain decomposition intermix, you can add code snippets alike

```
int cellsPerCore = peano4::parallel::SpacetreeSet::getInstance().getGridStatistics().getNumberOfLocalUnrefinedCells() / coreCount;
logInfo( "main()", "should host " << cellsPerCore << " cells per core" );

for (int thread=1; thread<coreCount; thread++) {
  if ( not peano4::parallel::SpacetreeSet::getInstance().split(0,cellsPerCore,0)) {
   logWarning( "runParallel(...)", "failed to assign thread " << thread << " " << cellsPerCore << " cell(s)" );
  }
}
```

This is however your decision. Peano does not impose any constraints on you that you have to separate load balancing, load re-distribution and computations.

9.3 A parallel AMR Jacobi solver

R This example corresponds to `amr-parallel.py` script, too.

Part III

Low level (C/C++) programming

Peano 4's core is a plain C++ application using MPI plus a shared memory library. It can add support for HDF5, vtk, and so forth; depends how your code is configured. These underlying libraries/APIs are hidden away from user code by the core. At the same time, we provide a Python API which hides away most of the core. At one point, you will however be required to program Peano 4 directly with C++.

In the following sections, I run through some of the C++ techniques and features of Peano 4. Many users will never see the majority of them, as they are hidden away—in particular if you work with high-level wrappers around Peano 4 such as ExaHyPE 2. However, some low-level stuff is interesting for everybody: How to use the logging and tracing, what support of unit tests does exist, or how to program dimension-generic loops. Other stuff is more techi, and serves more as a documentation guideline for core developers. Yet, they also provide rationale why certain things in Peano 4 behave the way they do, once you have a large application up and running.

Chapter 10

Guiding principles, high-level concepts and design decisions

10.1 Generic structure of a Peano 4 application

Though I do not tell users exactly how to write their code, there's some kind of generic structure that is common to all Peano simulations. We can consider it to be some kind of pattern/best practice.

First, we distinguish the global master—typically rank 0—and all the other ranks. It is the global master that makes all the decisions and hosts the main logic. It tells the other ranks what to do in each step. Typical actions are

- run through your mesh once,
- split up the domain (though each rank can decide to do this totally independently),
- plot some data, or, eventually,
- terminate.

Therefore, the global master basically sets a counter/id which uniquely identifies which step to run next. It then says "go ahead" and runs this step itself. All the other ranks are noticed (via an internal broadcast in the core) which step to run, do the same thing, and then wait for the next wake-up call. At the end, the global master sets the id of a terminate. As all the ranks loop until they receive this terminate id, they do go down as well in the end.

A typical main thus looks as follows

```
if (tarch::mpi::Rank::getInstance().isGlobalMaster() ) {
  while ( selectNextAlgorithmicStep() ) {
    step();
  }
}
else {
  while (peano4::parallel::Node::getInstance().continueToRun()) {
    step();
  }
}
```

where the routine `selectNextAlgorithmicStep()` issues one call of

```
peano4::parallel::Node::getInstance().setNextProgramStep(myStepNumber);
```

The `step()` routine then is usually a big switch statement:

```
int stepIdentifier = peano4::parallel::Node::getInstance().getCurrentProgramStep();
switch (stepIdentifier) {
  case 0:
    {
      observers::CreateGrid observer;
      peano4::parallel::SpacetreeSet::getInstance().traverse(observer);
    }
    break;
  case 1:
    {
      // split up domain
    }
}
```

Nothing stops you from triggering multiple traversals (or none at all) within a step.



On the highest level, Peano4 implements a BSPish structure where the global rank issues a step and then all ranks run this step simultaneously. You can however define the notion of a superstep, i.e. make one superstep issue multiple grid traversals. It is just important that all spacetree sets do exactly the same number of iterations.

The numbers within the case statements are kind of magic constants and you can redefine them. However, I recommend to read the documentation of `setNextProgramStep()` (there are some predefined constants that you should not use), and I recommend that you use the predefined enums in `observers::StepRepository` if you work with the Python API, too. In this repository, there are predefined integers for all of the algorithm steps you've defined.

With the above structure, the Peano execution trace is close to trivial: You simply run through your grid n times and after each step, the core exchanges all of the data.

10.2 MPI and shared memory programming

If you run Peano4 with MPI, you buy into MPI's SPMD paradigm, i.e. every single rank hosts one instance of Peano. Each Peano instance in return hosts multiple subspacetrees, i.e. multiscale domain partitions. It is the user's responsibility to ensure that the ranks do coordinate with each other. That is, the user has to ensure that whenever you run a certain type of grid sweep on one rank, then the other ranks run this sweep as well. For this, each rank has to host one spacetree set, and these sets have to be configured with the same parameters (domain offset plus domain size). I do provide Python interfaces/APIs to model this behaviour, but it might be reasonable to understand how it works.

A typical Peano4 code distinguishes the global master (rank 0) from the workers (all other ranks). The global master hosts all the program logic, i.e. decides which steps to run in which order. There's no reason for you not to emancipate from this pattern, but it has proven of value. The main of this pattern always looks similar to

```
  if (tarch::mpi::Rank::getInstance().isGlobalMaster() ) {
    // All the program logic, i.e. all decisions here
  }
  else {
    // React to decisions what to do next on all other ranks
  }
```

Before we start, lets assume that each individual step (type of run through the mesh) has a unique positive number.

A typical parallel grid sweep looks as follows:

1.

10.2.1 Realising a domain decomposition

This is a howto for domain decomposition. For details about the underlying algorithmics and design decisions, please consult Chapter 16. As mentioned in various places, Peano does not fundamentally distinguish shared memory and distributed memory parallelisation unless you work explicitly with tasks. That is, all split remarks here do also apply to shared memory.

Peano's core domain decomposition contact point is the singleton `SpacetreeSet` on each compute node (rank). You can tell this set to decompose one of the local spacetrees further through its `split` routine. After each split, the involved spacetrees need at least two iterations to "recover" before they are available for further spilts.

Split accepts the number of fine grid cells that are two be given away to another rank. The domain decomposition follows the Peano space-filling curve. If 3^d cells along this curve are to be deployed to a different rank, then Peano also deploys the parent of this 3^d assembly to a remote rank. With dynamic AMR (or throughout an initial grid construction), it can happen that Peano runs into a cell that is to be moved over to another rank and is refined at the same time. In this case, the cell plus all of its children is deployed.

R Splits imply that some spacetree data has to be copied and (temporarily) is held redundantly. In principle, you can issue as many splits in one rush as you like. In practice, it often makes sense to constrain the maximum number of splits issued per grid traversal. This ensures that you don't run out of memory. Also, splits require the Peano core to run some steps serially. So also from a time-to-solution point of view, limiting the number of splits (in particular if they affect the same rank) is a good idea.

The issue with (real) dynamic AMR is that it is very difficult to know how many cells will be around after a grid sweep. The same holds for the grid construction. For runtime and memory reasons, you don't want to build up the grid completely before you start to decompose it. The following realisation pattern has turned out to be advantageous:

- Build up an initial grid on the first rank up to a certain, reasonable, grid level. It should host at least $3^d \cdot r$ fine grid cells if you have r ranks. This seems to be a reasonable rule of thumb.
- Split up this initial grid into r spacetrees where each spacetrees is associated with a node of its own.
- Wait for one tree traversal, i.e. run through the grid without adding further mesh elements. This ensures that the distributed mesh has been migrated completely before we continue.
- Continue to build up the grid.
- Compute a good guess how many cells should roughly end up per core. Store this guess.
- While the grid is constructed, decompose it further.
- Once the grid is complete, recompute how many cells should end up per core. Split up all the trees that are too big further and distribute these further trees among the ranks.

R Irgendwie darf man net 2x traverse aufrufen in meinem Code. Evtl. liegt das aber auch am Datenaustausch. Aber ist halt einfach bad smell in meinem Code!

10.2.2 Global communication

Peano 4 handles all (multiscale) boundary data transfer and data migration within the mesh. Everything not tied to the mesh is *not* handled by Peano 4. That means there's not even a reduction of the grid statistics over MPI. We do provide reduction and broadcast features however. If you need these global data, I recommend that you realise all reductions and broadcasts in your main rather than inside of the Peano 4 mappers or other classes.

Chapter 11

Logging, tracing and assertions

11.1 Logging, tracing and assertion architecture

Peano is built upon a technical architecture (`tarch`) which provides logging, tracing and assertions. The two directory of relevance here is therefore `tarch/logging` plus the file `Assertions.h`. The latter file holds all the assertion macros, while the logging directory holds a file `Log.h` which holds all of the essential logging and tracing macros.

When we build Peano, we do distinguish different "debug" levels:

- If a code is translated with `-DPeanoDebug=0`, all logging, tracing and assertion macros are disabled, i.e. replaced by empty statements. Warning, information and error messages are still displayed.
- If a code is translated with `-DPeanoDebug=1`, all macros from `-DPeanoDebug=0` remain defined. This time however, all tracing functionality is enabled. With this mode, it is possible to do production runs plus to extract some performance data, e.g.
- If a code is translated with `-DPeanoDebug=2`, all the tracing macros from `-DPeanoDebug=1` remain defined. On top of this, Peano's assertions now are enabled, i.e. assertion macros do not degenerate to nop but actually run tests. This might increase the runtime significantly yet the code permanently runs test.
- If a code is translated with `-DPeanoDebug=4`, logging and assertion macros from `-DPeanoDebug=4` remain defined. Furthermore, any debug information written through Peano's log interface is now not discarded (removed at compile time) but actually shown. This is the slowest mode yielding most of the data.

11.2 Using the logging and tracing interface

Peano relies on a plain interface to write out user information. All constituents of this interface are collected in the package `tarch::logging`. Documentation on this package can be found in the corresponding header files, files ending with `.doxys`, or the Peano webpage (section on sources). The usage paradigm is simple:

1. Each class that wants to write logging information requires an instance of class `tarch::logging::Log`. As such an instance is required per class, it makes sense to make this field a static one.

```
#include "tarch/logging/Log.h"
...
```

```
class MyClass {
  private:
    static tarch::logging::Log _log;
};

tarch::logging::Log MyClass::_log( "MyClass" );
```

Please keep to the nomenclature of the class field to make all macros work. Please use as string argument in the constructor the fully qualified class name. See the remarks on log filters below for an explanation for this.

2. Whenever you want to log, trace, write an error message or a warning, you should use Log's operations to write the messages. Alternatively, you may want to use the log macros from `Log.h`. They work with stringstreams internally, i.e. you may write things along the lines

```
logInfo( "runAsMaster(...)", "time_step " << i << ": dt=" << dt );
```

where you concatenate the stream with data.

Peano offers four levels of logging through its macros:

- **Info**. Should be used to inform your user about the application's state.
- **Warning**. Should be used to inform your user about bothering behaviour. The MPI code uses it, e.g., if many messages arrive in a different order than expected. Messages written to the warning level are piped to `cerr`.
- **Error**. Should be used for errors. Is piped to `cerr` as well.
- **Tracing**. Used to keep track of when a method is entered and then left again.
- **Debug**. Should be used to write debug data. It goes to `cout` and all debug data is removed if you do not translate with the compile flag `-DDebug`. Notably, use the `logDebug` macros when you write to the debug level, as all required log operations then are removed by the compiler once you create a release version of your code.

11.3 Logging devices (formats)

The `Log` instance forwards the information to a logger. By default, this is the `tarch::logging::ChromeTraceFileLogger`. You may want to write your own alternative implementation or switch to another option. Logging can be very time-consuming. I therefore decided to link the Log objects to one particular log device statically. If you want to "redirect" your outcome, you can either manually edit the logging header, or you can retranslate your code with `-DUseLogService=xxx`. `xxx` is the class to be used. I detail the variants I currently ship with Peano below.

11.3.1 CommandLineLogger

To select this logger, compile with `-DUseLogService=CommandLineLogger`. This logger is the default.

This logger dumps all information in a human readable format. It can be tailored towards your needs. For this, the logger provides a particular setter. Please consult the header or the webpage for details on the semantics of the arguments:

```
tarch::logging::CommandLineLogger::getInstance().setLogFormat(
  " ", true, false, false, true, true, "my-fancy.log-file" );
```

The interface also allows you to pipe the output into a file rather than to the terminal. This is particular useful for MPI applications, as each rank is assigned a file of its own and messages are not screwed up. Typically, the logger is configured in the `main` of the application.

If you run Peano for long simulations and, notably, if you run Peano with debug information switched on, log files soon become massive. To ease the pain, Peano's command line logger offers an operation

```
tarch::logging::CommandLineLogger::getInstance().closeOutputStreamAndReopenNewOne();
```

that works if you have specified an output file before (see `setLogFormat` above). Per close, you close all output files and start to stream the output into a new file. Typically, developers use this operation in their iterative schemes to stream each iteration to a different file. The output files are enumerated automatically.

11.3.2 ChromeTraceFileLogger

To select this logger, compile with `-DUseLogService=ChromeTraceFileLogger`.

This logger writes the one json file per rank. The format conforms to Google Chrome's tracing format, i.e. you can open the file in Google Chrome by typing in the URL `chrome:///tracing` and loading in the file in the GUI that pops up. As the format is prescribed, there are no opportunities to configure the output. While the majority of all output data goes into these Chrome files, warnings, errors and info messages are also dumped to the terminal in a format which can be read by humans easily.

11.3.3 NVTX logger (NVIDIA SDK)

To select this logger, compile with `-DUseLogService=NVTXLogger`.

This logger forwards all trace commands to NVIDIA's profiling/tracing library.

11.3.4 ITAC logger

To select this logger, compile with `-DUseLogService=ITACLogger` and configure `--with-toolchain=intel`

This logger forwards all trace commands to Intel's profiling/tracing library.

On Hamilton8 you can activate the ITAC logger as follows:

```
export FLAVOUR_NOCONFLICT=1
module load gcc advisor oneapi intelmpi itac

./configure --with-multithreading=omp --enable-exahype --enable-loadbalancing --enable-blockstructured --with-mpi="I_MPI_CXX=icpx
```

Remember to ask SLURM for a lot of memory as the trace data soon becomes huge: `#SBATCH --gres=tmp:400G`. Also be sure to add `-trace` when you run the python glue code and filter the trace data as desired in `exahype.log-filter` (described above).

To do a run with the trace analyzer add `-trace` to your mpirun command:

```
mpirun -trace -np <N> ./Peano
```

And visualise the results with -X forwarding with:

```
traceanalyzer peano4.stf
```

Vtune output can also be enhanced by the ITAC logger. For vtune the following structure works and creates data that can be viewed using the `vtune-gui`:

```
module load vtune
OMP_NUM_THREADS=2 mpirun -trace -np 4 vtune -collect hotspots -r my_result_dir ./peano4 --timeout 2000 --threading-model backfil
```

11.4 Log filter

The amount of log information often becomes hard to track; notably if you run in debug mode. Often, you are interested only in a subset of all log messages. For this, Peano offers log filters which provide a blacklist and whitelist mechanism to filter messages before they are written. A log filter entry is created by

```
tarch::logging::LogFilter::getInstance().addFilterListEntry( tarch::logging::LogFilter::FilterListEntry(
  tarch::logging::LogFilter::FilterListEntry::TargetInfo,
  tarch::logging::LogFilter::FilterListEntry::AnyRank, "peano4", false,
  tarch::logging::LogFilter::FilterListEntry::AlwaysOn
));
```

The log filter is a singleton, and log filters can be used totally independently of the log output format.

Configuring log filters in your source code is a convenient option when you start a new project. On the long run, it is cumbersome if you have to recompile every time you want different log information. Therefore, the `CommandLineLogger` also offers a routine that allows you to load log filter entries from a text file. This facilitates work with log filters. The usage is straightforward

```
tarch::logging::LogFilterFileReader::parsePlainTextFile( "my.log-filter" );
```

and the format is very simple:

```
# Level             Trace                     Rank          Black/white  When on
# info/debug/trace  trace of class affected   -1=all ranks
  debug             tarch                     -1            black       always-on
  debug             peano4                    -1            black       always-on
  info              tarch                     -1            black       always-on
  info              peano4                    -1            black       always-on
  trace             tarch                     -1            black       always-on
  trace             peano                     -1            black       always-on

# Lines starting with hash are comments

# Switch on info output from one class on one rank only
info     examples::subdirectory::myClass    15    white  my-phase
```

For performance reasons, it is important to take two facts into account:

1. Black- and whitelisting works only on the class level. While tracing, logging, and so forth all tell you which operation has logged a certain entry, you cannot filter on the method level.
2. Black- and whitelisting are not dynamic. The very first time a class wants to dump information, it queries the log filter. The result of the log filter then is hold persistently.

R Whether a log message is written or not results from the combination of compile flags (`PeanoDebug` flag) plus the log filters you have in place. If you translate with `-DPeanoDebug=0`, debug log filters play no role, as the underlying messages are not written anyway. In this context, you have to pay attention against which libraries you link. Peano builds the technical architecture, the core and tools in different versions with different optimisation levels

and different log settings. Select the right one if you wanna see messages from the core or tarch.

You can have different log entries for different program phases. By default, the log filter does not keep track of the program phase, i.e. it does not know which program phase is on. You can use

```
tarch::logging::LogFilter::getInstance().switchProgramPhase("my-phase");
```

From hereon, only the log filter entries labelled with `my-phase` or with `always-on` will be used. You can use any string for `my-phase`. Some higher-level frameworks such as ExaHyPE 2 define some program phases a priori.

11.5 Assertions

Peano provides its own assertion format which is independent of C++ assertions. Assertions are offered through macros, and if you choose a `PeanoDebug` level which does not support assertions, then they are removed at compile time from your code.

Peano's assertions interlink with the logging, i.e. before they shut down a rank, they ensure that all logs are flushed. Furthermore, they provide a few convenient variations that

1. support numerical "equality" checks,
2. support the logging of parameters, and
3. support vector-valued comparisons.

Chapter 12

Unit tests

Peano introduces its own lightweight test concept with picks up best practices from the JUnit world. While it is simple and has no external dependencies (all test routines are collected within the `tarch` subdirectory), it provides a few macros to handle numeric data—which might be convenient for AMR applications; so there's some added value. It also provides test routines for vectors.

12.1 Test architecture

A Peano test is a class that implements and extends `tarch::tests::TestCase`. By convention, I place my tests always into subdirectories/subnamespaces of the respective component, but this is not a must. A test class must implement a `void run()` method. This is the only convention. The Peano toolkit by default generates one (empty) test per project that can act as a blueprint if you don't want to write your test manually. Feel free to delete the class if you don't want to write unit tests on your own (though the test itself is empty and thus does not harm the test runs).

A proper test class should realise a couple of `void` operations without arguments that run the individual tests. A minimalistic test thus looks as follows:

```
namespace myproject {
  namespace tests {
    class TestCase;
  }
}

class myproject::tests::TestCase : public tarch::tests::TestCase {
 private:
  /**
   *These operations implement the real tests.
   */
  void test1();
  void test2();
 public:
  TestCase();
  virtual ~TestCase();

  virtual void run();
};
```

The test's `run` operation has to register the individual void routines running the actual tests:

```
void exahype::tests::TestCase::run() {
  testMethod(test1);
  testMethod(test2);
}
```

12.2 Running the tests

Once these steps are handled, you can run the tests within your `main`. For the core and tarch routines, I provide factory methods

```
tarch::getUnitTests();
peano4::getUnitTests();
```

which deliver a test suite. To run them, I typically insert snippets alike

```
int unitTestsErrors = 0;
tarch::tests::TestCase* tests = nullptr;

tests = tarch::getUnitTests();
tests->run();
unitTestsErrors += tests->getNumberOfErrors();
delete tests;
```

into my main. It returns an output similar to

```
running test case collection "exahype2.tests.c" ..... ok
running test case collection "exahype2.tests.solvers" .. ok
running test case collection "exahype2.tests" .... ok
running test case collection "peano4.grid.tests" ..... ok
running test case collection "peano4.grid" ok
running test case collection "peano4.heap.tests" .. ok
running test case collection "peano4.heap" ok
running test case collection "peano4" ok
running test case collection "tarch" ok
running global test case collection ok
```

The output returns a hierarchical overview over all tests ran. The number of dots per package identifies the number of actual test functions in a namespace. If tests fail, the interface reports the exact location including file name and line number.

R It is convenient to run tests only if you have translated your code with a sufficient debug level (cmp. Section 11). When you write your own tests, I often add test attributes to the classes—fields that are not required but help me to validate object states. Since I protect these attributes with ifdefs reading Peano's debug level, they are removed from the production code.

12.3 Test routines

A typical unit tests builds up some classes, befills them with data, invokes some operations and finally checks the class state via some getters or routines returning results. Peano's way to do the actual tests are `validate` macros. You find all these macros in the file `tarch/tests/TestMacros.h`. There are three classes of macros:

- Standard validation macros similar to assertions that test a boolean condition.
- Comparison macros that compare two values through the `operator==` (which might be overloaded).
- Comparison macros that compare doubles, vectors or matrices numerically, i.e. up to a prescribed precision.

All of these macros come along with variants that allow you to plot additional information if a test has failed, i.e. to plot the values of further variables, e.g. All validation macros automatically set a test to failed if they fail, and they write a verbose message on the reason why some tests failed to the terminal.

Tests tend to become spaghetti code and some of them are very expensive to translate as the compiler's optimiser has a hard time to translate the long code fragments. I thus recommend to insert

```
#include "tarch/compiler/CompilerSpecificSettings.h"

#ifdef UseTestSpecificCompilerSettings
#pragma optimize("", off)
#endif

// All the implementation

#ifdef UseTestSpecificCompilerSettings
#pragma optimize("", on)
#endif
```

into your implementation file around all your code. This switches off all optimisation explicitly for the tests.

Chapter 13

Parallel programming

Peano has originally been written for MPI+X on CPU-based architectures. On the MPI side, I provide some wrappers around mainstream MPI routines which augment the MPI things with deadlock detection, explicit buffering of unexpected messages or global synchronisation. On the X-side, Peano 4 has its own

13.1 Shared memory

13.1.1 Protecting shared memory access

In the tradition of Intel's original TBB version, I offer an abstraction over critical sections aka exclusive code blocks. That is blocks that can only be executed by one thread at a time.

To use it, you define a `BooleanSemaphore` object. It should be global (or static) and is an abstraction around C++/TBB/OpenMP. To lock it, you can do so manually, but actually I offer a `Lock` object which locks the semaphore and automatically frees it once the object is destroyed.

For details, please consult the documentation of the class `tarch::multicore::BooleanSemaphore`. The `multicore` namespace also hosts variants to support recursive semaphores.



It might be faster to use atomics if you want to protect the access to a single native variable. In this case, I recommend to stick to C++ atomics. The semaphore is a wrapper around C++ and OpenMP (so your code will work with both). If you use only C++ atomic, you should still be fine when you switch to OpenMP.

13.2 Distributed memory

Distributed memory programming in Peano can be tricky, as we have a lot of different threads per rank and the ranks run pretty asynchronously. Therefore, it is always a bad idea to use blocking or synchronising routines. Here are a few recommendations:

1. Do not use a blocking send or receive, even if routines are logically blocking. Instead, use a non-blocking routine and poll the termination via `MPI_Test`. While you poll, invoke `receiveDanglingMessages()` on the services to allow Peano 4 to free the queue from other messages such as load balancing requests which might be in your way.

2. Register your own routine as a service (see `tarch::services::Service`) and equip it with a `receiveDanglingMessages()` routine which probes the MPI queues. If a message of relevance is in the MPI queue, receive it. If you can't react directly, buffer it internally. This routine ensures that the MPI buffers can't overflow. Overflows force MPI to fall back from eager to rendeszouz communication which can lead to deadlocks.
3. Do never impose any synchronisation unless absolutely required. Notably, try to avoid synchronisation within the action sets! In the ideal case, you implement a Jacobi-like data exchange: The action sets send stuff out. There's a service (see above) which receives the data. This data is then used by the action set (or another one) in the subsequent grid pass.
4. If you have a global operation, use a non-blocking collective and wait for the collective's test in the subsequent grid run-through.



If a message consists of multiple parts, you might have to fuse it into one message to avoid that the first part of a message is received and queue polling (on another thread) then grabs the second part misinterpreting it for the first part of yet another message.

13.2.1 Synchronisation

I also offer a boolean semaphore accross all MPI ranks. This one is quite expensive, as it has to lock both all threads and all ranks and thus send quite some messages forth and back.

The usage is similar to the shared memory lock, but you have to give the semaphore a unique name (you might construct a string using the `__LINE__` and `__FILE__` macro). It is the name that Peano 4 uses to ensure that the semaphore is unique accross all ranks. In this way, the semaphore is similar to an OpenMP named critical section. I recommend to avoid MPI barrier whenever possible. Details can be found in `tarch::mpi::Rank::barrier()`.

13.2.2 Exchange global data between ranks (reduction)

If you reduce data betwen the ranks, I recommend that you use Peano 4's wrappers. They have two advantages: They automatically provide deadlock tracking, and they poll the MPI queues, i.e. if the code stalls as too many other messages for other purposes (global load balancing, plotting, ...) drop in, and remove these messages from the buffers such that MPI can make progress. The usage is detailed in `tarch::mpi::Rank::allReduce`, but the code usually resembles:

```
double myBuffer = _myVariable;

tarch::mpi::Rank::getInstance().allReduce(
  &myBuffer,
  &_myVariable,
  1, MPI_DOUBLE,
  MPI_MIN,
  [&]() -> void { tarch::services::ServiceRepository::getInstance().receiveDanglingMessages(); }
);
```

Chapter 14

Tools

14.1 Loop macros

14.1.1 d-dimensional loops

I wrote Peano to support arbitrary spatial dimensions. By default, I build only 2d and 3d versions of the code/core, but in principle you can write also simulations over 4d or 5d domains, e.g. The choice of a particular dimension d is done at compile time via the flag `-DDimension=d`. As everything is generic w.r.t. dimensions, I have many places in the code where I need d-dimensional loops.

Peano's utility directory/namespace `peano4::utils` offers a header `Loop` which comprises a set of macros that support d-dimensional loops. Details can be found in the source code documentatation within the file or online, but the collection of macros allows you to write code alike

```
dfor(k,20) { // d-dimensional loop over 20x20x20x... patch
  // There is now a d-dimensional vector k over integers which holds the
  // loop index. So we can write for example
  const tarch::la::Vector<Dimensions,double> somePosition =
    offset + k.convertScalar<double>() *h;
  [...]
}
```

There are also macros for loops skipping one dimension, or macros for particular iteration ranges (2,3,4) which are highly optimised through lookup tables. Finally, `Loop` hosts z-loops, i.e. loops which meander forth and back through an array similar to the Peano curve.

To use the d-dimensional loops, you have to include

```
#include "peano4/utils/Loop.h"
```

For Cartesian meshes, arrays, the file also offers a routine `peano4::utils::dLinearised` which converts the d-dimensional loop counter into one integer. Again, I do offer variants of this routine which are optimised to particular choices of d or array ranges.

14.1.2 d-dimensional exclusive loops

There's a variation of the d-dimensional loops which loops over all dimensions except one. I name this loop type `dfore`. Its usage is straightforward: You again specify a counter name and you pass in the loop range. Let `k` be the counter name and `N` the range. You furthermore specify a dimension and you fix the value for this dimension.

The loop will run over a `N` $\times$ `N` $\times$ 1 $\times$`N` range with the 1 being in the specified direction. The loop counter is a vector over integer named `k`. Its entry along the 1-dimension is fixed to the default value specified in the macro.

Part IV

Extending the Python API

Chapter 15

Introducing a new pre-defined action set

This chapter discusses how you make an action set (set of event implementations) of your code available to other projects as a pre-defined action set. As a result of such a refactoring, others can inject your ideas straightforwardly into their project. The name refactoring suggests that I highly recommend that you start from a working action set that you've written manually and extensively tested.

15.1 Prepare classes

Create a new subclass of `peano4.solversteps.ActionSet`

```
import peano4.solversteps.ActionSet

class MyActionSet(peano4.solversteps.ActionSet):
  def __init__(self):
    pass
```

and ensure that Python finds the class in its path.

15.2 Implement new action set

To implement the new action set, you have to tell Peano what the function bodies look like. Before we do so, we instruct the toolkit whether this is an action set that users should later on edit; that is, your new predefined action set serves as blueprint. Alternatively, your action set can be read-only and users shall not modify it. In this case, Peano will place it in the observers' directory and always regenerate it automatically.

The first step hence is to give your action set a clear name (which will also be used as class name part for the generated file) and to provide this information:

```
  def get_action_set_name(self):
    // My default implementation that I use in the toolboxes is this following
    // line. You are however allowed to use your own, bespoke name
    // return __name__.replace(".py", "").replace(".", "_")
    return "MyName"

  def user_should_modify_template(self):
    return False
```

The actual events are generated through the operation bodies. All these bodies are queries through one operation. The query's arguments clarifies which operation body is requested

```
def get_body_of_operation(self,operation_name):
  # An examples where we only do something for :
  if operation_name == peano4.solversteps.ActionSet.OPERATION_CREATE_CELL:
    return "mycell.add(...)\n"
  return "// Nothing_to_implement\n"
```

and its values are those constants in `Mapping` which start with `OPERATION_`.

There are further routines to control the refinement et al. These routines are self explaining if you study the base class/interface.

More interesting is the question how to make the actual method bodies use grid information. For this, the canonical way is that you extend the constructor such that an object of your action set always "knows" which data they work on. The interfaces of all action sets are always the same, so it should be straightforward to make your generated method bodies use the grid data once the constructor is told what they are.

15.3 Use your new action set

To use the action set, you create your algorithmic steps. Once created, you also create and instance of your action set and you add it to the step with `add_action_set`. The order in which action sets are added is important. See `add_action_set`'s documentation of information on this.

15.4 Conventions

By default, the Peano toolkit equips each action set with a couple of features/attributes:

- Every action set has a static attribute `_log` which is an instance of `tarch::logging::Log`. That is, you can use Peano's logging commands straightaway.

Part V

Design decisions

Chapter 16

Domain decomposition and data flow

Peano 4 starts from spacetrees and thinks in whole trees and only supports two types of decomposition operations: a split and a join. At the same time, it does not distinguish shared memory and distributed memory parallelisation. It only thinks in terms of trees. The trees and their traversal can either be deployed to ranks or threads or combinations of the two.

This statement is only weakened once you work with Peano 4's task interface. If trees issue tasks, then these tasks team up with the tasks handling the individual spacetrees. Your task graph starts to contain a mixture of tree tasks and user-defined tasks. This section discusses solely the tree decomposition aspect within Peano 4.

Peano 4 relies on a multiscale non-overlapping domain decomposition. Within the trees, we distinguish three different types of parallel data flow:

- **Horizontal data exchange** is exchange between two cells of one level along their boundary.
- **Vertical data exchange** is exchange between trees where one resolution meets the other. Vertical data exchange is also used to stream data from one rank to the other if we fork. Joins do not stream logically.

16.1 Multigrid topology

A tree in Peano is split along the Peano space-filing curve. This implies that the decomposition of the finest mesh is a non-overlapping decomposition where each cell is assigned to exactly one rank. I use rank as term for rank or thread or task. See the discussion above.

Also Peano's coarser cells are uniquely assigned to one tree. Between the ranks, we have a tree topology again. That is, a splitting of a tree is always realised such that there's a unique master-worker topology.

Some examples in Figure 16.1 sketch the implications: In the left example, the yellow tree has split up into yellow and green. The green tree has further split into green and red. The code usually tries to keep whole trees, i.e. children and parents, within one tree. In the left example, it would be natural to make the very right green cell on the first child level a red one, too. This way, a whole tree would reside in red. However, if we made this single cell red, then red would become a child of yellow, i.e. we would change the rank topology. Peano never does so.

In the right example conversely, I've asked the yellow tree to split into yellow, green and red in one rush. This time, the right level 1 cell becomes red already.

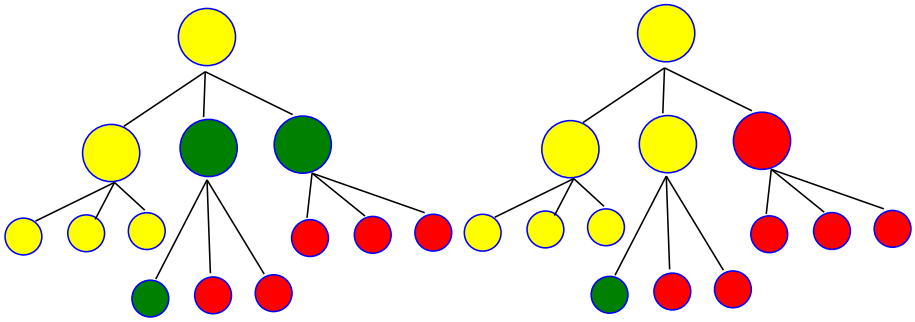


Figure 16.1: Two examples of a tree decomposition.

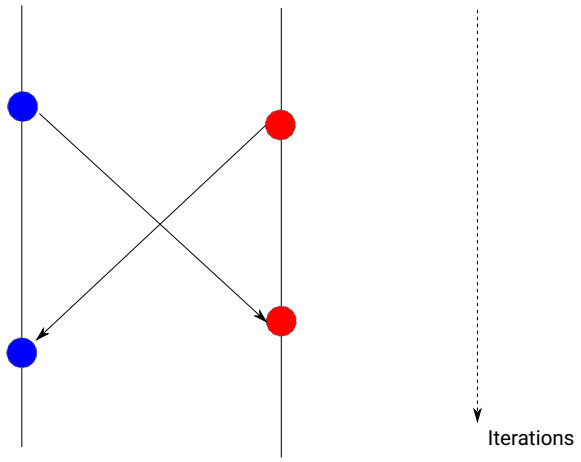
Both examples show that a split of one tree into n trees never results in the fact that the original tree becomes empty. Otherwise, we woudl again change the tree topology upon the ranks.

R As Peano 4 preserves the logical topology between trees, it is reasonable to strive for as wide tree topologies as possible. Otherwise, your meshes (coarsening) are very constrained. That is, it is better to split a tree a into five parts a, b, c, d, e with b, c, d, e being children of a rather than splitting a into b, c and then b and c again.

16.2 Data flow on stationary domains

16.2.1 Horizontal data flow

Peano 4 uses a non-overlapping domain decomposition. Therefore, ie shares/exchanges faces and vertices between different trees. It never shares cells of one level. I explain the code's data exchange pattern with vertices. Yet, the same pattern holds for faces, too. After a code has read/written a vertex for the very last time throughout a traversal, it sends a copy of this vertex away. In-between two iterations, all trees exchange their data. That is, in the next iteration any sent out data is available on the destination rank and can be merged into the local data structures. This merge happens on the destination prior to any other operation.



Peano 4 consequently realises a data exchange that is similar to Jacobi smoothers: Whatever you do on one tree won't be visible on the other trees in this very tree traversal. Prior to the next tree sweep, this information however becomes available there.

16.2.2 Vertical data flow

This yet has to be written.

16.2.3 Synchronous vertical data flow

This yet has to be written.

16.3 Splits

Whenever you split a tree, Peano creates the new trees (either as threads on the local node or remotely via MPI). Each new tree receives a whole copy of the original tree. This includes both the core tree data and the user data. Once the tree is replicated, the individual trees start to coarsen. If they are not responsible for tree parts, they successively remove this part. After one or two iterations, each rank thus really works only on local tree.



Peano 4 uses information from the actively used data to decide which data to replicate. If you have cell and vertex and face data but you use some of these data only in some substeps of you algorithm, please ensure that those steps that trigger the domain decomposition and those that run immediately after this do use all data types. Otherwise, Peano 4 can't know what data is to be replicated when it splits trees.



Splits require significant memory temporarily. It thus might be convenient not to split a tree too often in one rush.

16.3.1 Temporal ordering of tree traversals

With the information of the data flow above, any splits actually has to be realised over two grid sweeps which internally decompose into substeps:

1. The user asks a tree to split. From hereon, we have a master with some neighbours and a worker. The latter does not physically exist yet.

2. The master runs through its domain and tells all neighbouring ranks that the new tree will drop in in the future. At this time, these neighbours are not yet aware of the split (the information becomes only available in the subsequent iteration), so they continue to send their stuff to the master even though it might be vertices which will, in the future, belong to the new worker. We say that that we have a **split-triggered** in this iteration.

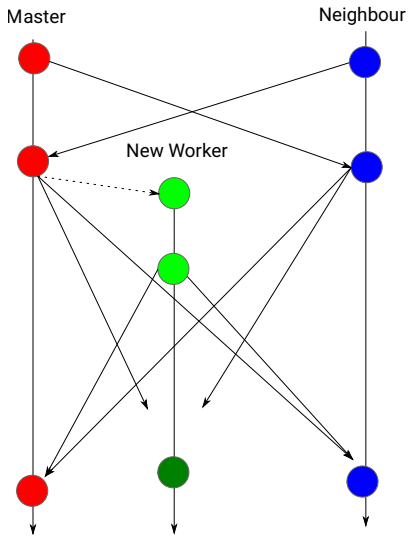
3. In the next iteration, the master is **splitting**. It still holds the ownership of all the data, as it receives for example all neighbour data. However, in the splitting phase, the master already sends out data to the worker where new domain boundaries will arise—even though the new worker does not exist yet. The same happens to all the neighbours. They send out boundary data to this new worker that will now enter the game.

4. Once the master has terminated, the responsible rank issues a couple of postprocessing steps:

   (a) It clones the whole master tree. This is only the grid structure, but it is a complete clone.

(b) It instantiates the new worker with this cloned data. As we clone after the master's traversal has terminated, a lot of trees have already sent data to the new worker that we just cloned. However, this worker has not yet sent out any data, so we've broken the symmetry of our data exchange. Therefore, we run through the tree once as **empty tree**. In this sweep, we do nothing. We only run once through the tree to ensure that the stack data is in the right order.

(c) Immediately after that, we do a **new tree** sweep. This sweep does not invoke any user events. It however sends out the data the others expect from us.



16.3.2 User data

There is no need for the user to implement something manually, as long as the user ensures that the grid observer replicates the master tree on all new workers. The `SpacetreeSet` offers a function to do this automatically.

16.3.3 Data ownership

Throughout the split traversals, the code continues to interact with the user and continues to compute, i.e. all splitting is seamlessly interwoven with standard compute mesh traversals. Throughout the whole split, the logical ownership of all data resides with the master, i.e. it is the master automaton who asks the user code what operations to invoke on the data.

16.4 Joins

Joins are completely hidden from the user, i.e. there's no way to actively join trees. To understand why Peano issues joins from time to time, it is important to define vetoed coarsening.

D Peano vetos any coarsening, if removing some parts of the grid would affect a child or alter the tree topology of the ranks. If a coarsening is vetoed, Peano remembers this veto. As soon as a rank whose master is affected by a veto hosts a tree cell which is unrefined, it joins/merges this cell into the master.

Vetoed coarsening typically does delay any grid coarsening. You ask for a coarsened grid, and it first of all applies this coarsening to all ranks that have not decomposed further. Once these have

finished their coarsening, it joins those ranks that stop further coarsening into their master. Once this is complete, it continues to coarsen. This process can continue recursively.

For this reason, the join of partitions in Peano 4 follows a few mantraic principles:

1. We only join partitions if they do not hold refined cells. We call such partitions deteriorated (in a tree sense). If a partition holds a refined cell, it cannot be merged into its father.

2. We hide joins from the user, i.e. a user can split up a partition and thus implement load balancing, but a user is never allowed to join. Joins are always triggered by Peano 4's core internally. The policy is as follows:

   - We join a deteriorated partition if the master resides on the same rank and if this master would like to coarsen its domain but can't do so as this coarsening would violate the tree topology (see dicussion above).
   - We join a deteriorated partition if the master resides on a different rank. The idea here is that such deteriorated partitions always induce quite some overhead.

There's technical rationale behind some items: We don't join deep trees with their master as both master and worker work with their LET, i.e. try to coarsen their respective mesh as aggressively as possible after the split. Therefore, a master does not have all grid infrastructure of a worker available. It would require some complex zipping to bring the info of two trees together. If I realise a deteriorated-trees-only policy, I don't need such zipping as (almost) all information is available on the master already.

16.4.1 Temporal ordering

Analogous to the splitting process, each join is broken up into two traversals. Throughout these traversals, the code continues to interact with the user and to compute. In the first traversal, the worker tells everybody else that it intends to join with its master. However, nothing happens. Throughout this first traversal (**join-triggered**), all the neighbours of a rank as well as its master continue to send boundary data to the tree that intends to join, as they are not yet aware of the join intention.

In the second traversal (**joining**), we distinguish two steps. First, the worker traverses its domain. The traversal of the master is postponed. Throughout the traversal, the worker receives incoming data and merges it with the local information. Information on the grid topology is immediately streamed to the master. Immediately means after the vertex data is read for the first time and merged. See the discussion on the grid data below. After that, the worker continues, as it still own all data (see below). When it writes data to its output stream for the very last time, the data that is adjacent with the master is written to an output stream.

In the second step of the second traversal, the master now runs through its grid. Before that, both grid and user data from the worker are streamed in. The master takes the streamed in mesh data right after it has read its information. Whenever it runs through a mesh entity for the very last time, it merges its local user data with the streamed-in records from the (former) worker.

16.4.2 Data ownership

Throughout the split traversals, the code continues to interact with the user and continues to compute, i.e. all splitting is seamlessly interwoven with standard compute mesh traversals. Throughout the whole split, the logical ownership of all data resides with the master, i.e. it is the master automaton who asks the user code what operations to invoke on the data.

16.4.3 Grid data streaming

In principle, one might assume that no streaming is required if we merge only deteriorated trees. After all, if a rank deploys some subtree to another worker, it still holds its root node locally (though flagged as remote). However, there's one delicate special case: If a tree joins its master and another neighbour changes something within the mesh, too (merges as well or splits up), then the master won't be notified of this info. The update information will reach the worker while it is joining into its master. Therefore, a merging worker does stream its vertex data immediately into the master while it joins. This streaming requires us to break up the join process into two distinct traversal phases (see primary vs. secondary remark above).

We don't have dedicated streams in Peano 4, and there's no communication channel for joins either. However, there's by default for any piece of data a vertical exchange stream. For the joins, we hijack the vertical stacks and write the data to them. When we hand the data over to the master, we revert the order on the stack and thus effectively convert it into a stream.

16.4.4 User data streaming

R This section is incomplete and only partially implemented.

Hijack vertical stacks on grid side!

R The realisation of joins and splits in Peano 4 is totally asymmetric: For a split, all data of the splitting tree is replicated and then both the original tree and its new worker coarsen the partitions they are not responsible for. This process is realised through two additional traversals on the worker side.

Throughout the join, no data flows logically, as the master already has the mesh. Furthermore, only data that is actually owned by the worker is handed over to the master.

Part VI

Standard benchmarks

Chapter 17

Benchmark: Task priorities

This benchmark has been written as part of the Sheffield GPU Hackathon sponsored by NVIDIA where we took part with the ExCALIBUR project ExaClaw. It assesses whether the task realisation (incl. priorities) is appropriate for Peano 4 and ExaHyPE 2. That is, we have used this benchmark to uncover weaknesses in the OpenMP and TBB backbone of Peano 4 as well as our layer on top to connect to these libraries. The benchmark uses ExaHyPE 2's Euler equation implementation in 2d.



17.1 Preparation

Download Peano 4 (branch p4) and configure the code with ExaHyPE 2 support. Choose the multithreading backend of your choice. Depending on your system, I recommend you activate an appropriate tracing. Peano 4 supports at least support for ITAC (Intel)

```
./configure --
```

and NVTX (NVIDIA):

```
./configure --enable-exahype --with-multithreading=omp --with-nvidia \
  CXXFLAGS=-DUseLogService=NVTXLogger
```

All of the above examples ignore that you might have to set the appropriate compiler (`CXXFLAGS`) and linker (`LDFLAGS`) flags as well such that the build systems finds all required include and library files. In particular, your `LDFLAGS` have to link to `-lnvToolsExt`.

Build the code.

17.2 Set up the experiment

Change into the examples directory hosting the Euler solver. We rely on Peano 4's Python API here:

```
cd python/examples/exahype2/euler
```

For the benchmark, we rely on a simple Euler setup working with a regular grid. However, we use enclave tasking, i.e. the task production pattern is nevertheless already reasonably complex. We run the code for ten iterations on a rather smallish grid:

```
import os
import peano4
import peano4.datamodel
import peano4.solversteps

import exahype2

#
# Create a project and configure it to end up in a subnamespace (and thus
# subdirectory).
#
project = exahype2.Project( ["examples", "exahype2", "euler"], "finitevolumes", "." )

#
# Add the Finite Volumes solver
#
patch_size = 7
unknowns = 5
time_step_size = 0.000001
volume_max = 0.03
project.add_solver( exahype2.solvers.GenericRusanovFVFixedTimeStepSizeWithEnclaves(
  "Euler",
  patch_size,
  unknowns, time_step_size,
  flux = True,
  ncp = False
))


#
# Lets configure some global parameters
#
dimensions = 3
build_mode = peano4.output.CompileMode.Trace
project.set_global_simulation_parameters(
  dimensions, [0.0,0.0,0.0], [1.0,1.0,1.0],
  time_step_size *10, # end time
  0.0, 0 # snapshots
)


#
# Add parallelisation
#
project.set_load_balancing( "toolbox::loadbalancing::RecursiveSubdivision" )

peano4_project = project.generate_Peano4_project()
peano4_project.constants.export( "MaxHOfVolume", volume_max )
peano4_project.output.makefile.parse_configure_script_outcome( "../../../.." )
peano4_project.output.makefile.add_library( project.get_core_library(build_mode),
 "../../../../src/exahype2" )
peano4_project.output.makefile.add_library( "ToolboxLoadBalancing" +
 project.get_library_postfix(build_mode),
 "../../../../src/toolbox/loadbalancing" )
peano4_project.output.makefile.set_mode(build_mode)
peano4_project.build(make_clean_first=True)
```

The important bit here, compared to other experiments, is that you use the tracing target. After you have passed this config file through the Python frontend, ensure that you have a `exahype.log-filter` file in your work directory. It should contain at least the following statements:

```
info tarch -1 white
info peano4 -1 white
```

```
info examples -1 white
info exahype2 -1 white
info toolbox -1 white

trace tarch -1 black
trace peano4 -1 black
trace examples -1 black
trace exahype2 -1 white
trace toolbox -1 black

trace exahype2::fv -1 black
trace peano4::parallel::SpacetreeSet -1 white
```

17.3 Run the code

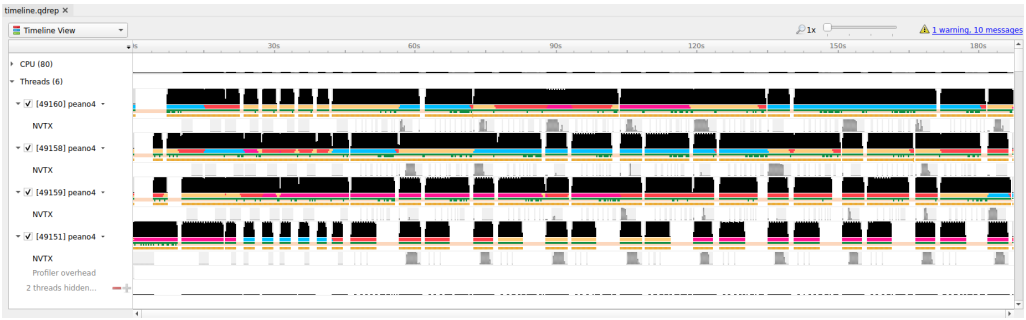
Run the code without any further arguments

```
./peano4
```

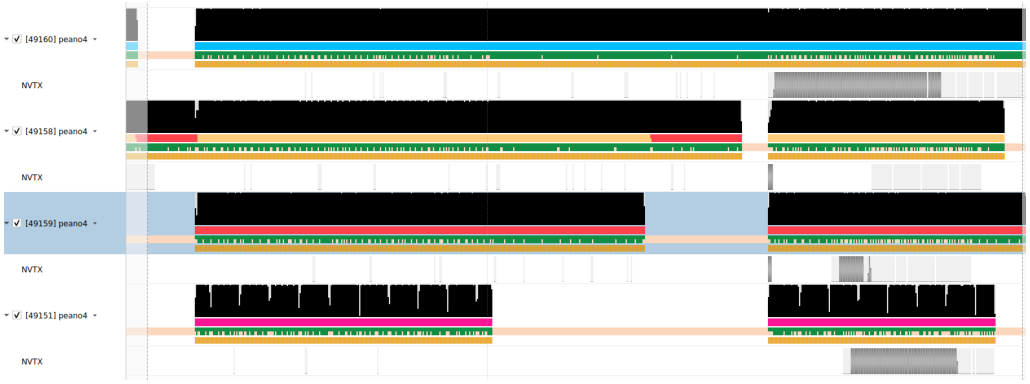
The command line output (it differs depending on which command line logger you have chosen) should tell you that you have

- 729 cells in total, each one hosting a patch of $7 \times 7 \times 7$ finite volumes;
- These cells are split up into four segments. The default splitting should be something similar to 81, 216, 216, 216 cells. That means the partitioning is not very good.
- We have in total 1485 cells, i.e. the setup suffers from a high number of outer cells that do not carry any information.

17.4 Visualise the outcome



The above illustration is an illustration with NVIDIA Nsight Systems 2020.3.1. The code did nine time steps, and each time step consists of two grid sweeps. We see this pattern clearly in the last thread. We also see that this decomposition is ill-balanced, as two threads are very busy all the time, one is reasonably busy, and one faces quite some idle time. This setup is ill-balanced and the task distribution does not work as it should be.



A zoom-in (remember that two blackish bars above represent one time step) reveals that the majority of enclave tasks is handled on the first thread which is the slowest ones. These are the dark grey bars in the illustration below.

17.4.1 What we should see

Peano 4 splits up each time step into two mesh traversals. Each mesh traversal runs through one of the (four) subdomains as introduced above. The first traversal runs through the mesh and issues a lot of tasks. It also solves some Finite Volume problems and issues the MPI messages (not relevant here). The tasks should go into the background, as they are low priority (background tasks). This first traversal is ill-balanced, as the mesh decomposition is slightly ill-balanced.

In the second traversal, each thread runs again through its domain. This time, it waits per cell for which we have issued a background task before until its task is done. Throughout this wait, it processes some of the tasks. The traversals per se again are ill-balanced.

What we should see is that three out of four traversals needs pretty much the same time, as the grid is decomposed into three big chunks plus one smaller one. We should also see that the background tasks (dark items) slot into the idle time of the briefer task.

17.4.2 Data flow spec

The underlying code is realised as follows (this is a generic pattern that I use for all implementation backends):

1. The code runs through the domain parts with a big parallel for.

2. The parallel fors spawn background tasks. They can either be real tasks (with low priority) or we enqueue them manually into a task container.

3. The code issues tells the task system once that it has finished its traversal (see below).

4. The code runs through the domain parts once more with a big parallel for.

5. Per cell, it checks whether it has spawned a task before and whether this task outcome is already there.

6. If the task is not there, the traversal processes one task from the background queue (it could also yield) and checks again, i.e. the checks realise (logically) some kind of busy polling.

17.4.3 Code regions causing the behaviour

All code that leads to the discussed behaviour can be found in `src/tarch/multicore/tbb/Tasks.cpp` or `src/tarch/multicore/omp/Tasks.cpp`, respectively. The routine that is responsible for the parallel mesh run-through (grid traversals) is called `tarch::multicore::spawnAndWait()`.

The handling of the parallel tasks is done in `tarch::multicore::processPendingTasks()` which each traversal calls

- once after it has finished its local traversal (with the parameter 0 to indicate that it is not waiting for anything but it might be a good idea go launch some task progression threads);
- every time the code waits for a background task outcome (with the parameter 1, as it says "do one task and return so I can check whether the result I'm waiting for meanwhile has dropped in").

17.5 Flaws identified

17.5.1 OpenMP: task group

I have used a simple OpenMP task group

```
void tarch::multicore::spawnAndWait(
  const std::vector< Task* >& tasks
) {
  #pragma omp taskgroup
  for (int i=1; i<tasks.size(); i++) {
    #pragma omp task
    {
      bool reschedule = tasks[i]->run();
      assertion( not reschedule ); // these tasks should run only once
      delete tasks[i];
    }
  }
}
```

It seems that this snippet uses five threads for four subpartitions, i.e. creates a task tree of height one where the parent task of the newly created tasks does some busy waiting. This is unfortunate, as further tasks might remain in the system which should/could be processed meanwhile (see next item).

17.5.2 OpenMP: task group

If I use a task group, it seems that no further OpenMP tasks are progressed while I wait for all child tasks to terminate. Idle time is thus not used efficiently. The tasks in Peano 4 can last for quite a while (they are whole domain traversals), i.e. kicking one thread out of the system through busy polling/active waiting is kind of a waste. Therefore, we need a more elegant solution here.

Part VII

Appendix

Appendix A

Postprocessing: The visualisation toolkit, IO and Peano 4 scripts

We offer five different ways to handle Peano 4 output, i.e. the native Peano patch files:

1. You visualise them directly within Paraview (Section A.1).
2. You use a Python-based visualisation suite from the command line (Section A.2).
3. You convert the patch files via a C++ command line tool into VTK and postprocess the data from there (Section A.3).
4. You can convert patch files via a python tool into legacy VTK. These can be generated directly on the cluster without the need for paraview/pvpython. This is useful if the cluster does not have the version of paraview installed that is required by the C++ command line tool mentioned above. See section A.4 for more detail.
5. If you want to extract the data for a single point directly from a patch file or set of patch files without any need for Paraview/pvpython/VTK then it is worth exploring the probe mechanism (section A.5)

The following five sections discuss these three routes, before I give some general visualisation comments in Section A.6. Furthermore, I sketch further Python scripts that can be used to postprocess some output you get from any Peano 4 application. This is information about the general runtime or load balancing distribution, e.g.

A.1 Displaying Peano 4 files directly within Paraview



Peano 4's postprocessing scripts use Python 3. This means you need a Paraview version with Python 3 support (oldish Paraview installations only support Python 2.7, and some Paraview downloads don't come along with Python support at al).

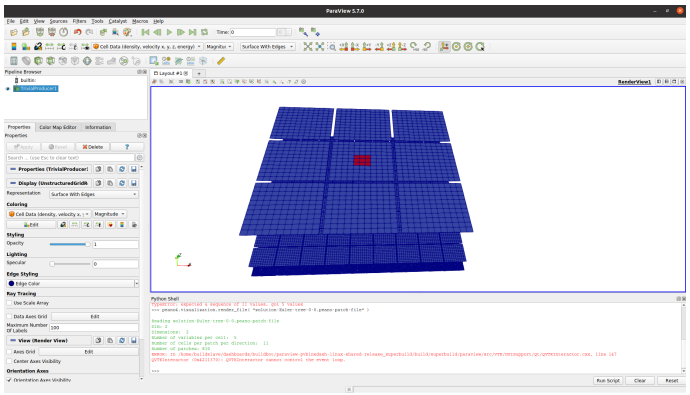
You can display Peano 4's patch files directly within Paraview. Again, there are different ways. For all of them, Peano 4's Python environment has to be available within Paraview:

Prepare system. Before you start up Paraview, ensure that both the Peano 4 `python` directory and any third-party Python lib are in your path. On some systems, Paraview seems to only know its internal libraries, so you have to add these manually. On my system, I use

```
export PYTHONPATH=.../Peano/python:/usr/lib/python3/dist-packages
```

The latter holds `jinja2` on my system.

Start Paraview's Python environment. You can either manipulate all data directly within Paraview or use Paraview's Python terminal to convert data as a batch from Peano's native format into native Paraview data. I recommend the direct Paraview terminal route to interactively explore data. The actual data processing (of time-dependent data) however can be very slow. Once I know what information I need, I thus usually switch to Paraview's Python interpreter to convert all data into Paraview's native file formats in one rush.



For the batch mode, open the terminal `pvpython`. For the interactive GUI mode, open Paraview and activate the Python Shell by clicking the check box in the View menu.

Both visualisation routes first require you to load Peano 4's Python tools—either within the `pvpython` terminal or the Paraview Python window:

```
import peano4.visualisation
```

Work with a whole dataset. Peano 4 provides a Python class `Visualiser` which allows you to navigate through the whole data:

```
import peano4.visualisation
visualiser = peano4.visualisation.Visualiser( "solution-Euler.peano-patch-file" )
visualiser.display()
visualiser.append_filter(peano4.visualisation.ExtractFineGridFilter())
```

The snippet displays the first dataset (time step) from a Peano 4 data dump. Via

```
visualiser.select_dataset(10)
visualiser.reload()
```

you can, for example, select the 11th data set and display this one.

Running through data sets (time steps) manually is cumbersome in many cases. We have not yet managed to integrate our Python-based, bespoke postprocessing into Paraview's video replay features, and we also experience that it is relatively slow. In such a case, it is more convenient to convert all data in one rush into Paraview's native, binary file format vtu:

```
visualiser.write_vtu_time_series()
```

Once this routine terminates, you'll find a file with the extension `.pvd` in your directory which you can open and play as video. Obviously, this conversion can be triggered in Paraview's Python interpreter which is typically faster than Paraview's built-in terminal.

R You can trigger the Python postprocessing routines while the simulation is running.

Display a single file. Load the file you are interested in via

```
data = peano4.visualisation.render_single_file( "solution-Euler-tree-0-0.peano-patch-file", identifier="MyQName" )
```

This is only one snapshot file, i.e. one file written by one thread in one step. We can display the file with

```
tp = TrivialProducer()
tp.GetClientSideObject().SetOutput(data)
Show(tp)
```

If we write large parallel files or time series, Peano 4 will typically create one file like `solution-Euler.peano-patch-fil` which links to all the files such as `solution-Euler-tree-0-0.peano-patch-file`. The meta file (with the links) is plain text, so you can study it via a text editor. Instead of picking a particular file, you can display a particular data set (snapshot) from the meta file:

```
import peano4.visualisation
data = peano4.visualisation.render_dataset( \
  "solution-Euler.peano-patch-file", \
  display_as_tree=False, \
  filter=[peano4.visualisation.ExtractFineGridFilter()], \
  dataset_number=0, identifier="MyQName" \
)
tp = TrivialProducer()
tp.GetClientSideObject().SetOutput(data)
Show(tp)
```

Filters. Peano 4's data are relatively complex. In practice, you typically want to filter data before you pipe it into Paraview. The most popular filter is the one which eliminates the coarse grid levels: Peano 4 holds all data in multiple resolution. If your code does not exploit this feature, you might not be interested in the coarser resolutions, so removing it makes sense.

There are more filters or you can write your own custom filters. Study the source code for further details.

A.2 Offline/command line conversion with Python/Paraview

R This is my favourite starting point for data if I'm interested in whole movies and not only particular snapshots. It is inconvenient for large datasets, as it is too slow. For large datasets, you might have to move on to a conversion via the precompiled scripts (Section A.3).

The command line, Python-based conversion of Peano 4 works if and only if you have Paraview plus Python installed. First, ensure Peano 4's visulisation component is in your Python path:

```
export PYTHONPATH=.../Peano/python
```

or

```
export PYTHONPATH=.../Peano/python:/usr/lib/python3/dist-packages
```

After that, use the conversion script there to translate your `peano-patch-file`. It is important that you interpret this script through `pvpython`:

```
pvpython .../Peano/python/peano4/visualisation/render.py
```

You should see a usage message now.

A.3 Offline/command-line conversion (executable)

Installing Peano 4's VTK command line tools can be quite tricky—depending on your local VTK installation. At the same time, the command line conversion via executables, i.e. not via the Python terminal as discussed above, is by far the fastest route. It can even exploit multiple cores. I recommend to stick to the Python-based postprocessing route as long as possible and to switch to the offline postprocessing only for large-scale production data where Python is just too slow and/or all postprocessing has to happen on a remote supercomputer as you cannot transfer the (raw) Peano 4 data files.



If you configure Peano 4 with VTK support, Peano 4 builds a command-line tool `convert` which you can use to convert Peano 4's patch files into plain (binary) vtk.

Installing VTK support can be challenging. Here's some things you might want to consider/check:

- To build the Peano 4 conversion/visualisation tools, you have to translate the code with `--with-vtk`. If you don't specify a path, then Peano 4 assumes that all VTK is installed within `/usr/include`[1].
- We next have to know which VTK version you are using. VTK builds a different set of libraries in each generation. Peano 4 has to know which version you want to use to link against the right set of libraries. The installation script will automatically try to detect your version, but the detection is not very robust. If it is the wrong version, use the `--with-vtk-version=x` switch to set the version manually. We currently support `x` 7,8,9, i.e. we are only interested in the major version of your VTK installation. If you have version 8.90, it is the 8 we are interested in.
- Next, the script will search for the right libraries. By default, our script assumes that the libraries are have a suffix with the major dot minor number. So we assume that the VTK version 8.90 yields a library `vtkIOCore-8.90`. This is the default. The script will try to find this default library and give you some feedback (be careful: if it doesn't find the library, it will still continue as you might want to change the pathes later on). If your library naming convention is different—we've seen systems dropping the version numbers or Paraview installations which append something alike `pv8.90`—then specify your suffix manually through `--with-vtk-suffix`. If your installation does not have a version suffix, as is the case in Fedora, you should pass an empty string to this: `--with-vtk-suffix=''`

[1] Recent Ubuntu versions for example tend to place VTK in subdirectories of `/usr/local/include`, while SUSE et al indeed place them directly within `/usr/include`. In any case, you will have to select a subdirectory from these folders, as VTK usually goes into separate subfolder (depending on their version). If you have not installed VTK as stand-alone but as part of the Paraview developer version, e.g., then you have to search for the VTK directory within the Paraview folder. In any case, search for a filte `vtkVersion.h`. It is the folder hosting this file that we are searching for here.

- If your compile passes through but fails in the linking state (object not found), then you have to check if the VTK libraries are in the search path. If they are not (very likely), then add them prior to the compile call:

```
export LDFLAGS="-L/opt/vtk/lib64"
```

Once you have managed to configure with VTK and the build process has terminated successfully, your directory `src/convert` holds the actual `convert` script. If you call `convert` without arguments, you get a usage message. `convert` allows you to extract data from a data dump, and store it within the patch files under a new identifier. It also allows you to extract a dataset from patch files into vtu. vtu is one of VTK's binary file formats.

A standard postprocessing workflow on the command line reads as follows:

1. Use `convert` to extract certain data from the dumped data files. You might want to extract only the finest grid level, e.g., or to pick a particular unknown. `convert` is given a new identifier (name) for the new tdata set, and it stores the extracted data either within the existing patch files or a new, separate one.
2. Use `convert` to pick one particular dataset via its identifier and to dump it into a vtu file.
3. Use Paraview or VisIt to display the vtu file.



Unfortunately, our C++ conversion kernels do not offer all the functionality we now find within the Python-based tools. Things alike mesh distortion are missing here.

A.4 Writing Legacy VTK files

Here we describe a method of translating native Peano patch files into Legacy VTK files that can be visualised within Paraview. The advantage of this method of translating patch files is that it does not require an installation of Paraview/pvpython (as is the case in section A.3) and therefore might be of particular use to those who work with very large datasets on clusters that do not have Paraview installed.

To convert a single patch file into Legacy VTK format issue the following commands:

```
cd Peano
export PYTHONPATH=python
python3 python/peano4/visualisation/legacy_vtk.py --input-file=<path-to-peano-patch-file>
    --n-dims=<num-dimensions> --output-dir=<path-to-output-dir>
```

Here we translate the patch file (.peano-patch-file) passed to –input-file. The number of dimensions (either 2 or 3) must be specified by the argument –n-dim. Finally, the –output-dir is an optional argument that allows users to specify a directory in which they want the generated Legacy VTK files to be saved. The default output directory is named vtk_files and will be created in the current working directory.

Further, legacy_vtk.py can translate a whole set of patch files simultaneously if the –input-file argument is swapped for the –meta-file argument. Users should set the –meta-file argument to the main file that links to a set of patch files. The algorithm will then translate all files that are listed in the main file and place them all in the output directory.

Naming conventions for Legacy VTK files are as follows: a patch file named 'solution-SWE-tree-0-94.peano-patch-file' will be mapped to 'vtk_file_solution-SWE-tree-0-94.vtk'. That is, the

patch file's name will be prepended with the string 'vtk_file_' and its file handle will be changed from '.peano-patch-file' to '.vtk'.



If you work with partial patch files - i.e. cases where patches that belong to the same mesh are saved in multiple patch files - you need to merge these files before translating them into Legacy VTK. This can be done by calling merge_partial_patch_files.py:

```
python3 python/peano4/visualisation/merge_partial_patch_files.py
    --meta-file=<path-to-meta-file>
```

This script will merge partial patch files and record the newly created files in the same directory as the existing metafile. It will also generate a new metafile for the newly merged patch files. This will also be saved in the same directory as the existing metafile and is named merged_meta_file.peano-patch-file. Be sure to pass this new metafile to legacy_vtk.py.

A.5 Probing data

Probe.py can be used to read out the unknowns at a given point within a Peano patch file. That is, it identifies the unknowns that are associated with an individual cell that is specified through an x, y and (optional) z coordinate. Crucially, the single cell is found at the finest level of granularity recorded in the peano patch file.

The algorithm does not rely on Paraview/pvpython.

To probe an individual patch file use the following commands:

```
cd Peano
export PYTHONPATH=python
python3 python/peano4/visualisation/Probe.py --input-file=<path-to-peano-patch-file>
    -x=<x-axis> -y=<y-axix> --set-identifier=<set-identifier>
```

Here we probe the patch file passed to –input-file. The point to be probed is given as a coordinate to -x and -y. If you work with three dimensional data add a -z argument with the value of the z axis to be probed. The unkowns at the point (x, y) will be saved in a csv file called probe_output.csv, though the name of this file can be configured by adding the –ofname flag to the above command and setting it to the name of the output file.

We can also probe whole sets of patch files at once by swapping the –input-file flag used above with the –meta-file flag instead. The –meta-file should be set as the path to the main file that links to a set of Peano patch files. The algorithm will then probe all files that are listed in the main file.

In this case, each row in the outputted csv file will list the unknowns from an individual patch file. The rows are ordered chronologically. This csv, then, will capture the change in unknowns at a given point within the mesh over time.

A.6 Tweaking your pictures

Peano's default VTK plotters project Peano's tree-based grid onto an unstructured mesh, as VTK does not support tree meshes. The underlying mechanism is straightforward:

1. If a vertex is touched/read the first time, it is dumped as vertex of the unstructured output mesh if it is not refined, i.e. if no other vertex does exist at the same location on a finer mesh level.
2. If a hanging vertex is created, we add a new vertex to the output dump if this hanging vertex has not been written before. The plotters typically maintain one big hash map to bookkeep which hanging vertices have already been written.

3. If the tree traversal runs into an unrefined cell, this cell is made a cell of the unstructured output mesh.

The block format/the conversion tools in contrast generate all the points multiple times. If you have to cells sharing a vertex, the vertex is dumped twice. For patch-based problems, this is okish, as the number of vertices that are shared is way smaller than the total number of vertices in the domain (vertices within a patch are not dumped multiple times). As a consequence of the redundant vertex dumps, isosurfaces, e.g., might not work. If you need them, it is reasonable to project/map the actual data onto a regular mesh first.

If your PDE has m quantities, Peano's block format dumps them into a vector of cardinality m per vertex or cell, respectively. Most visualisation filters/tools however require data to be either scalar or vector data. In Paraview, use the calculator filter to extract only some quantities from the data dump. Alternatively, you can ask the convert tool to extract only some data into the vtu output.

Please note that most visualisation codes (such as Paraview) do interpolate bi-/tri-linearly within the cells. If a cell's face is adjacent to cells on finer levels, hanging vertices on the face exist. If they do not hold the linearly interpolated value, you will discover visualisation artifacts.



For smooth output pictures of meshes with adaptivity, you have to

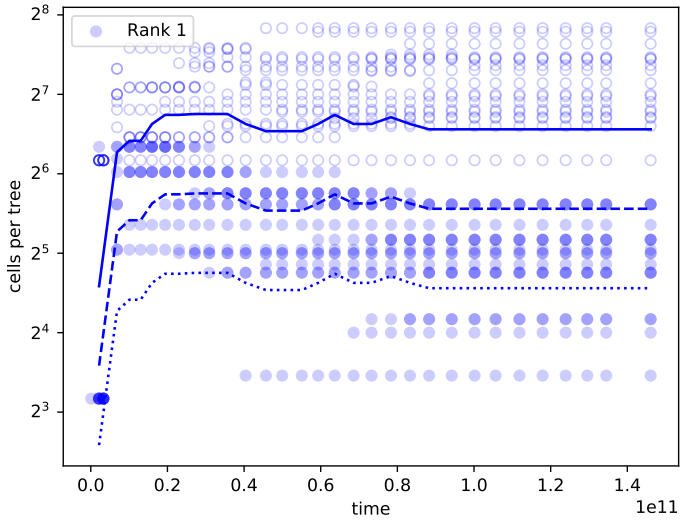
- set the plotted properties on hanging vertices due to d-linear interpolation, and
- you have to add the plotter mapping after you have invoked the mapping that does initialise the hanging vertices.

Please note that the dumped unstructured mesh is a non-conformal mesh. Algorithms such as isosurface identification thus might yield invalid results—it depends on your visualisation algorithms.

A.7 Useful scripts

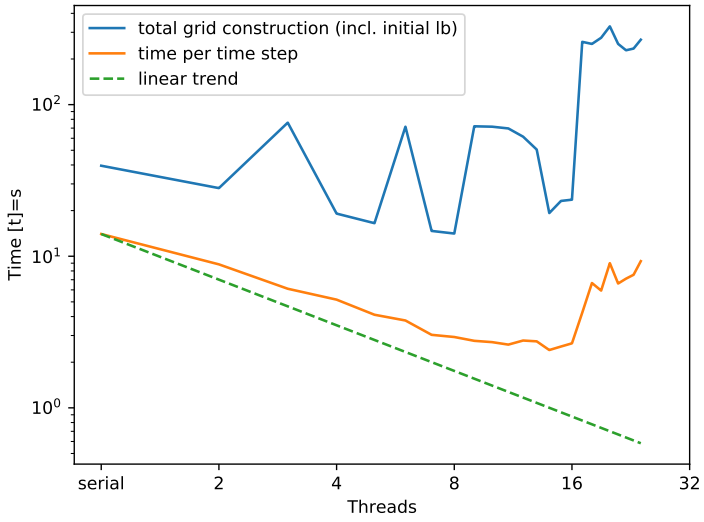
For many of Peano 4's features, I offer Python/matplotlib postprocessing scripts to translate the terminal output into graphs. These scripts can be a reasonable starting point for more sophisticated graphs (in papers, e.g.).

Load balancing toolbox. In the `loadbalancing` toolbox, you find a script `plot-load-distribution.py` which yields plots over the data distribution.



The script works for shared and distributed memory and gives you a scatter plot over the load distribution. Above, you see an (outdated) screenshot of the type of plots—the up-to-date version should have a legend that is easier to digest.

ExaHyPE 2 postprocessing. ExaHyPE 2 has a tailored postprocessing module in its Python API. It yields graphs alike the one below. The likely most interesting feature is that it works on archives of data dumps and thus keeps the disk space usage (and disk accesses) down.



There's a couple of parameters to pass to this script to control what exactly is presented:

- Are you interested in both the grid construction and the solution time?
- Do you want the script to take the whole simulation time into account or only the last time step?
- ...

Appendix B

Peano's block-structured output format

Peano has introduced its own block-structured output format. Its idea is that a mesh is dumped as an unstructured set of cell where each cell is uniquely identified through its offset and its size. The mesh data format lacks any topological information (connectivity). A cell always is a patch, i.e. holds a (topological) Cartesian mesh of dimensions $k \times k$ or $k \times k \times k$, respectively. If you work without patches, $k = 2$. In this case, the block format obviously can yield a significant overhead.

B.1 A minimal ASCII example file

```
#
# Peano output file
# Version 0.1
#
format ascii
dimensions 3
patch-size 6 6 6

begin vertex-metadata "identifier_A"
  number-of-unknowns 58
  meta-data "This_is_some_fancy_meta_data"
end vertex-values

begin vertex-metadata "time"
  number-of-unknowns 1
  meta-data "This_is_some_fancy_meta_data"
end vertex-values

begin patch
  offset 0.0 0.0 0.0
  size 0.5 0.5 0.5
  begin vertex-values "identifier_A"
    18.0 123.0 ...
  end vertex-values
  begin vertex-values "time"
    1.0 1.0 ...
  end vertex-values
end patch

begin patch
  offset 0.5 0.0 0.0
  size 0.25 0.25 0.25
  ...
end patch
```

This spec file specifies a simple grid that

- The actual grid plotted consists of two patches (boxes discretised with a small regular grid).
- Both small grids consist of $6 \times 6 \times 6$ cells. They all have the same size, i.e. are equally spaced.
- Each vertex holds two sets of unknowns. The first set is called `identifier A` and each entry (per vertex) comprises 58 double unknowns. The second set of unknowns per vertex is called `time` and is a scalar quantity.
- The data per cube is ordered lexicographically, i.e. we run along the x-axis first, then along the y-axis, and finally along the z-axis.
- All data is held as SoA, i.e. we first give all the 58 unknowns of the first vertex within the cube, then the 58 unknowns of the right neighbour vertex, and so forth.
- All output data is ASCII.
- Patches may overlap in both space and time.

B.2 More sophiciated dumps

- The file format supports `cell-values`.
- Each `vertex-values` or `cell-values` section in the header of the file, i.e. not embedded into a patch, may have a section

```
begin mapping
  0.0 0.0 0.0
  0.1 0.1 0.1
  0.4 0.4 0.4
  ...
end mapping
```

  If no mapping is present, our code dumps a regular subgrid (patch) per `patch` region. If a mapping is present, the mapping has exactly $(6+1)\cdot(6+1)\cdot(6+1)$ entries in the example from the previous section. Each entry is a 3d coordinate relative to the unit cube and specifies how the topologially regular grid prescribed within a patch is to be mapped. You might for example plot points of the topologically regular grid with Gauß-Legendre spacing.

- The plotter by default always creates two types of files: The actual data files as described above and one meta file. The meta file solely links to the actual data:

```
#
# Peano patch file
# Version 0.1
#
format ASCII
begin dataset
  include "conserved-1-rank-0.peano-patch-file"
end dataset
begin dataset
  include "conserved-2-rank-0.peano-patch-file"
end dataset
```

  Per snapshot (typically time step), the plotter adds on `dataset` entry. Each data set holds one data file per active rank. Each rank writes its actual data into a separate file.

Appendix C

Selected HPC platforms and toolchains

C.1 Clang toolchain

Both NVC++ and Intel's OneAPI compilers build upon Clang. However, they modify the baseline compiler. Therefore, they have separated sections in this chapter (below). The present section focuses on the baseline Clang compiler suite:

```
./configure CC=clang CXX=clang++ CXXFLAGS="-O3 -std=c++20 -mtune=native -march=native" \
 --enable-exahype --enable-loadbalancing --enable-blockstructured
```

OpenMP

R On Ubuntu and many other systems, clang is not shipped automatically with OpenMP. Instead, you have to install the package `libomp-dev`.

```
./configure CC=clang CXX=clang++ \
 CXXFLAGS="-O3 -std=c++20 -mtune=native -march=native -fopenmp" \
 --enable-exahype --enable-loadbalancing --enable-blockstructured \
 --with-multithreading=omp
```

Should you require OpenMP offloading, i.e. GPU, support, you have to add the OpenMP target instructions:

```
./configure CC=clang CXX=clang++ \
 CXXFLAGS="-O3 -std=c++20 -mtune=native -march=native -fopenmp" -fopenmp-targets=nvptx64 \
 --enable-exahype --enable-loadbalancing --enable-blockstructured \
 --with-multithreading=omp
```

C.2 Intel OneAPI toolchain

R OneAPI is what we currently use on Hamilton 8, i.e. our AMD EPYC testbeds. However, the modules there are built against old versions of GCC which do not provide the C++ standard library that we need for Peano 4. Therefore, you have to manually load the newest GCC *after* you have loaded the OneAPI module.

Before we start, you either load a fitting OneAPI module, or you set the environment variables via Intel's script:

```
source /opt/intel/oneapi/setvars.sh
```

The `source` command is included within a proper module. The LLVM-based Intel compiler is straightforward to use:

```
./configure CC=icx CXX=icpx \
 CXXFLAGS="-Ofast -std=c++20 -xAVX -mtune=native -march=native" \
 --with-multithreading=omp --enable-exahype --enable-loadbalancing --enable-blockstructured
```

which sets the most aggressive generic optimisation.

OpenMP For OpenMP, we usually use the Intel runtime:

```
./configure CC=icx CXX=icpx \
 CXXFLAGS="-Ofast -std=c++20 -fiopenmp -xAVX -mtune=native -march=native" \
 LDFLAGS="-fiopenmp" \
 --with-multithreading=omp --enable-exahype --enable-loadbalancing --enable-blockstructured
```

There are many different ways to specify the GPU (backend), but we often use the spir64 fat binary approach:

```
./configure CC=icx CXX=icpx \
 CXXFLAGS="-Ofast -std=c++20 -fiopenmp -xAVX -mtune=native -march=native -fopenmp-targets=spir64" \
 LDFLAGS="-fiopenmp" \
 --with-multithreading=omp --enable-exahype --enable-loadbalancing --enable-blockstructured \
 --with-gpu=omp
```

Intel MPI Intel's `mpiicpc` wrapper serves both the legacy Intel compiler and the new LLVM version. We recommend to switch to the new build:

```
export I_MPI_CXX=icpx
```

before you invoke

```
./configure CC=icx CXX=icpx ... --with-mpi=mpiicpc
```

Obviously, you have to fill in the remaining flags that you need for your build. Notably, do not forget to add a multithreading backend, as Peano 4 does not particularly like sole MPI builds without multithreading.

Intel OneAPI toolchain (ITAC) For Intel's ITAC, we have to load the itac module and reconfigure. We found many supercomputers to deploy itac to modules of its own, i.e. it is not loaded by default.

```
./configure CXXFLAGS="-DLogService=ITACLogger \
 --with-toolchain=intel
```

switches the logging to ITAC and activates the Intel toolchain. Please note that it depends on your log filter file (Chapter 11) what data is piped into ITAC and which data isn't.

C.3 NVIDIA NVC++ toolchain

The NVIDIA toolchain requires us to make `CXX` and `CC` point to the C++ compiler, as both are passed the same arguments by configure, which are actually only understood by the C++ version.

```
export NVCPP=/opt/nvidia/hpc_sdk/Linux_x86_64/2022/compilers/bin/nvc++
./configure CXX=$NVCPP CC=$NVCPP \
 CXXFLAGS="-O3 -std=c++17" \
 --enable-exahype --enable-blockstructured --enable-loadbalancing
```

Some compute kernels are not available with the NVIDIA tools, as the compiler is pretty picky when it decides which temporary variables it might place on the call stack.

OpenMP and OpenMP offloading NVIDIA's compiler does not support all of OpenMP. Therefore, its OpenMP implementation is rather picky when it comes to copy constructors and other C++ features. Peano 4 has internal workarounds for all of these items, i.e. the build should in principle succeed yet the code might be slightly slower than the Intel or native Clang counterpart.

```
export NVCPP=/opt/nvidia/hpc_sdk/Linux_x86_64/2022/compilers/bin/nvc++
./configure CXX=$NVCPP CC=$NVCPP \
 CXXFLAGS="-O4 -std=c++17 -mp" \
 --with-multithreading=omp \
 --enable-exahype --enable-blockstructured --enable-loadbalancing
```

The OpenMP offloading requires us to specify both the GPU OpenMP target and a corresponding CUDA

```
export NVCPP=/opt/nvidia/hpc_sdk/Linux_x86_64/2022/compilers/bin/nvc++
./configure CXX=$NVCPP CC=$NVCPP \
 CXXFLAGS="-O4 -std=c++17 -mp=gpu -gpu=cc75" \
 --with-multithreading=omp --with-gpu=omp \
 --enable-exahype --enable-blockstructured --enable-loadbalancing
```

It is important to recognise that `--with-gpu=omp` activates the offloading from Peano 4's point of view. However, as long as the NVIDIA compiler is not told which device to use, it will give you

```
Found 0 devices
```

if you run the kernel benchmark from page 16.

C.4 Some particular supercomputer testbeds

C.4.1 Hamilton 8

One Hamilton 8 (AMD EPYC), we prefer to use the OneAPI toolchain via the pre-installed modules. It is important to load an appropriate GCC backend, too.

```
module purge
module load oneapi
export FLAVOUR_NOCONFLICT=1
module load gcc

./configure CC=icx CXX=icpx ...
```

The `oneapi` is built against a GCC version which is too old and lacks important features within its standard library. Therefore, you have to manuall load the latest `gcc`. This is usually prevented

by the system and leads to a conflict (so that you can't have multiple GCC versions loaded). We therefore have to disable conflict checks a priori.

For MPI, I recommend to use IntelMPI:

```
module load intelmpi

export I_MPI_CXX=icpx
./configure CC=icx CXX=icpx ... --with-mpi=mpiicpc
```

C.4.2 DINE (Durham Intelligent NIC Environment)

Log into the first DINE node as by documentation (`ssh -X b101`) and run

```
module purge
module load python/3.6.5 gnu_comp/10.2.0 openmpi/4.0.5

./configure --enable-exahype --enable-loadbalancing --enable-blockstructured \
  --with-multithreading=omp --with-mpi=mpicxx \
  CXXFLAGS="--std=c++17 -fopenmp -march=native -O3" \
  LDFLAGS="-fopenmp"
```

For some applications (such as CCZ4), you will need additional packages (`module load gsl/2.5` for example). Old MPI implementations seem to struggle with single sided communication. In this case, you have to add `-DnoMPISupportsSingleSidedCommunication` to the `CXXFLAGS`. With the module combination from above, the single-sided communication flag seems not to be necessary anymore.

R The current MPI installation of DINE seems to face issues when we assign cores to MPI ranks: Often, only two cores are assigned to the first rank, while the other ranks get only one core. In this case, you have to tell `mpirun` explicitly that it has to use multiple cores per rank:

```
mpirun -n myNoOfRank -cpus-per-rank CoresPerRank ./peano4 ...
```

C.4.3 Cosma

On COSMA 7, I have successfully worked with the following toolchain:

```
module load oneAPI/2021.1.8
module load intel_comp/2020-update2
module load itac/2021.1-beta08
module unload python
module load python/intelpython3
export PYTHONPATH=../../../python

./configure CXXFLAGS=-I${VT_ROOT}/include --enable-exahype --with-mpi=mpiicpc --with-multithreading=omp --with-toolchain=int
```

With TBB, I had to fall back to the manual/minimalist tools:

```
module load intel_comp/2020-update2
module load intel_mpi/2020-update2
module unload python
module load python/intelpython3
export PYTHONPATH=../../../python

./configure LDFLAGS="-L${TBBROOT}/lib/intel64/gcc4.8 -ltbb_debug" --enable-exahype --with-mpi=mpiicpc --with-multithreading=
```

No success so far with the OneAPI environment:

```
module load oneAPI/2021.1.8-all
./configure --enable-exahype --with-mpi=mpiicpc --with-multithreading=tbb --with-toolchain=intel \
 CXXFLAGS="-I${VT_ROOT}/include -I/cosma/local/intel/oneAPI_2021.1.8/tbb/2021.1-beta08/include" \
 LDFLAGS="-L${VT_LIB_DIR} ${VT_ADD_LIBS}"
```

I have not been able to see any difference with the ucx module. It seems that we've well hidden most of the exchange.

Sollte nur oneapi sein, nciht das all

C.4.4 Bede

Access to Bede can be requested through SAFE (`https://safe.epcc.ed.ac.uk/`). The machine then is reachable through `ssh` with `ssh <username>@bede.dur.ac.uk`

Prepare the Python and bash environment You will need to run this commmand once to obtain a proper compile environment:

```
echo builder > ~/.application_environment
```

Then log out and log in again.

To install Python with all packages, run

```
python3 -m venv $HOME/peano-python-api
source $HOME/peano-python-api/bin/activate
pip install jinja2 sympy
```

Build with LLVM and MVAPICH LLVM is the recommended compiler for Peano 4 on Bede. OpenMPI initially has been more robust on Bede, but the installation struggles with single-sided MPI. We therefore prefer newer versions of mvapich:

```
module load nsight-systems cuda llvm
module load mvapich2/2.3.5-2
./configure --enable-exahype --enable-loadbalancing-toolbox \
 --with-multithreading=omp --with-mpi=mpicxx \
 CXX=clang++
```

The GPU support is added via

```
module load nsight-systems cuda llvm
module load mvapich2/2.3.5-2
./configure --enable-exahype --enable-loadbalancing-toolbox \
 --with-multithreading=omp --with-mpi=mpicxx --enable-gpu=nvidia \
 CXX=clang++ \
 CXXFLAGS="-fopenmp -fopenmp-targets=nvptx64-nvidia-cuda"
```

Build with LLVM and OpenMPI OpenMPI initially has been more robust on Bede, but the installation struggles with single-sided MPI. We therefore prefer newer versions of mvapich (see above). If you insist on OpenMPI, you have switch off the non-critical assertions:

```
module load nsight-systems cuda llvm
module load openmpi
./configure ... CXXFLAGS="..... -DnoMPISupportsSingleSidedCommunication"
```

Appendix D

Configuring Peano 4 in Eclipse

You can integrate Peano 4 straightforwardly into Eclipse.

D.1 Clone directly from git

Clone the repository locally and invoke

```
libtoolize; aclocal;
autoconf; autoheader;
cp src/config.h.in .;
automake --add-missing
```

If you have downloaded a distribution of Peano4, i.e. if you have not cloned the raw repository, you can omit these steps.

D.2 Integrating the C++ core as project

This step is required if and only if you want to develop core Peano 4 routines. If you have no intention to do this, you can simple run configure and make on your git clone and then skip this section. Otherwise, open the Eclipse GUI and run the following steps:

- Select `New Project`.
- Select a C++ project. Older Eclipse versions want you to pick the mode `Managed Build`.
- Locate the project at a remote directory (where you git deposits the clones).
- Select `GNU Autotools`. The Empty Project is the right one to select.
- You need only one target configuration. By default, Eclipse offers you to build Build and Debug. Disable the debug variant.

Some Eclipse wizards tend to overwrite Peano's `configure.ac` file. If this is the case, replace the files through

```
git restore configure.ac
git restore Makefile.am
git restore src/Makefile.am
```

and clean the project in Eclipse.

Part VIII

Quick help

Appendix E

FAQs (programmers' point of view)

E.1 C++ core

- **Where is the information whether a point or face is at the boundary or not?** I do not provide any such feature. Many codes for example use the grid to represent complicated domains, and thus would need such a feature anyway. So what you have to do is to add a bool to each vertex/face and set this boolean yourself.
- To be continued ...

E.2 Parallelisation

- **Why do I (temporarily) get an adaptive grid even though I specify a regular one?** This phenomenon arises if your adaptive refinements and (dynamic) load balancing happen at the same time. Load balancing is realised by transfering a whole tree part (subpartition incl. coarser scales) to another rank after a grid sweep. Refinement happens in multiple stages: After a grid traversal, the rank takes all the refinement instructions and then realises them throughout the subsequent grid sweep. So if a rank gets a refinement command and then gives away parts of its mesh, then it might not be able to realise this refinement. At the same time, the rank accepting the new partition is not aware of the refinement requests yet. So it might get the refinement request, but with one step delay: It sets up the local partition, evaluates the refinement criterion, is informed about a refinement wish for this area, and subsequently realises it. For most codes, that delay by one iteration is not a problem as the newly established rank will just realise the refinement one grid sweep later, but the point is: refine and erase commands in Peano 4 are always a wishlist to the kernel. The kernel can decide to ignore it—at least for one grid sweep.
- **Why do I get imbalanced partitions even though I tried to tell my load balancer of choice to balance brilliantly?** Peano 4 is based upon three-partitioning, i.e. whenever it refines a cell it yields 3^d new cells. Most users scale up in multiples of two. As 3^d cannot be divided into two equally-sized chunks, we always have some imbalance.
- **Why do some load balancing schemes report that they have to extrapolate data?** A typical message you get is "global number of cells lags behind local one. ..."' Most of my toolbox load balancing relies on asynchronous (non-blocking) collectives to gather information about the total grid structure. As a consequence, this information always might lag behind the actual status quo (load balancing info is sent out while the mesh refines,

e.g.). Some load balancing schemes can detect such inconsistencies—they only arise due to AMR and typically are sorted out an iteration later anyway—and apply heuristics. If they detect that the global cell number seems to be too small, e.g., they assume that every rank meanwhile has refined once more and multiply the global cell count with 3^d.

- **I get a timeout but Peano 4 claims that there are still messages in the MPI queue. How do I know what certain MPI messages tags mean?** All tags used in Peano 4 are registered through the operation `tarch::mpi::Rank::reserveFreeTag`. If you translate with any positive value of `PeanoDebug` (the trace, debug and assert library variants of Peano 4 do so, e.g.) then you get a list of tag semantics at startup. These lists are the same as the ones you use in release mode, i.e. when this information is not dumped. Please note that the info goes straight to cout, i.e. you can't filter it. That's because the tag allocation usually happens before our logging landscape is up and configured properly.

- To be continued ...

Appendix F

Troubleshooting

R Most developers exchange stuff via the Peano 4 slack. Contact Tobias Weinzierl to get an invitation to the slack and post your questions there.

F.1 Python API

- `import peano4` **yields not found error**. You haven't set your `PYTHONPATH` properly. Ensure that it points to Peano 4's `python` subdirectory. So use something similar to

```
export PYTHONPATH=~/git/Peano/python
```

- **/usr/bin/ld: cannot find -lExaHyPE2Core3d_trace**. Your Python script has not been able to locate the Peano 4 libraries. You can either tweak your systems parameters, i.e. set the search pathes, or you can manually add the library:

```
peano4_project.output.makefile.add_library(
  project.getLibrary(peano4.output.CompileMode.Debug),
  "mypath/src/exahype2"
```

- To be continued ...

F.2 Compilation

- **My compiler reports something like**

```
tarch/multicore/multicore.cpp:13:19: error: `align_val_t` is not a member of `std`
   13 | return new (std::align_val_t(Alignment)) double[size];
```

  We have seen this error with GCC if the supported C++ generation is too old. Try to return configure with `CXXFLAGS=-std=c++0x`. Depending on your compiler, the flag might also be `CXXFLAGS=--std=c++0x` (two hyphens).

- **My compiler yields errors like**

```
`warning \#3191: scoped enumeration types are a C++11 feature`
```

  Your compiler seems not to have C++11 (or, newer) enabled by default. Rerun configure with the flag `CXXFLAGS=-std=c++0x`.

- **My compiler terminates with**

```
`error: unknown attribute 'optimize' ignored*`
```

  We have seen this issue notably on macOS with CLANG replacing GCC. Unfortunately, CLANG seems to pretend to be GNU on some systems and then the wrong header is included. Ensure that `CompilerSpecificSettings.h`[1] includes a file such that the compiler `CompilerCLANG` is defined.

- **My compiler requires ages to translate the unit tests**. Ensure that the flag `UseTestSpecificCompilerSettings` is enabled. This effectively disables any optimisation for the unit tests. Without the flag, unit test can become really big due to template instantiation.

- `./configure` **crashes when I try to translate with TBB**. I have seen this on a couple of systems and found it annoying. First, ensure that configure find the TBB header. To ensure it does, check that `CXXFLAGS` are properly set. Next, set `LDFLAGS=-Lmypath`. Usually, I add `-ltbb` to this parameter and then everything is fine. However, some configures then don't like the configure script anymore. So I instead use `export PEANO_LDFLAGS="-ltbb"`. For some systems, it then still remains necessary to add pthreads, too.

- **My compiler terminates with errors when I enable OpenMP multithreading** . Make sure that you are using a compiler with OpenMP 5.0 support, such as GCC 9.x.

- To be continued ...

F.3 Linking

- **My linker yields errors like 'undefined reference to std::vector ... '**. We have seen this with some outdated GCC compilers. Ensure you meet the minimum GCC requirements or switch to another compiler.

- **The linker says that the vtk libraries were not found**. VTK can be tricky to get up and running. See a comprehensive discussion on VTK issues in Chapter A. It includes compiler/linker issues.

- **If I use Fortran as well, I get errors alike *undefined reference to for_stop_core' or 'for_write_seq_list'***. This tells you that the linker failed to find the Fortran standard libraries. For the GNU compilers, you need for example the flag `-lgfortran`, while the Intel compiler requires you to add ... To add these flags, set the `LDFLAGS` before you run `configure`:

```
# GNU
export LDFLAGS=-lgfortran

# Intel
export LDFLAGS=-lifcore
```

  On the Python API side, I considered adding these flags to the template makefile or offering an API function to add them manually to the makefile from the Python script. I dropped these ideas, as these libraries are specific to the build and hardware system and have nothing to do with a particular application.

[1] Peano relies on a header `tarch/compiler/CompilerSpecificSettings.h`. Whenever I find incompatibilities between different compilers, I try to resolve them through this file. The header reads out some compiler preprocessor directives and then includes the one it find most appropriate. You may always include your own file derived from one of the other headers in the directory.

- **The complains about some routines/classes from the technical architectures, but they are definitely there**. GCC/C++ is extremely sensitive when it comes to the order of the libraries used. You might have to edit your makefile manually to get a "valid" ordering that works. My recommendation is that you always links from the most abstract library to the lowest level lib. In one ExaHyPE 2 project, e.g., the following order worked whereas all others resulted in linker errors: `-lExaHyPE2Core2d_asserts`, `-lToolboxLoadBalancing2d_asserts`, `-lPeano4Core2d_asserts`, `-lTarch_asserts`.

- To be continued ...

F.4 Debugging

- **I can't see my log/trace statements.** There's a whole set of things that can go wrong.

  - Check that your log filter specifies the **class name**. In Peano 3, there was the opportunity to filter messages out on a routine level. For efficiency reasons, I had to remove this. So now you can only filter per class. Ensure you don't specify methods. If you do so, Peano 4 thinks your method name is a (sub)class name and won't apply your filter.
  - Ensure that your main code is translated with a reasonable debug flag. The makefile has to translate your code with `-DPeanoDebug=4` (or 1 if you are only interested in tracing).
  - Ensure you link to libraries which are built with a reasonable debug level. For this, run your code and search for the header

    ```
    build: 2d, no mpi, C++ threading, debug level=4
    ```

    on the terminal. Again, has to be 1 or 4 at least.
  - Ensure you have a whitelist entry for the routine you are interested in. Ensure that it is the right level (`TargetDebug` or `TargetTrace`, respectively):

    ```
    tarch::logging::LogFilter::getInstance().addFilterListEntry(
      tarch::logging::LogFilter::FilterListEntry(
        tarch::logging::LogFilter::FilterListEntry::TargetInfo,
        tarch::logging::LogFilter::FilterListEntry::AnyRank,
        "examples::algebraicmg",
        tarch::logging::LogFilter::FilterListEntry::WhiteListEntry
      )
    );
    ```

  - If you are still unsure which log filters are active, insert

    ```
    tarch::logging::LogFilter::getInstance().printFilterListToCout();
    ```

    into your code and see which entries it does enlist.

- **gdb says "not in executable format: file format not recognized"**. automake puts the actual executable into a `.libs` directory and creates bash scripts invoking those guys. Change into `.libs` and run gdb directly on the executable. Before you do so, ensure that `LD_LIBRARY_Path` points to the directory containing the libraries. Again, those guys are stored in a `.libs` subdirectory, so the library path should point to that subdirectory.

- **The interesting details are hidden in the debugger. It says something about inlined and doesn't give proper line numbers**. I recommend that you set `CXXFLAGS` to `-fno-inline` (Intel), rerun the `configure` script and recompile. This should stop the compiler from inlining too aggressive.

- To be continued ...

F.5 MPI troubleshooting

- **My compile fails with a message like the one below**:

```
[login1:35513] ***An error occurred in MPI_Win_create
[login1:35513] ***reported by process [629932033,0]
[login1:35513] ***on communicator MPI_COMM_WORLD
[login1:35513] ***MPI_ERR_WIN: invalid window
[login1:35513] ***MPI_ERRORS_ARE_FATAL (processes in this communicator will now abort,
[login1:35513] ***and potentially your MPI job)
```

  This is a know error with MPI installations that do not support one-sided MPI. Peano 4 uses one-sided MPI to raise/report (non-critical) exceptions accross MPI boundaries. Non-critical means that the code continues to finish the current compute step, dumps the solution (for debugging) and then terminates. It does not terminate immediately. If you disable one-sided MPI (pass `CXXFLAGS=-DnoMPISupportsSingleSidedCommunication` to your `configure` call), then such a non-critical assertion will also terminate immediately.

- **My code crashes unexpectedly with malloc/free errors**. Please rerun your code with Intel MPI and the flag `-check_mpi`. You should not get any error reports. If you do, we have had serious problems as some of Peano's classes use `MPI_C_BOOL` or `MPI_CXX_BOOL`. They seem not be supported properly.

- **With more and more ranks, I suddenly run into timeouts**. Peano 4 has a built-in timeout detection which you can alter by changing the timeout threshold. This is done via the `--timeout XXX` parameter. The other thing is that Peano 4 subdivides the partition such that each core (!) gets one SFC-fragment. For smallish problems, this can lead to a massive overhead, as the partitions are too small. In this case, you either have to restrict the threads per rank via `--threads XXX` (this is problematic, as it effectively reduces the code's efficiency) or you instruct your load balancing not to split up that often. See the documentation of your particular load balancing strategy.

- To be continued ...

F.6 OpenMP troubleshooting

- **My linker fails, as OpenMP or pthreads are not found**. We see this frequently with GNU compilers. Ensure that your `LDFLAGS` holds `-fopenmp` when you run `configure`.

- **My code uses only one core**. If you pass `--with-multithreading=omp` to `configure`, ensure that you add all flags that your compiler needs to `CXXFLAGS`, too. If you compile requires you to pass `-openmp`, e.g., to make it understand OpenMP pragmas, then you have to add `CXXFLAGS=-openmp` to your configure invocation. We've seen multiple times with GNU that the GNU compiler translates your codd if you use `--with-multithreading=omp`, but effectively ignores the OpenMP pragmas as you forgot to specify `CXXFLAGS=-openmp`. You end up with core libraries which do not support multithreading or even deadlock.

- **My code hangs**. Check how many threads OpenMP tries to book. You find this information at the startup where you typically obtain a message similar to

```
Peano 4 (C) www.peano-framework.org
build: 2d, no mpi, omp (8 threads), debug level=2
```

  The number of threads should not be bigger than the physical threads and I strongly recommend not to count hyperthreads as real threads. If you find that you have to reduce the thread count (as your system by default uses SMT), pass in `--threads X` to reduce the thread count to `X` manually. Your initial notification will continue to report all SMT threads,

but there will be a message immediately after that clarifies that you manually reduce the thread count. If you work in a SLURM/batch environment, changing the thread/core count in the batch script might be more convenient.

- To be continued ...

F.7 External tools

- **The Intel tools yield invalid or messed up results**. Ensure you have built your code with `-DTBB_USE_ASSERT -DTBB_USE_THREADING_TOOLS`. Please consult the supercomputer/-tool remarks on page **??**.
- To be continued ...

F.8 PDT (legacy)

This section is deprecated and is of relevance if and only if you use the Python API with the legacy DaStGen support to model application-specific data.

- **The PDT does not pass as the jar file is built with the wrong Java version**. Download the whole Peano project (from sourceforge via subversion), change into the directory `pdt/src`, and run

```
make clean
make createParser
make compile
make dist
```

  Use the `pdt.jar` from the `pdt` directory now.